

Parallel programming



Welcome!



Lessons: Theory: Thursday 14-16 (E8) – Lab: Friday 11-13 (B4)

The course



- **This part of the course will consist in two sections** held by the two lecturers
 - Manycore (GPU) computing (Francesco Leporati)
 - Lab computing (Marco Gazzoni)
- **Material onto kiro**
 - Slides of the lessons
 - Manuals of the programming environments
- **Exam (verification)**
 - Projects

Bibliography



General references:

- J. Cheng, M. Grossman, T. Mckercher, Professional Cuda C Programming, Wrox Pr Inc. (1. edizione), 2014 (cap 1)
- Jason Sanders, Edward Kandrot, CUDA by examples: an introduction to general-purpose GPU programming, Pearson Education, 2011
- <http://www.cse.unt.edu/~tarau/teaching/parpro/papers/Parallel%20computing.pdf>

NVIDIA docs:

- CUDA Programming: *<http://docs.nvidia.com/cuda/cuda-c-programming-guide/>*
- CUDA C Best Practices Guide: *<http://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>*

Structure of GPU computing part



- **Theory, Exercitations, Programming**
 - Introduction, GPU architectures, Cuda environment, ...
 - ...
- **Exercitations and Lab**
 - C language review
 - File access with dynamic memory allocation, Structures, Code partitioning
 - Image acquisition, processing, first parallelization
 - Filtering
 - Introduction to the final project



Why EXA?

EXA = 1,152,921,504,606,846,976 = 2^{60} = 1024^6 = $10^{18.06}$

PETA = 1,125,899,906,842,624 = 2^{50} = 1024^5 = $10^{15.05}$

TERA = 1,099,511,627,776 = 2^{40} = 1024^4 = $10^{12.04}$

GIGA = 1,073,741,824 = 2^{30} = 1024^3 = $10^{9.03}$

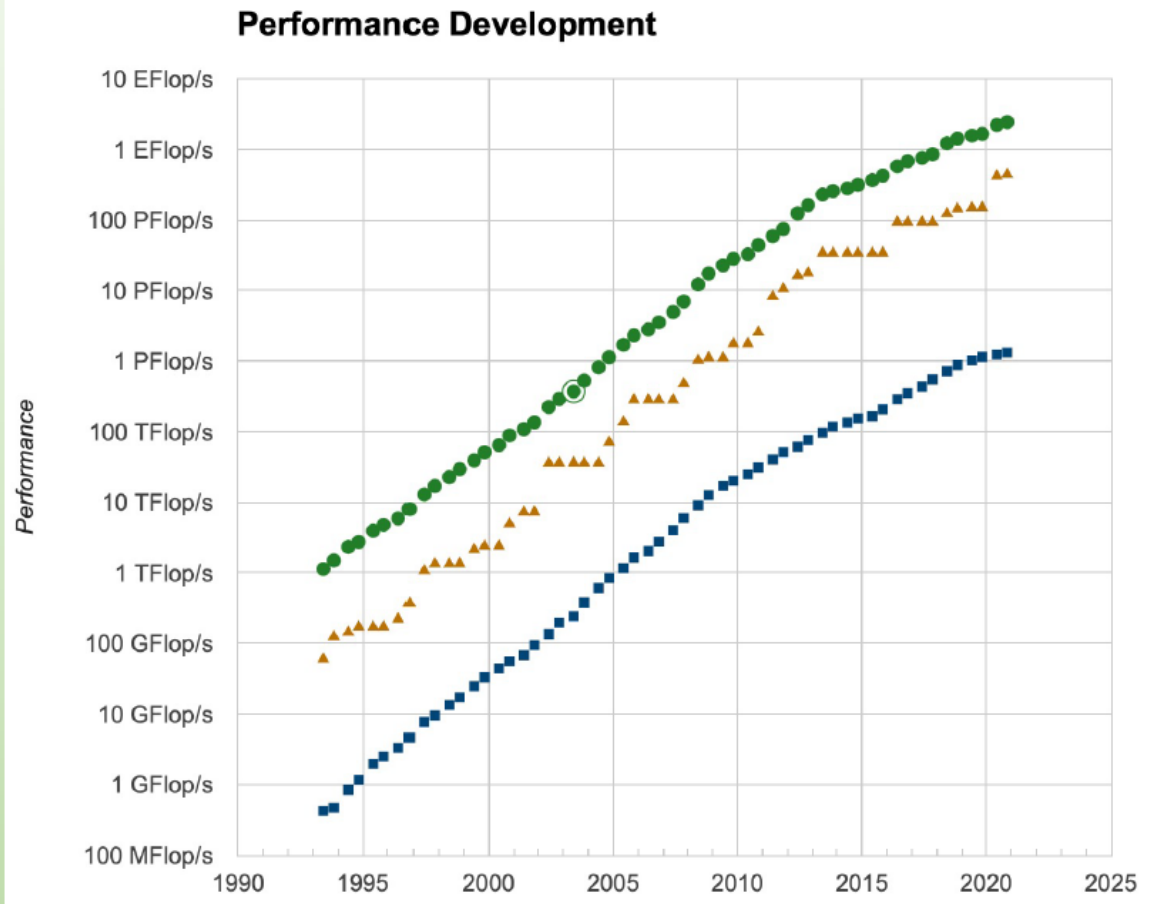
MEGA = 1,048,576 = 2^{20} = 1024^2 = $10^{6.02}$

KILO = 1,024 = 2^{10} = 1024^1 = $10^{3.01}$

Top 500 project (top500.org)



The **TOP500 project** was started in 1993 to provide a reliable basis for tracking and detecting trends in high-performance computing. Twice a year, a list of the sites operating the 500 most powerful computer systems is assembled and released. The best performance on the **Linpack benchmark** is used as performance measure for ranking the computer systems. The list contains a variety of information including the system specifications and its major application areas.



Top 500 project

Top supercomputers list



Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	Frontier - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE DOE/SC/Oak Ridge National Laboratory United States	8,699,904	1,194.00	1,679.82	22,703
2	Aurora - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	4,742,808	585.34	1,059.33	24,687
3	Eagle - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Microsoft Azure United States	1,123,200	561.20	846.84	
4	Supercomputer Fugaku - Supercomputer Fugaku, A64FX 48C 2.2GHz, Tofu interconnect D, Fujitsu RIKEN Center for Computational Science Japan	7,630,848	442.01	537.21	29,899
5	LUMI - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE EuroHPC/CSC Finland	2,752,704	379.70	531.51	7,107
6	Leonardo - BullSequana XH2000, Xeon Platinum 8358 32C 2.6GHz, NVIDIA A100 SXM4 64 GB, Quad-rail NVIDIA HDR100 Infiniband, EVIDEN EuroHPC/CINECA Italy	1,824,768	238.70	304.47	7,404
7	Summit - IBM Power System AC922, IBM POWER9 22C 3.07GHz, NVIDIA Volta GV100, Dual-rail Mellanox EDR Infiniband, IBM DOE/SC/Oak Ridge National Laboratory United States	2,414,592	148.60	200.79	10,096
8	MareNostrum 5 ACC - BullSequana XH3000, Xeon Platinum 8460Y+ 40C 2.3GHz, NVIDIA H100 64GB, Infiniband NDR200, EVIDEN EuroHPC/BSC Spain	680,960	138.20	265.57	2,560
9	Eos NVIDIA DGX SuperPOD - NVIDIA DGX H100, Xeon Platinum 8480C 56C 3.8GHz, NVIDIA H100, Infiniband NDR400, Nvidia NVIDIA Corporation United States	485,888	121.40	188.65	

Parallel computing: co-operation & concurrency



- ✓ **Parallel computing:** the use of multiple computers or processors working together on a common task
- Each processor works on its section of the problem
 - Processors are allowed to exchange information with other processors

Grid of a Problem
to be Solved

Process 0 does work for this region	Process 1 does work for this region
Process 2 does work for this region	Process 3 does work for this region

From traditional computing ...



Central Processing Unit

- Primary computing resource
- Enough powerful for general purpose application
- Moore law (4x / 3ys)
- Robust technology
- Weaknesses ...



From traditional computing ...



Critical issues

- Since the last 30 years the clock passed from 1 MHz to 4 GHz but cannot grow up indefinitely
- The dynamic power is proportional to f^3 -> problems of power dissipation

$P = \alpha C_L V^2 f$ with alpha switch probability, C_L capacity, V voltage ($\approx V^2$), f frequency

- Moore law in the recent years does not work well as before
- Multicore processors to increase computational capability ...
- ... but there is an intrinsic dimensional limit to frequency augmentation

If c (light velocity) = 300.000 Km/sec = $3 * 10^9$ dm/sec and $f = 4$ GHz

$$s = c * t = 3 * 0.25 = 0.75 \text{ dm}$$

To GPU computing ...



Graphics Processing Unit

- Nasce per la grafica!
- Molto più potente per “applicazioni grafiche” rispetto alla CPU
- Tecnologia in continua evoluzione



Aspetti di interesse

- Migliaia di core vs 2,4,8 della CPU
- Utile solo per problemi altamente parallelizzabili (velocità 10x, 100x ...)
- Introduce il paradigma GP-GPU (General Purpose – GPU)

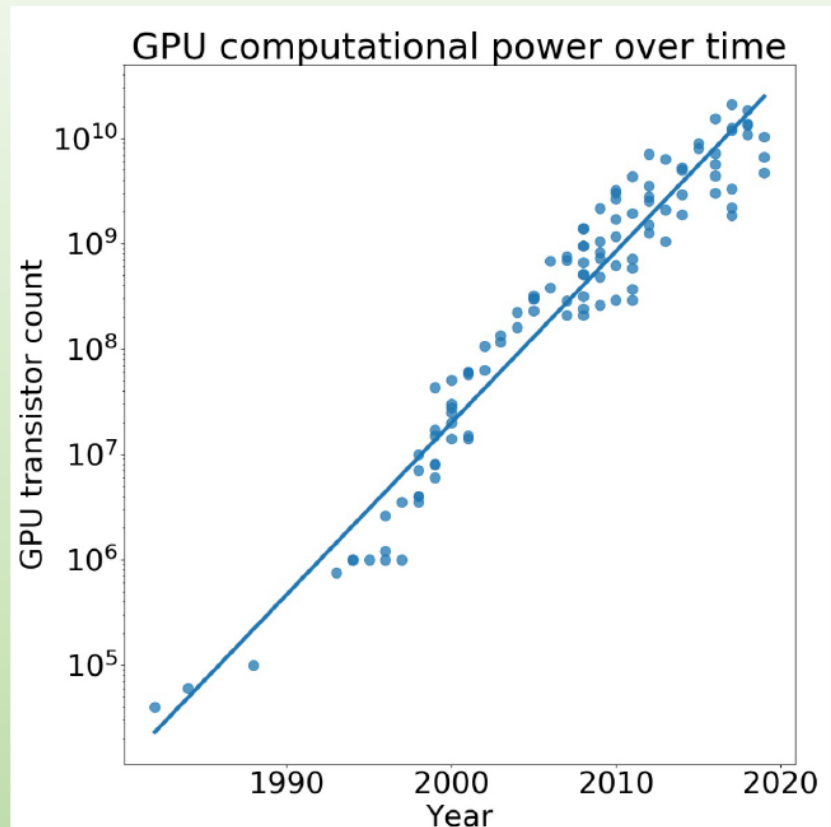
To GPU computing ...



Moore's Law

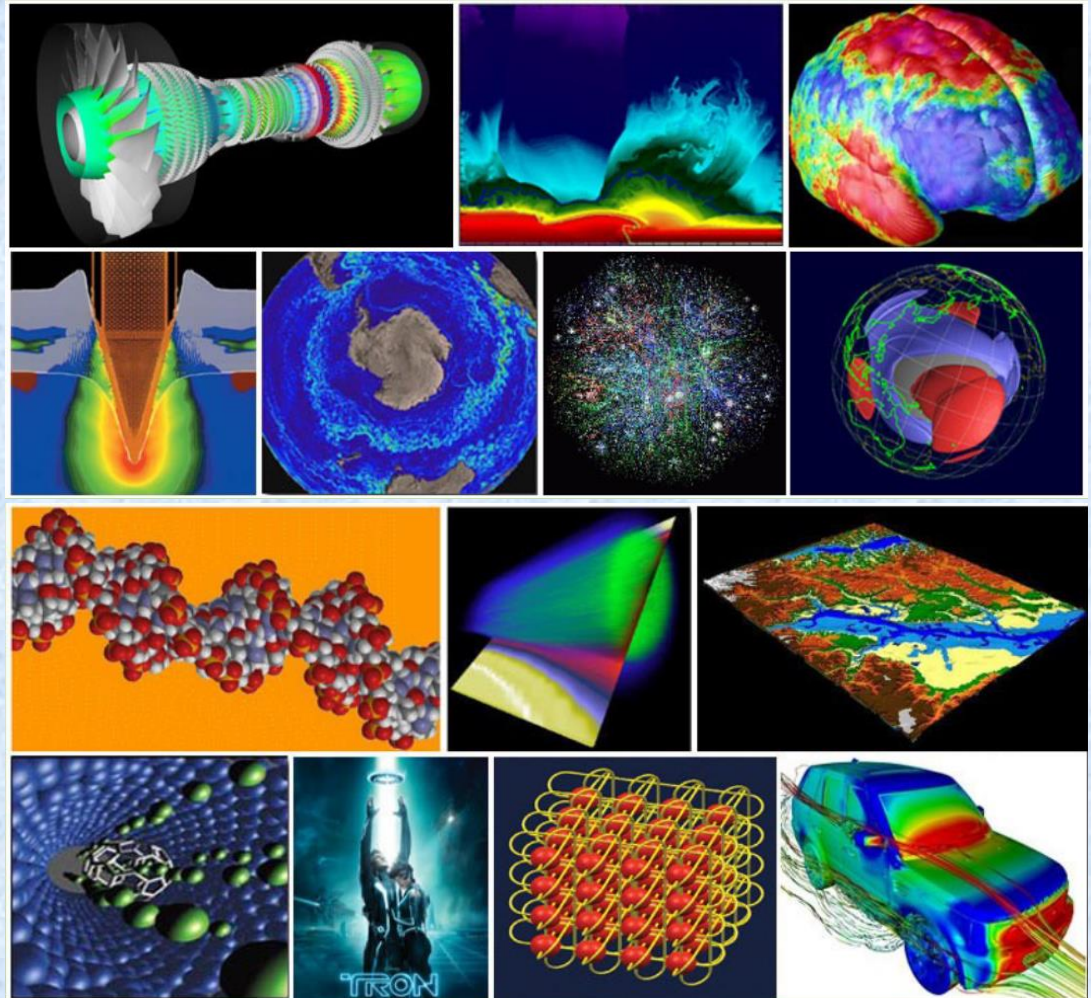
"The processor speeds, or overall processing power for computers will double about every 18 month's "

A logarithmic plot of GPU transistor count (proportional to computational power) versus time shows Moore's Law holds true for GPUs



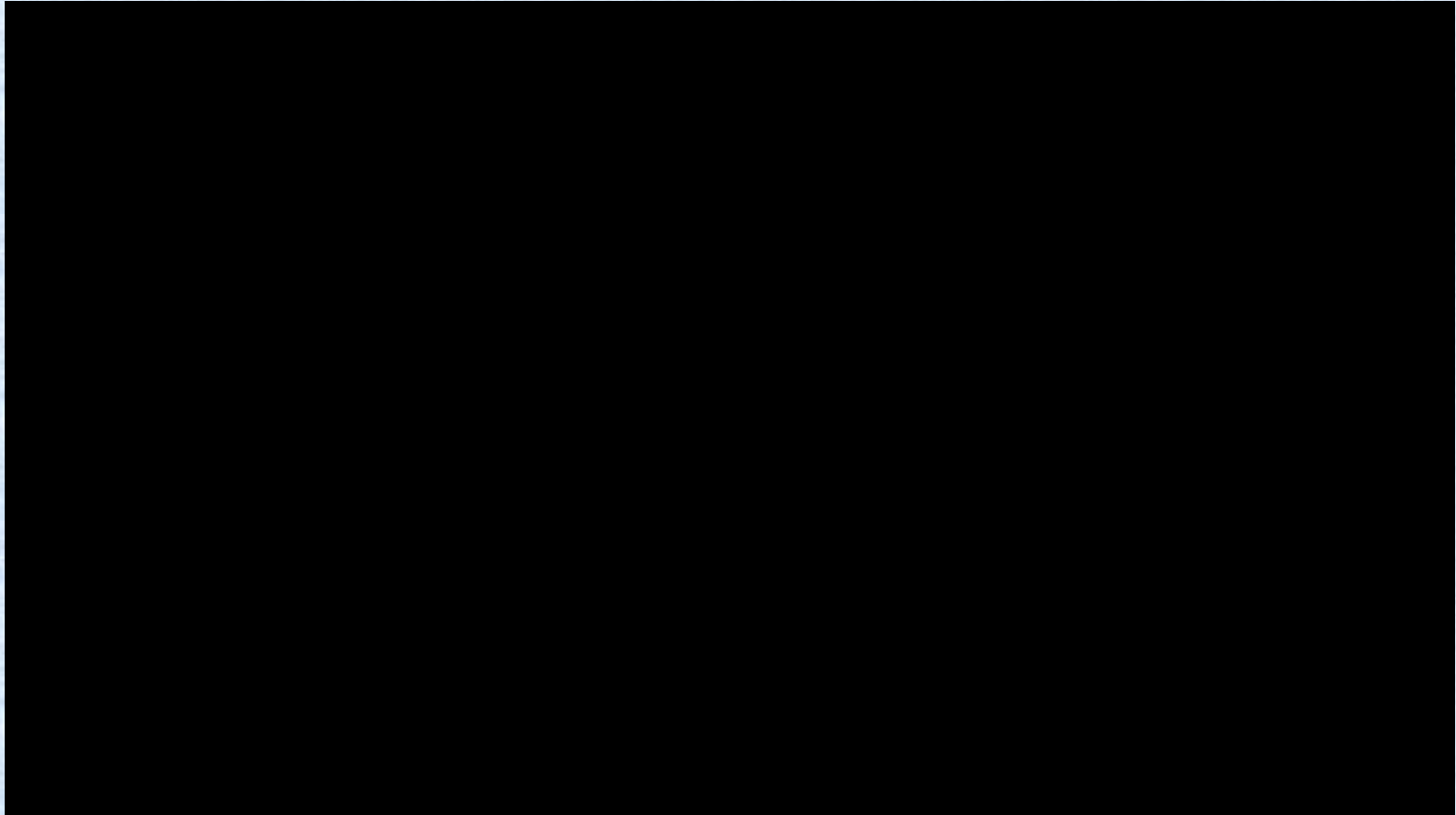


**Atmosphere, Earth,
Environment
Physics and Chemistry
Bioscience, Genetics,
Biotechnology
Geology and Seismology
Mechanics (prosthetics to
aircraft)
Electrical and Electronic
Engineering
Computer Science and
Mathematics
Defense
Cryptography and security
Financial models
...**



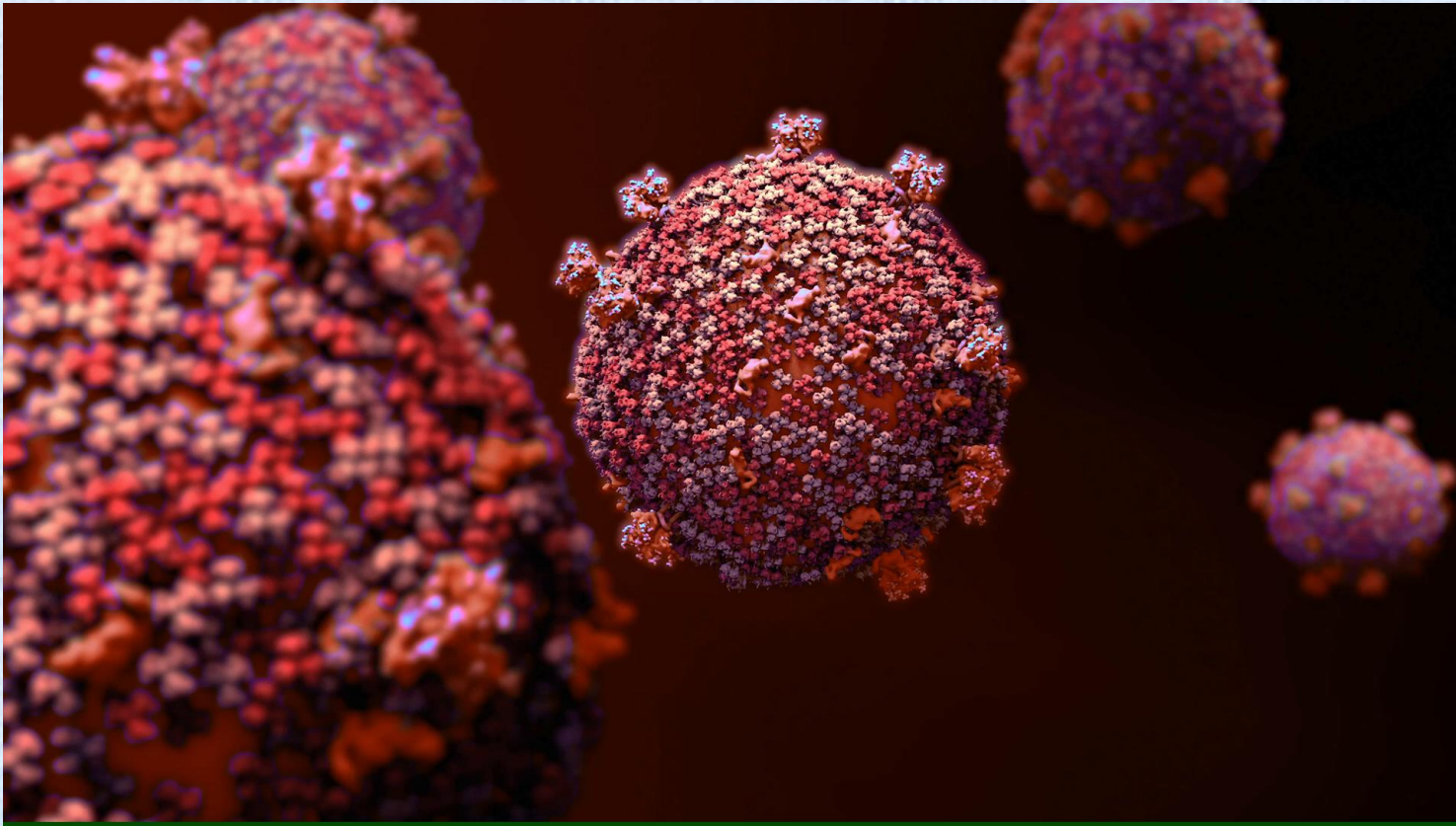


Accelerating Covid 19 research





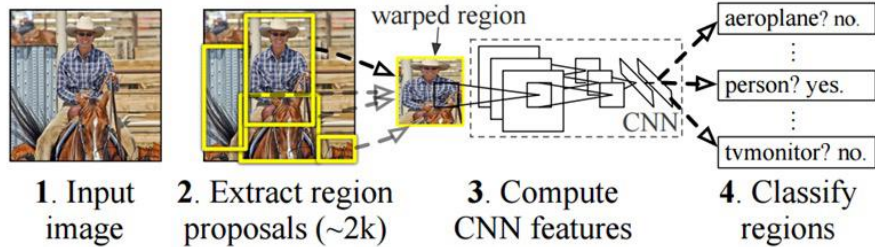
Federated learning for Med AI and Triaging



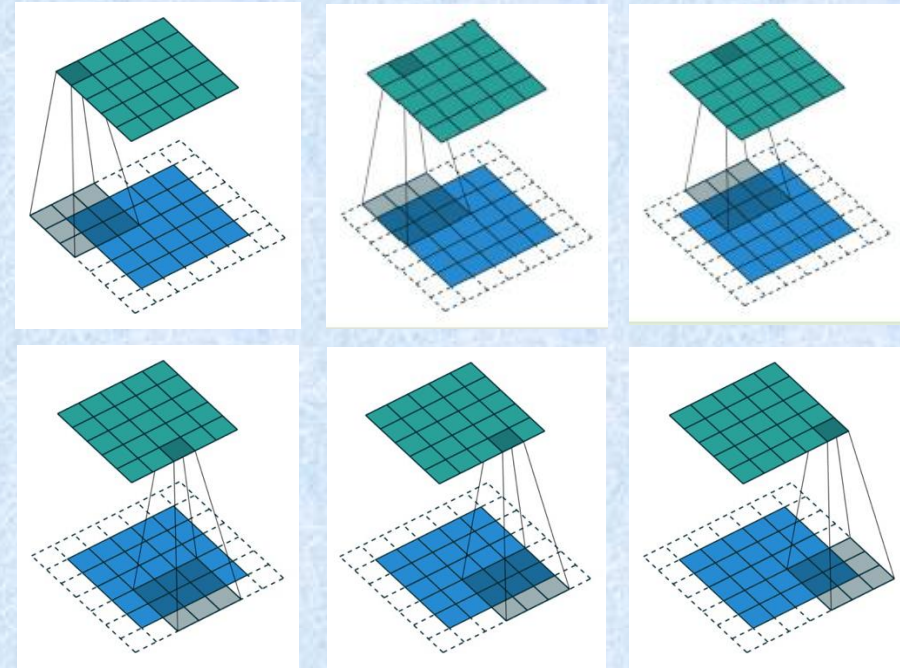
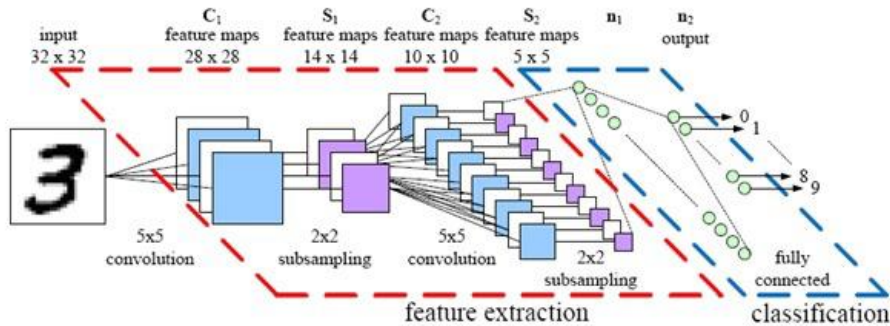


Suitability for GPU computing

R-CNN: Regions with CNN features

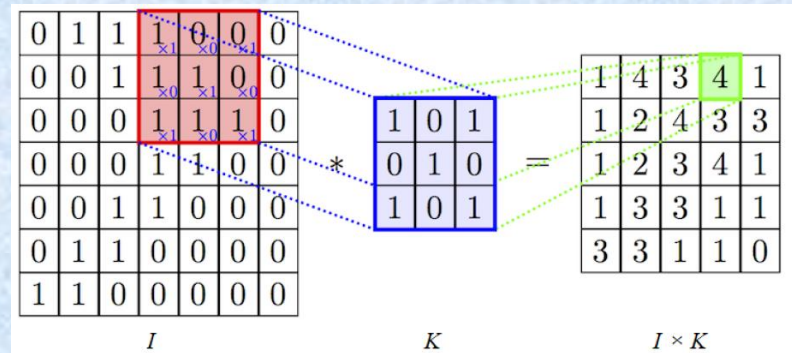


R-CNN workflow



Convolutional Neural Networks (CNN)

$$C(i,j) = \sum A(i,k) * B(k,j)$$



Suitability for GPU computing



Deep learning:

**Object detection
and segmentation**

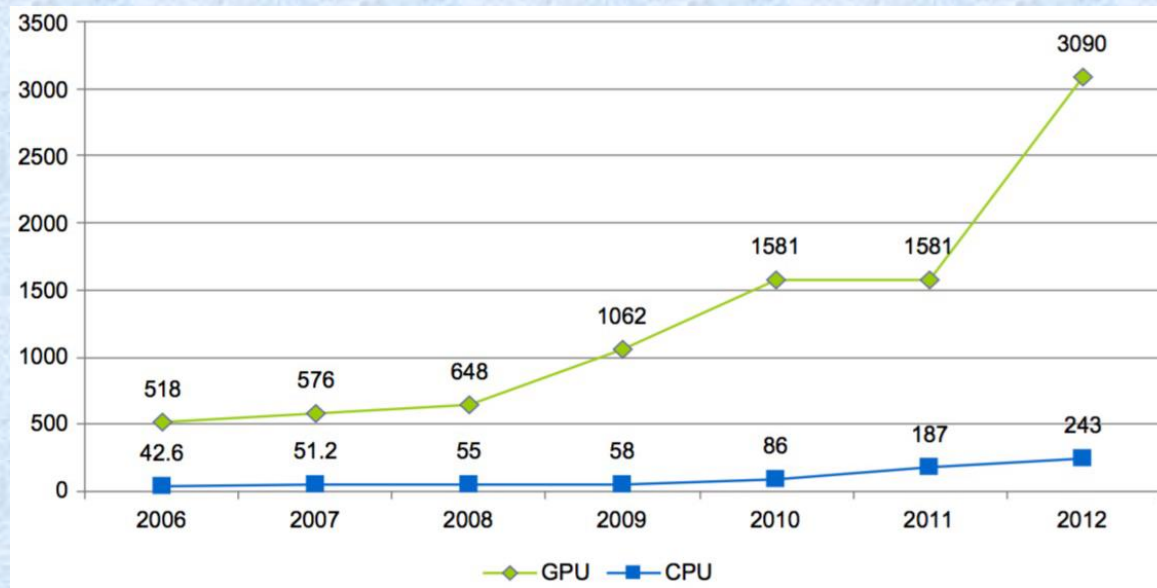
90-10 Rule

***The 90% of the time is spent in 10% of the code
i. e. repeatedly executed operations***

GPU in the past



- In 2007, NVIDIA introduces GPU in general purpose programming through a easy-to-use programming interface
- Since 2009 GPU overtakes significantly CPUs in terms of computing power (1 TFLOPS)
- CUDA (Compute Unified Device Architecture) General-purpose parallel computing platform for NVIDIA GPUs
- OpenCL (Open Computing Language) General heterogenous computing framework

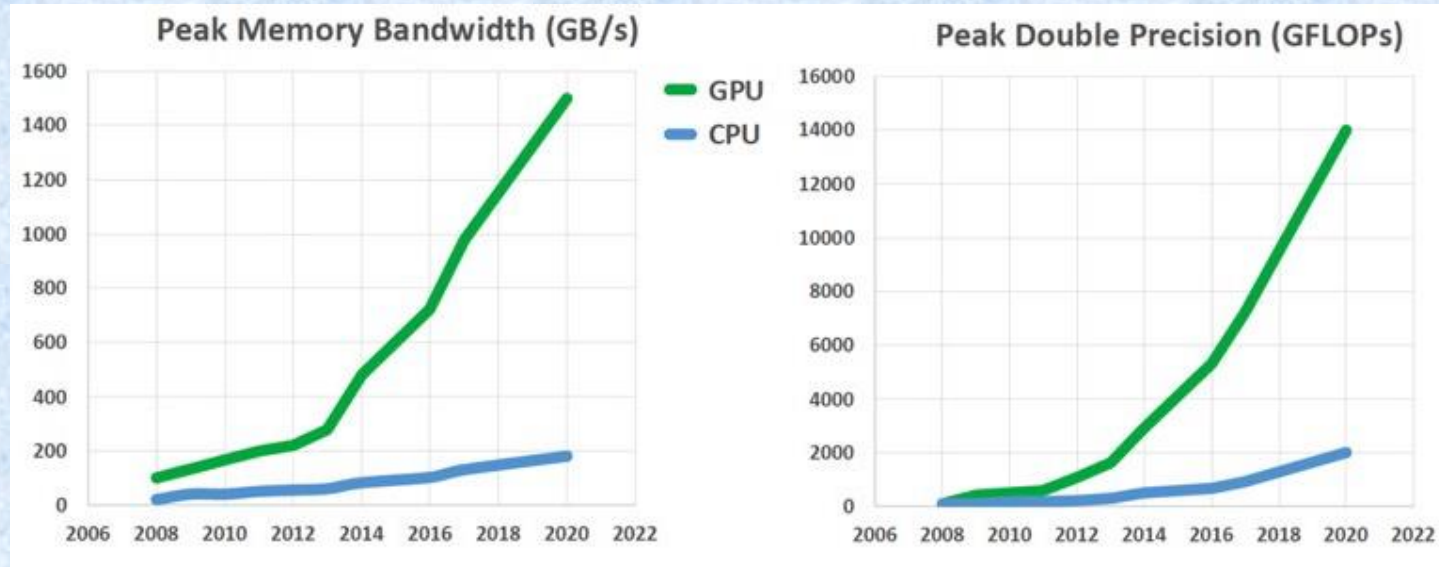


GPU now



- General-Purpose computing on GPUs (GP-GPU)
- Parallel computing
- Heterogeneous architectures
- Heterogeneous applications
- Big data

Volta V100 vs Intel Xeon SP





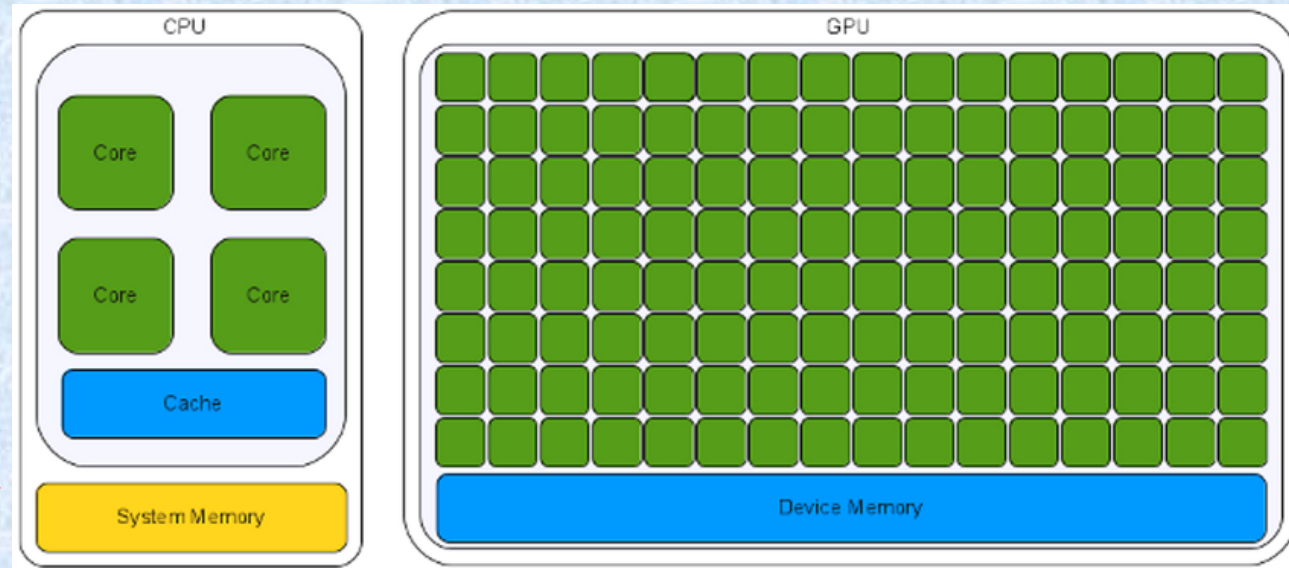
GP-GPU some concepts

- *General-purpose GPU* (GP-GPU) refers to the use of the GPU (graphics processing unit) for scientific, multimedia, engineering computations ... of a general nature
- *Heterogeneous model*: it involves the use of CPU and GPU together, creating a coprocessing calculation model
- *Separation*: the sequential part of the application runs on the CPU while the parallel part (the one with the highest computational intensity) is accelerated by the GPU
- *Transparency*: from the user point of view, the application runs faster taking overall advantage of the high performance of the GPU that plays the role of a transparent accelerator
- *Mapping*: it is possible to rewrite a function to “exploit” the parallelism of the GPU by adding the "C" annotations that move data and execute that function (kernel) on the GPU. Not efficient, not “pure CUDA”, not optimised, quick to do
- *Problem*: maximizing computing power by minimizing the consumed energy



Heterogenous architectures

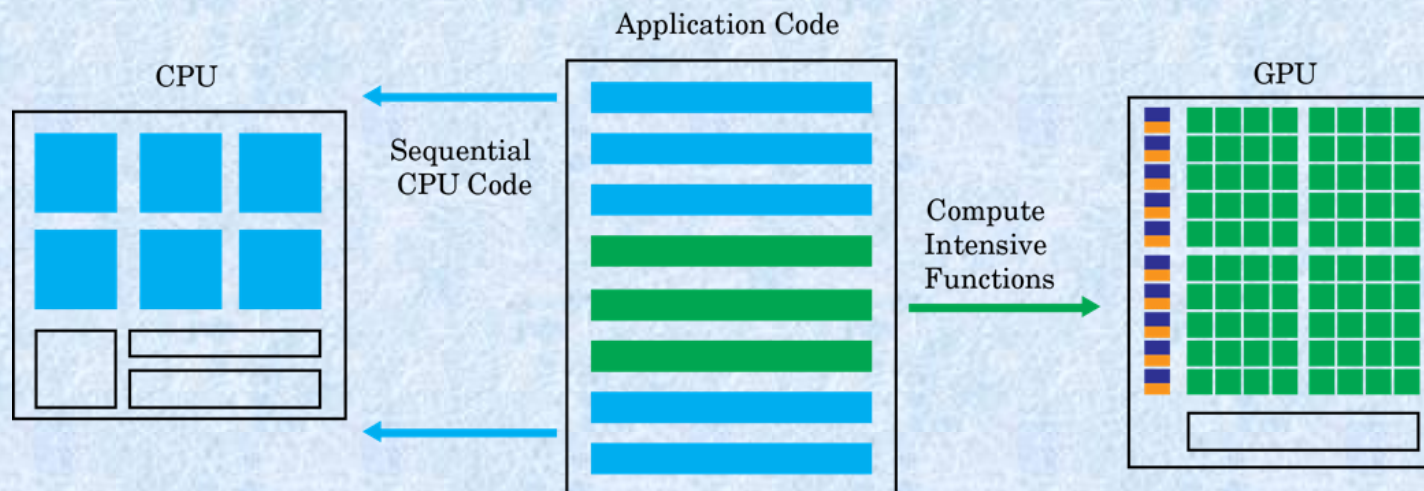
- The GPU is a coprocessor not a standalone unit (sometimes is called *accelerator*)
- CPU = *Host*, GPU = *Device*
- CPU and GPU communicate through a PCI express bus
- GPU needs direct transfers from memory *through CPU*
- CPU synchronises the processing
- CPU: few cores conceived for “serial” processing
- GPU: thousands of cores that efficiently elaborate concurrent tasks (*massively parallelism*)



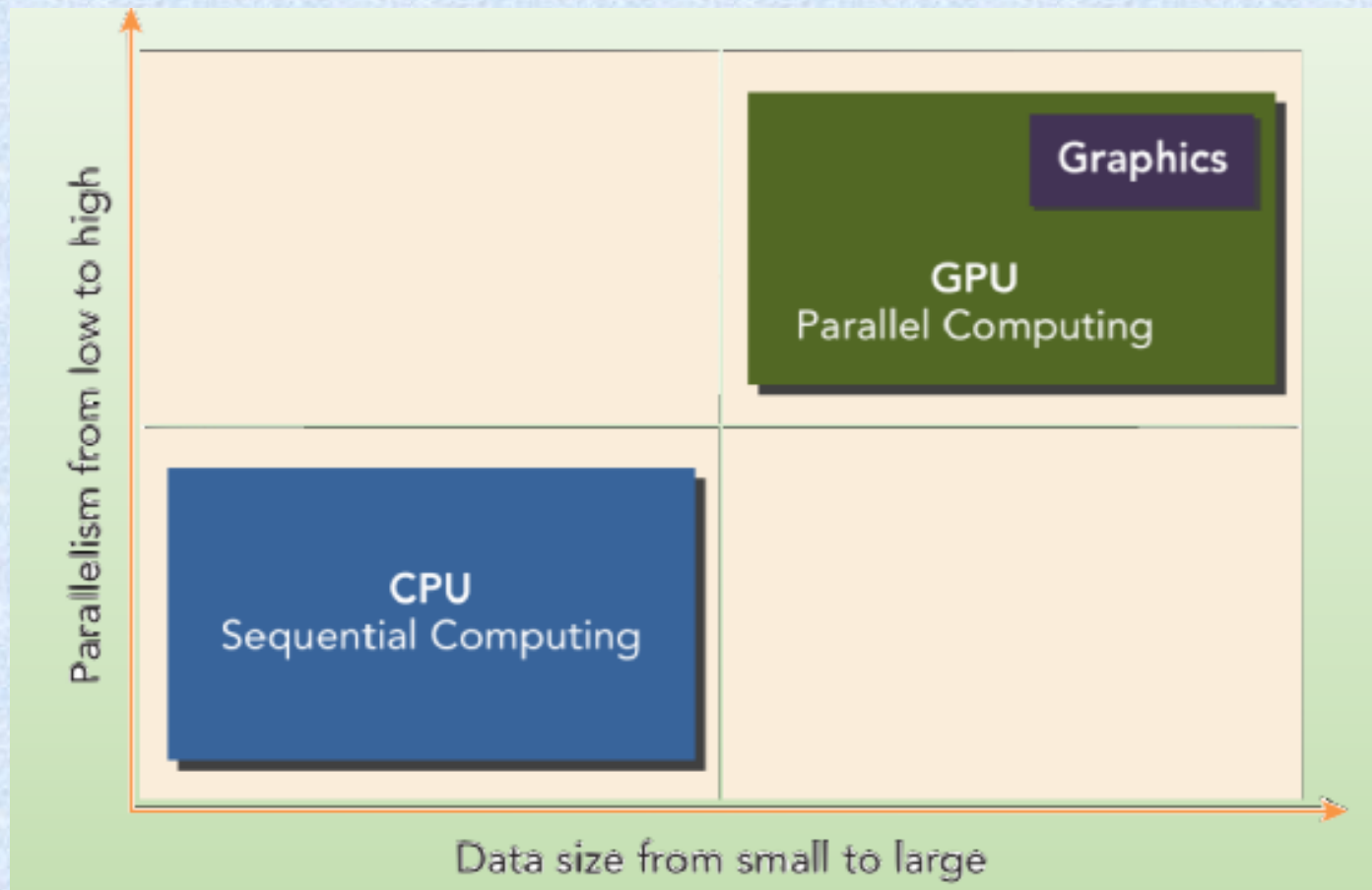


Heterogenous architectures

- Characterized by *Host code* (elaborated on the CPU) and *Device Code* (on the GPU)
- GPU used to accelerate *parallel code* on *parallel data* elaboration
- CPU code manages the local environment, local I/O and local data, then uploads the intensive tasks onto the *Device*
- CPU is optimised for operations flow that cannot be predicted (if, branches ...)
- GPU is designed for operations characterised by a simple control flow



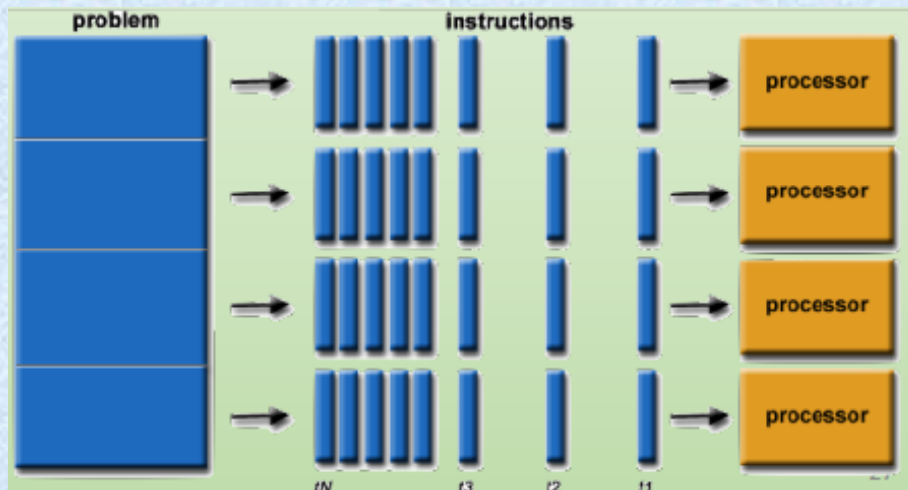
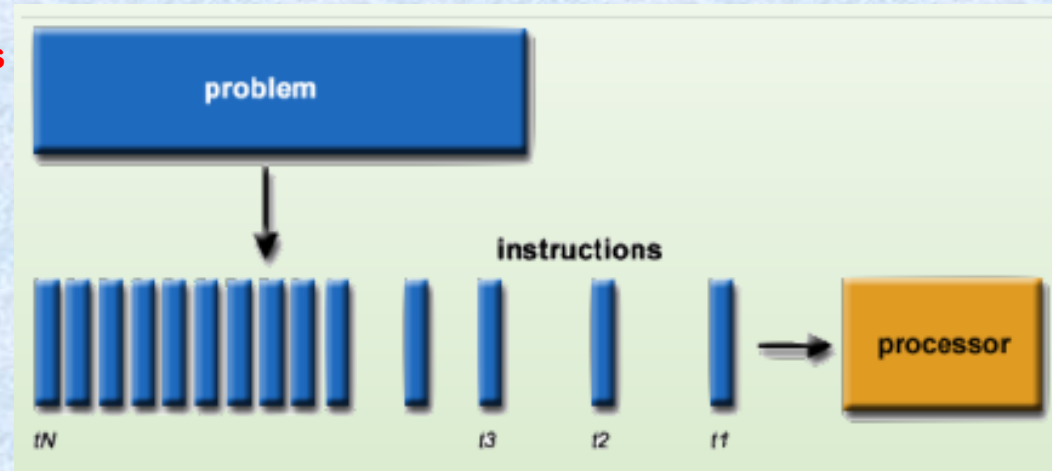
Data parallelism vs Big Data



Instruction level parallelism



- Problem as a finite sequence of instructions
- Serial execution
- One instruction every n cycles
- Single computing unit



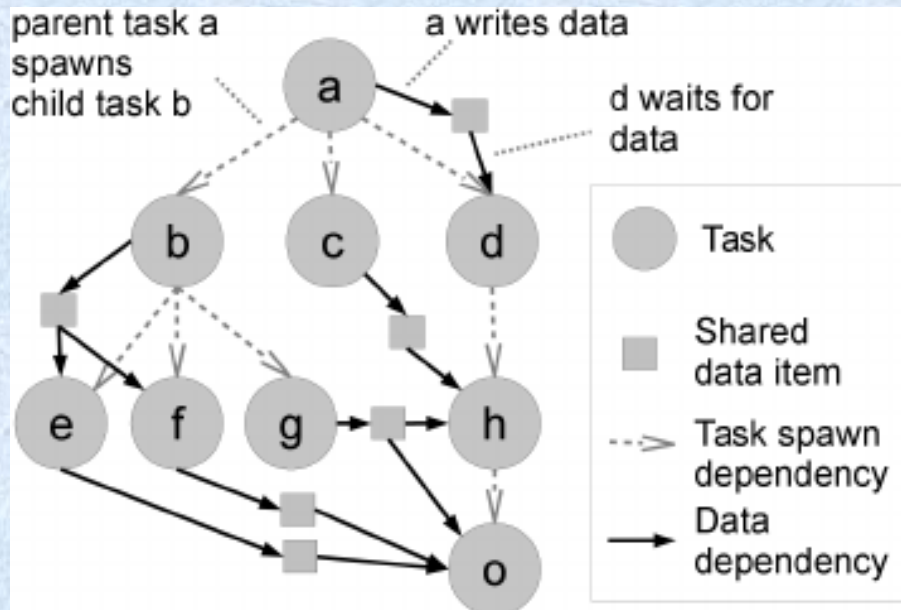
- Problem *partitioned* in concurrent sections
- Each section consists of single instructions
- Parallel processing on different processors
- Control/co-ordination mechanism to start the elaboration and collect the results

Parallel computing

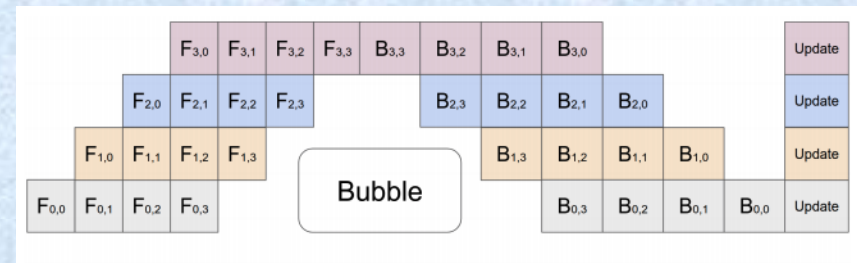
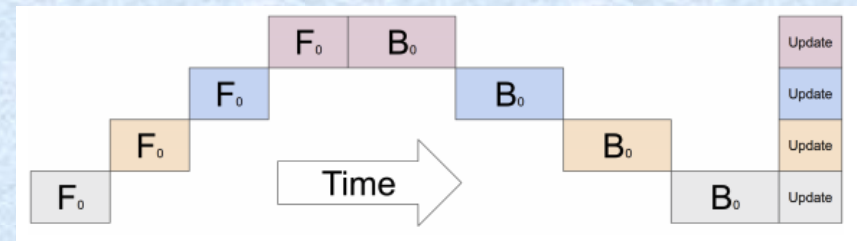


- **Goal**: to increase computation velocity
- **Benefit**: concurrent execution of problems partitioned in small tasks that can be solved simultaneously and independently one from the other (i. e. matrix row-column product)
- **Burden**: the programmer must directly manage calculations resources (core, memory I/O). Tradeoffs
- Two **kinds** of parallelism:
 - *Task level parallelism*: independent functions distributed among different cores
 - *Data level parallelism*: data can be elaborated concurrently since they are distributed among different *threads*
- Parallel computing consists of:
 - Parallel **architectures**
 - Parallel **programming**

Data dependency

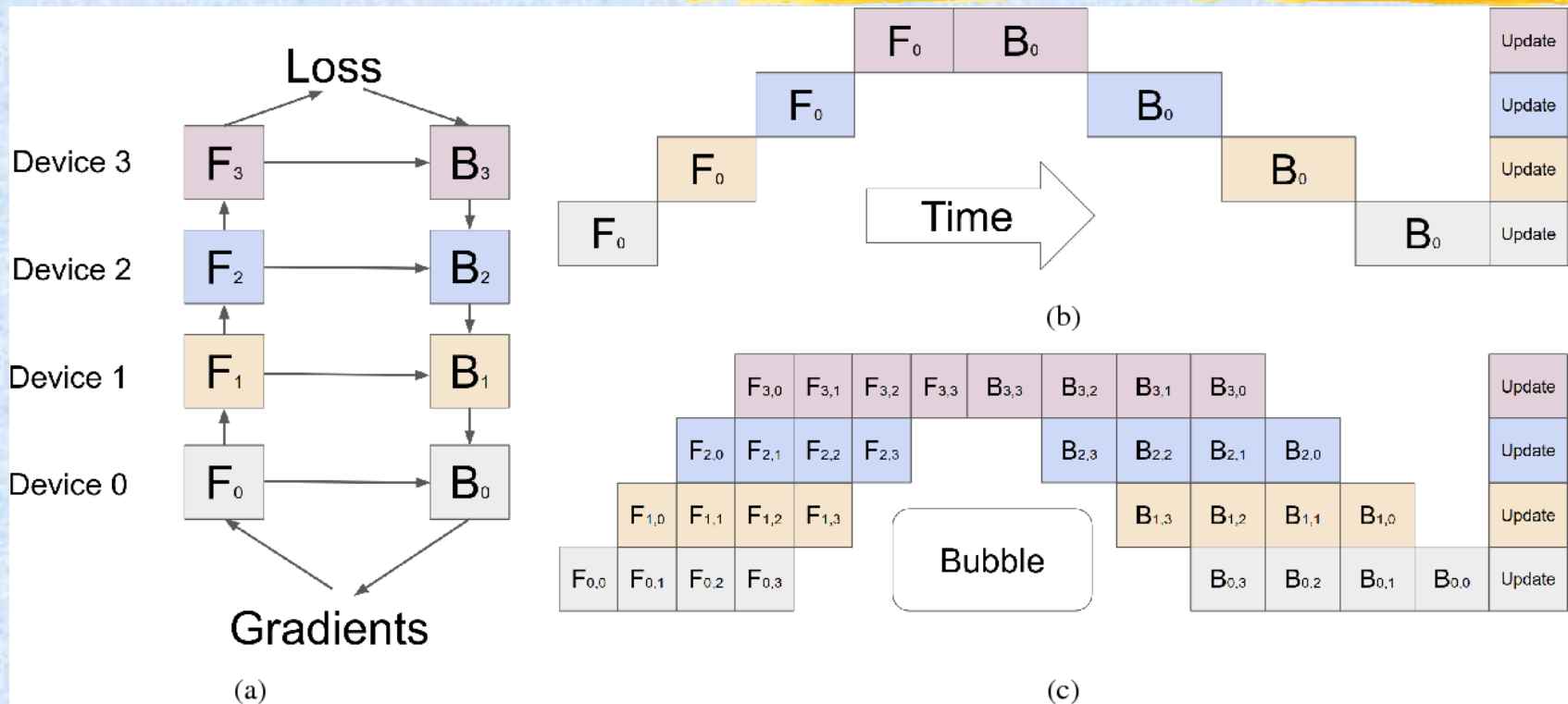


•Spawn and data dependency



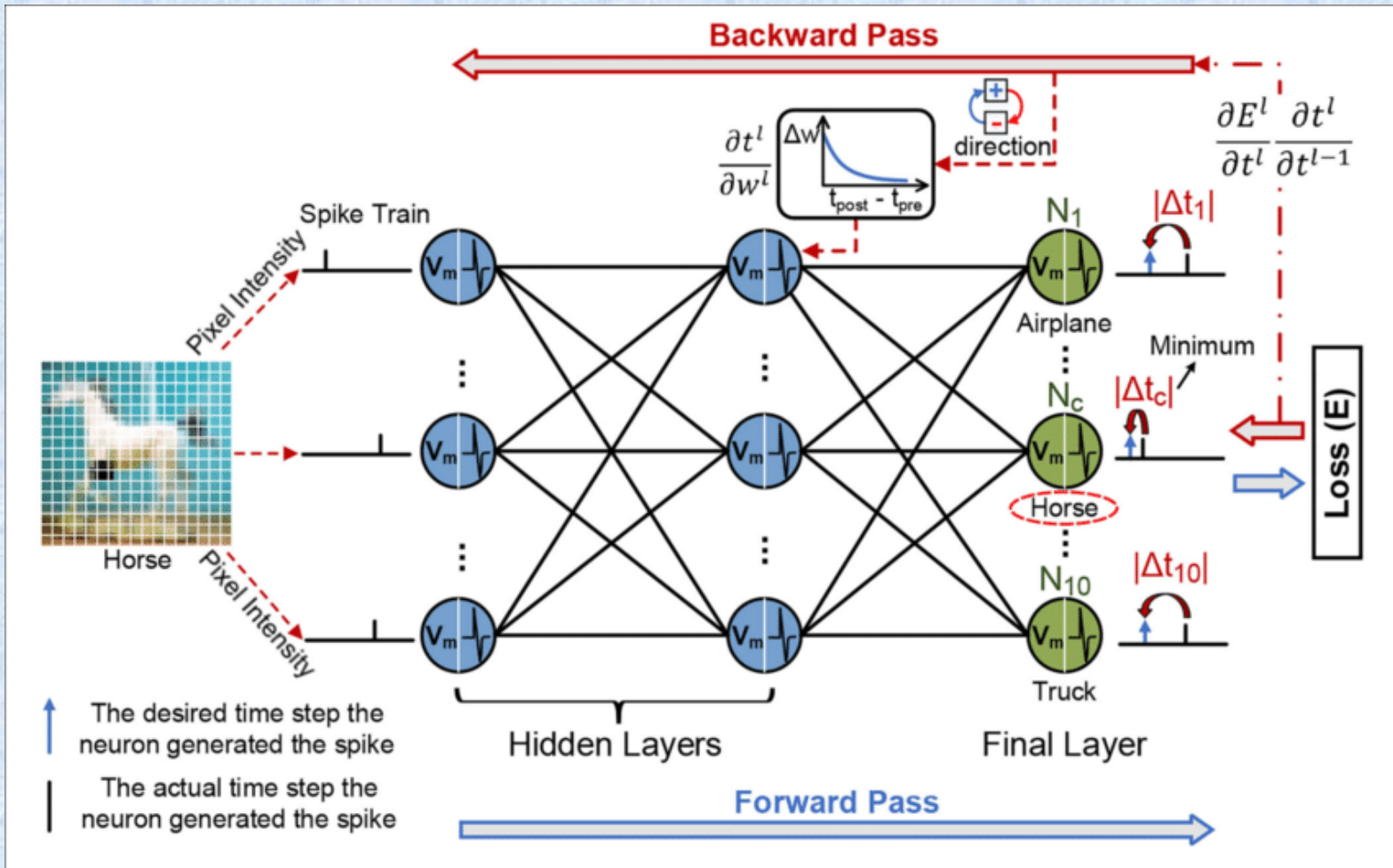
•Pipeline dependency (under utilisation and partition in mini batches)

Data dependency in pipelines



(a) An neural network with sequential layers is partitioned across four accelerators. F_k is the composite forward computation function of k -th partition. B_k is the corresponding backpropagation function. B_k depends on both B_{k+1} from upper layer and the intermediate activations of F_k . (b) The naive model parallelism strategy leads to severe under-utilization due to the sequential nature of the network. Only one accelerator is active at a time. (c) Pipeline parallelism divides the input mini-batch into smaller micro-batches. It enables different accelerators to work on different micro-batches at the same time. $F_{k,t}$ and $B_{k,t}$ refer to forward and backward computation of the t -th micro-batch on k -th partition. Gradients are applied synchronously at the end of each mini-batch.

Parallel architectures



Parallel architectures

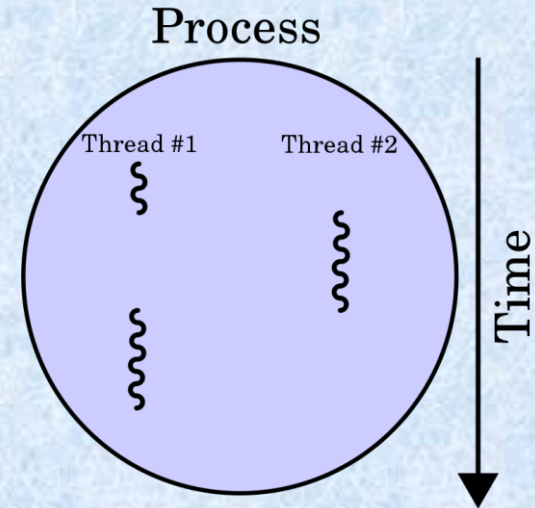


Multiple units parallelism

- **Functional units:** reciprocally independent (i. e. ALU, FPU, load/store unit, branch unit)
- **Parallelism:** different independent instructions can be executed in parallel by different functional units
- **Data dependency:** in superscalar processors dependency can be runtime identified dynamically and the scheduling unit dispatches instruction to the units dynamically

Process or thread level parallelism

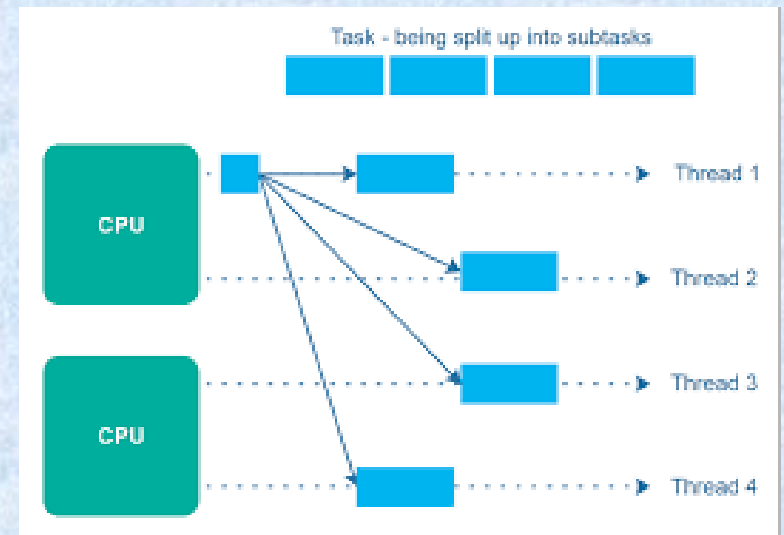
- **Alternative approach:** different independent core units hw integrated into the chip running **threads**
- **Control flow:** instead of a single sequential flow we have several ones (each for every core or thread)
- **Parallel programming:** coordination of instructions sequences, multiple accesses to the memory, cache sharing, synchronization and concurrency



Parallel architectures



	IF	ID	EX	MEM	WB					
	IF	ID	EX	MEM	WB					
i		IF	ID	EX	MEM	WB				
		IF	ID	EX	MEM	WB				
t			IF	ID	EX	MEM	WB			
			IF	ID	EX	MEM	WB			
				IF	ID	EX	MEM	WB		
				IF	ID	EX	MEM	WB		
					IF	ID	EX	MEM	WB	
					IF	ID	EX	MEM	WB	



•Superscalar processor

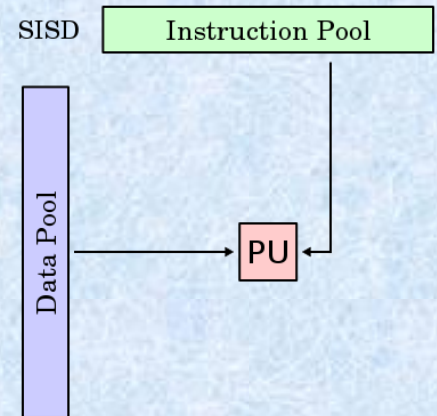
•Thread level parallelism

Flynn Taxonomy



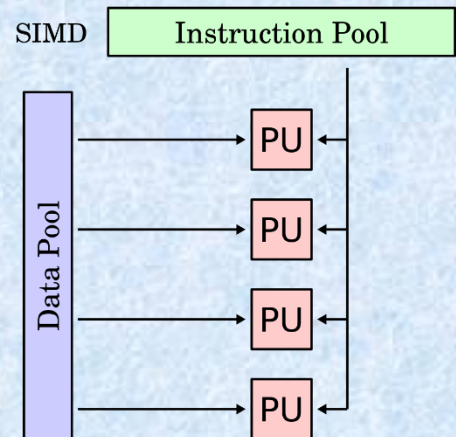
SISD (Single Instruction Single Data)

- **A single computation unit** accessing program and data
- **No parallelism**: sequential elaboration (Von Neumann)



SIMD (Single Instruction Multiple Data)

- **Many computation units** accessing private data memory (shared or distributed), global for instructions
- **Single instruction**: at every step the central unit sends an instruction to the units that execute it on their own data
- **Applications**: high level of parallelism (multimedia, computer graphics, simulations, ...)

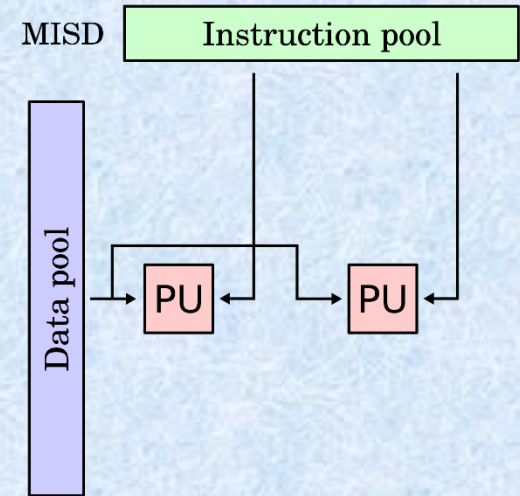


Flynn Taxonomy



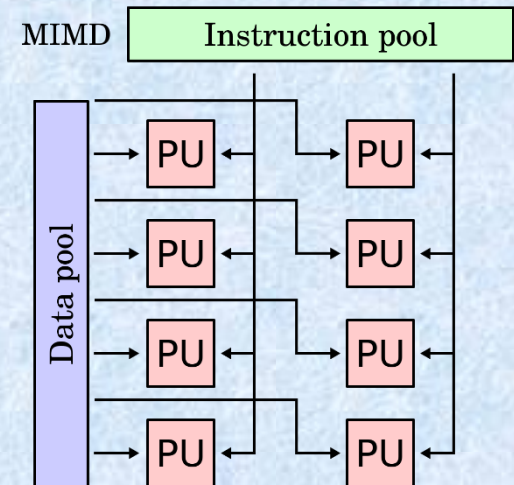
MISD (Multiple Instruction Single Data)

- **Many computation units** with private program memory and common access to a global unique memory for data
- **Single data**: at every step every unit receives same data and loads an instruction to elaborate it from the private memory
- **Instruction level parallelism** (no commercial implementation known)



MIMD (Multiple Instruction Multiple Data)

- **Many computing units** with separate access to data/instructions in shared or distributed memories
- **Every step** every unit loads its own instruction and executes it on data independently and asynchronously from the others
- **Examples**: multicore processor, distributed system

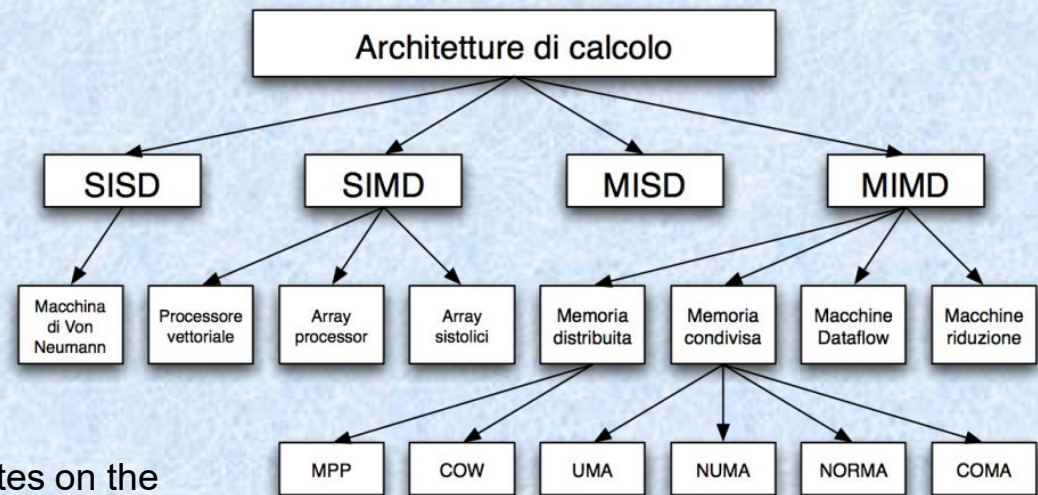


- multiple cores elaborate same instruction on different data
- «think serial» although with a good speedup
- the compiler makes things transparent to the programmer

- multiple cores on different data streams
- every stream = independent instructions
- SIMD is a sub-case

- serial architecture with a unique core
- one instruction on one data at a time

- non common architecture: every core operates on the same data stream through different streams made of independent instructions



Beyond Flynn Taxonomy: SIMT



SIMT (Single Instruction Multiple Threads)

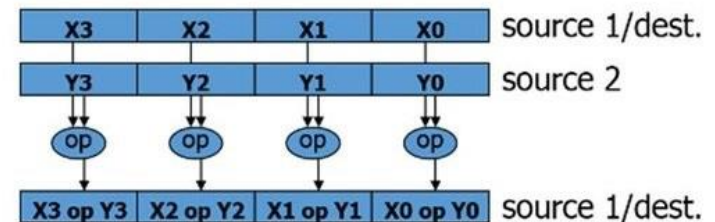
- **Multithreading**: in multitasking operative systems allows multiple threads to co-exist within a running process
- **SIMT** used when combining SIMD with multithreading
- **Shared resource** with independent executions providing a powerful abstraction of the concurrent execution concept
- **Extension**: a process can enable parallel executions on multiprocessor or multicore systems
- **Implemented** in many GPUs and mandatory for GPGPU. Some supercomputers join CPU with GPU

CPU

SIMD

1 instruction – multiple data

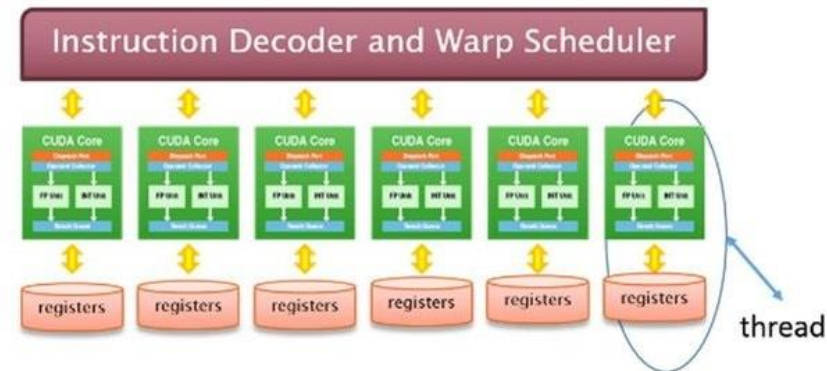
SSE2/3/4 – Neon – AltiVec
AVX – AVX2...



GPU

SIMT

1 instruction – multiple threads

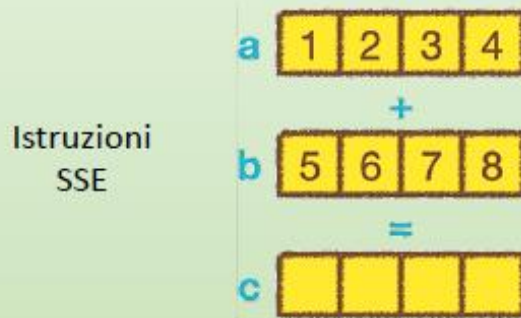


Beyond Flynn Taxonomy: SIMT



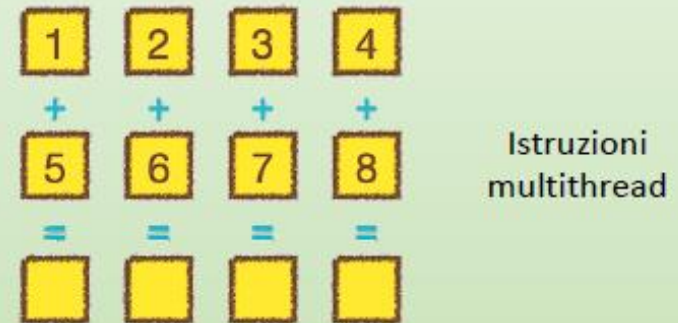
The **SIMT** model includes three characteristics not present in SIMD:

1. Every thread has its own **instruction address counter**
2. Every thread has its own **state register** and a **register set**
3. Every thread can have an independent **execution path**



```
__m128 a = _mm_set_ps (4, 3, 2, 1);  
__m128 b = _mm_set_ps (8, 7, 6, 5);  
__m128 c = _mm_add_ps (a, b);
```

SSE= Streaming SIMD Extension -> instructions both on scalar numbers and on floating numbers groups



```
float a[4] = {1, 2, 3, 4};  
float b[4] = {5, 6, 7, 8}, c[4];  
{  
    int id = ... ; // my thread ID  
    c[id] = a[id] + b[id];  
}
```

Beyond Flynn Taxonomy: SIMT



SIMT Execution Model

```
A[] = {4,8,12,16};
```

```
A: K = A[tid.x];
```

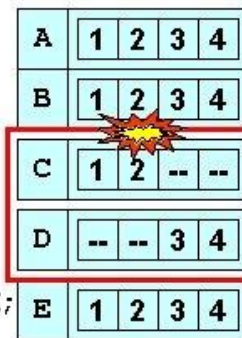
```
B: if (K > 10)
```

```
C:   K = 10;
```

```
   else
```

```
D:   K = 0;
```

```
E: B = C[tid.x] + K;
```



50% SIMD Efficiency!

Branch Divergence

In some cases: SIMD Efficiency → **20%**

Reconvergence Stack

PC	RPC	Active Mask
E	-	1111
D	E	0011
C	E	1100

NVIDIA GPUs



GPU Computing Applications						
Libraries and Middleware						
cuDNN TensorRT	cuFFT, cuBLAS, cuRAND, cuSPARSE	CULA MAGMA	Thrust NPP	VSIP, SVM, OpenCL	PhysX, OptiX, iRay	MATLAB Mathematica
Programming Languages						
C	C++	Fortran	Java, Python, Wrappers	DirectCompute	Directives (e.g., OpenACC)	
CUDA-enabled NVIDIA GPUs						
Turing Architecture (Compute capabilities 7.x)	DRIVE/JETSON AGX Xavier	GeForce 2000 Series	Quadro RTX Series	Tesla T Series		
Volta Architecture (Compute capabilities 7.x)	DRIVE/JETSON AGX Xavier			Tesla V Series		
Pascal Architecture (Compute capabilities 6.x)	Tegra X2	GeForce 1000 Series	Quadro P Series	Tesla P Series		
Maxwell Architecture (Compute capabilities 5.x)	Tegra X1	GeForce 900 Series	Quadro M Series	Tesla M Series		
Kepler Architecture (Compute capabilities 3.x)	Tegra K1	GeForce 700 Series GeForce 600 Series	Quadro K Series	Tesla K Series		
	EMBEDDED	CONSUMER DESKTOP, LAPTOP	PROFESSIONAL WORKSTATION	DATA CENTER		

NVIDIA GPUs



GPU Computing Applications

Libraries and Middleware

cuDNN TensorRT	cuFFT cuBLAS cuRAND cuSPARSE	CULA MAGMA	Thrust NPP	VSIPL SVM OpenCurrent	PhysX OptiX iRay	MATLAB Mathematica
-------------------	---------------------------------------	---------------	---------------	-----------------------------	------------------------	-----------------------

Programming Languages

C	C++	Fortran	Java Python Wrappers	DirectCompute	Directives (e.g. OpenACC)
---	-----	---------	----------------------------	---------------	------------------------------



CUDA-Enabled NVIDIA GPUs

NVIDIA Ampere Architecture (compute capabilities 8.x)				Tesla A Series
NVIDIA Turing Architecture (compute capabilities 7.x)		GeForce 2000 Series	Quadro RTX Series	Tesla T Series
NVIDIA Volta Architecture (compute capabilities 7.x)	DRIVE/JETSON AGX Xavier		Quadro GV Series	Tesla V Series
NVIDIA Pascal Architecture (compute capabilities 6.x)	Tegra X2	GeForce 1000 Series	Quadro P Series	Tesla P Series



Embedded



Consumer
Desktop/Laptop



Professional
Workstation

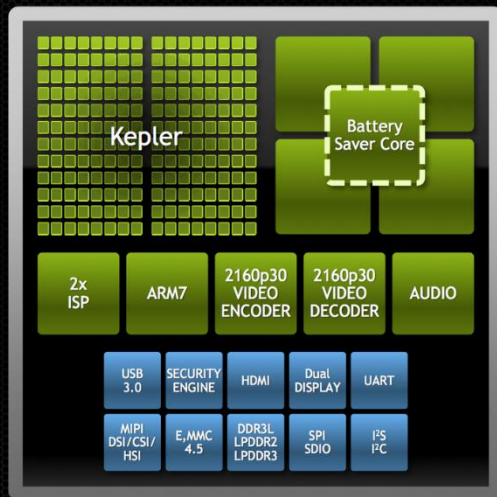


Data Center

TEGRA K1



Tegra K1



GPU	Kepler GPU (192 CUDA Cores) <i>Open GL 4.4, OpenGL ES3.0, DX11, CUDA 6</i>
CPU	Quad Core Cortex A15 "r3" <i>With 5th Battery-Saver Core; 2MB L2 cache</i>
CAMERA	Dual High Performance ISP <i>1.2 Gigapixel throughput, 100MP sensor</i>
POWER	Lower Power <i>28HPM, Battery Saver Core</i>
DISPLAY	4K panel, 4K HDMI <i>DSi, eDP, LVDS, High Speed HDMI 1.4a</i>



- **Embedded (Jetson Dev Kit)**
- **Mobile**
- **Graphics**



GeForce GTX TITAN



GeForce GTX TITAN Specifications

CUDA Cores	2688
Base Clock	837 MHz
Boost Clock	876 MHz
Single Precision	4.5 Teraflops
Double Precision	1.3 Teraflops
Memory Config	6GB / 384-bit GDDR5
Memory Speed	6.0 Gbps
Power Connectors	6-pin + 8-pin
TDP	250W
Outputs	2x DL-DVI HDMI Displayport 1.2
Bus Interface	PCI Express 3.0



- Gaming
- HQ Graphics



QUADRO 6000



	<i>Quadro FX 5800</i>	<i>Quadro 6000</i>
Memory Size	4GB GDDR3	6GB GDDR5
Memory Interface	512-bit	384-bit
Memory Bandwidth	102GB/s	144GB/s
CUDA Parallel Processor Cores	240	448
Max Power Consumption	189W	225W
Number of Slots	2	2
Display Connectors	1 DP, 2 DVI, Stereo	2 DP, 1 DVI, Stereo
OpenGL	3.3	4
Shader Model	4	5
DirectX	10	11
3D Vision Pro	Yes	Yes
Quadro Panoramic	Yes	Yes
Error Correcting Code ECC	No	Yes
Fast Double Precision	No	Yes
Triangles per Second	300 Million	1.3 Billion
MSRP (U.S.)	\$2,999	\$4,999

- **High professional Graphics**



TESLA K40



Type	Tesla k40	Pascal GTX 1080
Number of processor cores	2880	2560
SIMD lane width	8	
Maximum threads per processor	2048	
Maximum threads per block	1024	
Number of 32-bit registers	65536	
Maximum registers per threads	255	
Shared memory	64KB shared	96KB dedicated
L1 cache	64KB shared	64KB shared
Read-only data cache	48KB dedicated	64KB shared
L2 cache	1536KB	2048KB
GPU maximum clock rate	745Mhz	1607MHz
Memory clock rate	3004Mhz	10000MHz
Memory interface	384-bit	256-bit
Memory bandwidth	208 GB/sec	320 GB/sec
Memory size	12GB	8GB

- **High Performance Computing**



GPU GENERAL ARCHITECTURE



The GPU generic architecture consists of an array of scalable **Streaming Multiprocessors (SM)**

Every SM in a GPU is designed to support concurrent execution of hundreds of **threads**

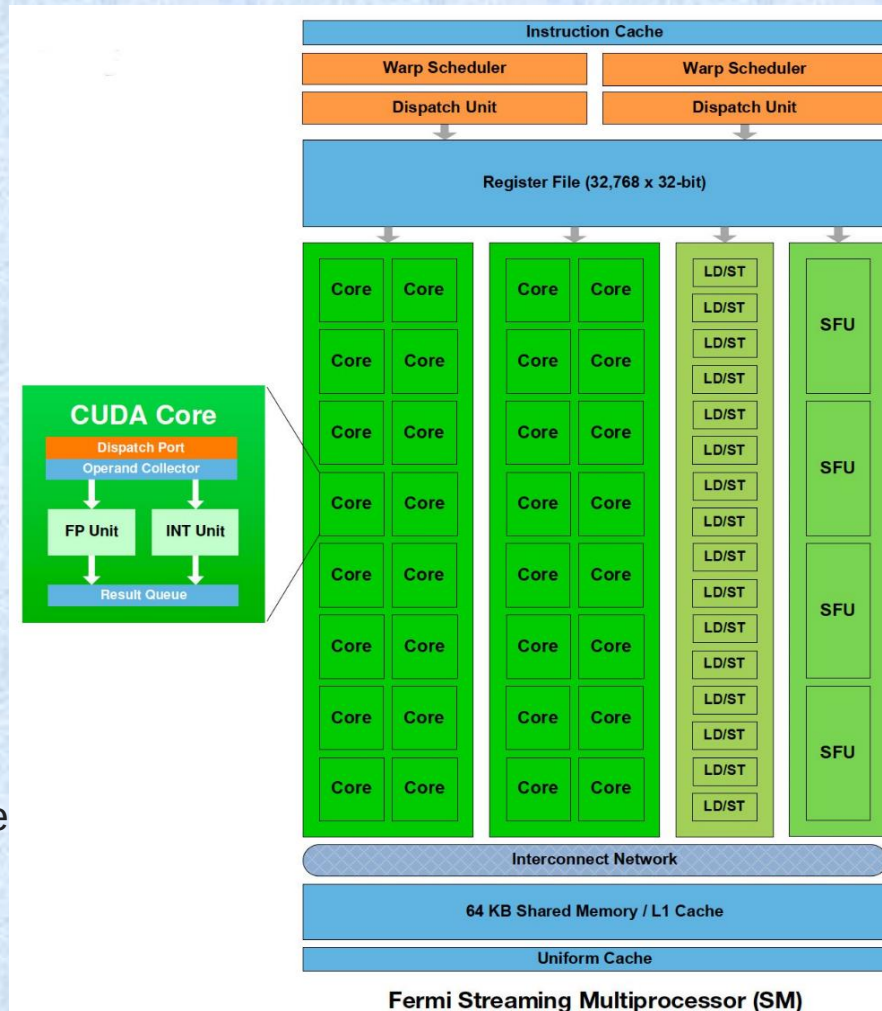
Multiple SM for NVIDIA GPUs run threads in groups of 32 called **warps**

All the threads in a warp execute the same instruction at the same time

GPU HW differ according to the “**compute capability**”(CC): the higher the better!

The Maxwell architecture (e.g. GTX980) has CC 5.2
Pascal architecture (e.g. GTX1080) has CC 6.0- 6.2. The
(last) Volta architecture has CC 7.0

SFU= special functions units for transcendental functions



GPU ARCHITECTURES



Fermi (2010): for **HPC applications**, such as seismic data processing, biochemical simulations, models of climate and fluid dynamics, signal processing, computational finance and data analysis

Kepler (2012): improves Fermi in terms of **computing power and energy consumption** adding new features, i. e. optimisation and augmentation of the **parallel workload**

Maxwell (2014): next-generation architecture that extends the previous programming model preserving compatibility with the previous versions, increases compute performance compared to Kepler, providing much higher **memory bandwidth**

Pascal (2016): over **5 TFLOPS** in double precision, **deep learning** (12x performance times in neural network training), artificial intelligence

Volta, Turing (2018): calculations on **tensors** for AI

Ampere (2020): AI deep learning (DL), data analytics, genomics, edge video analytics and 5G services, graphic rendering, cloud gaming

Compute capability



The **compute capability** of a device is represented by a version number, also sometimes called its "SM version". This version number identifies the **features** supported by the GPU hardware and **is used by applications at runtime to determine which hardware features and/or instructions are available on the present GPU.**

The compute capability includes a **major revision** number X and a **minor revision** number Y and is denoted by $X.Y$.

Devices with the same **major revision number** feature the same core architecture. The highest (so far) revision number is **8** for devices based on the **NVIDIA Ampere GPU** architecture, **7** for devices based on the **Volta** architecture, **6** for devices based on the **Pascal** architecture, **5** for devices based on the **Maxwell** architecture, **3** for devices based on the **Kepler** architecture, **2** for devices based on the **Fermi** architecture, and **1** for devices based on the **Tesla** architecture.

The **minor revision number** corresponds to an incremental improvement to the core architecture, possibly including new features.

Turing is the architecture for devices of compute capability 7.5, and is an incremental update based on the **Volta** architecture.

Compute capability



GeForce and TITAN Products

GPU	Compute Capability
NVIDIA TITAN RTX	7.5
GeForce RTX 2080 Ti	7.5
GeForce RTX 2080	7.5
GeForce RTX 2070	7.5
GeForce RTX 2060	7.5
NVIDIA TITAN V	7.0
NVIDIA TITAN Xp	6.1
NVIDIA TITAN X	6.1
GeForce GTX 1080 Ti	6.1
GeForce GTX 1080	6.1
GeForce GTX 1070	6.1
GeForce GTX 1060	6.1
GeForce GTX 1050	6.1
GeForce GTX TITAN X	5.2
GeForce GTX TITAN Z	3.5

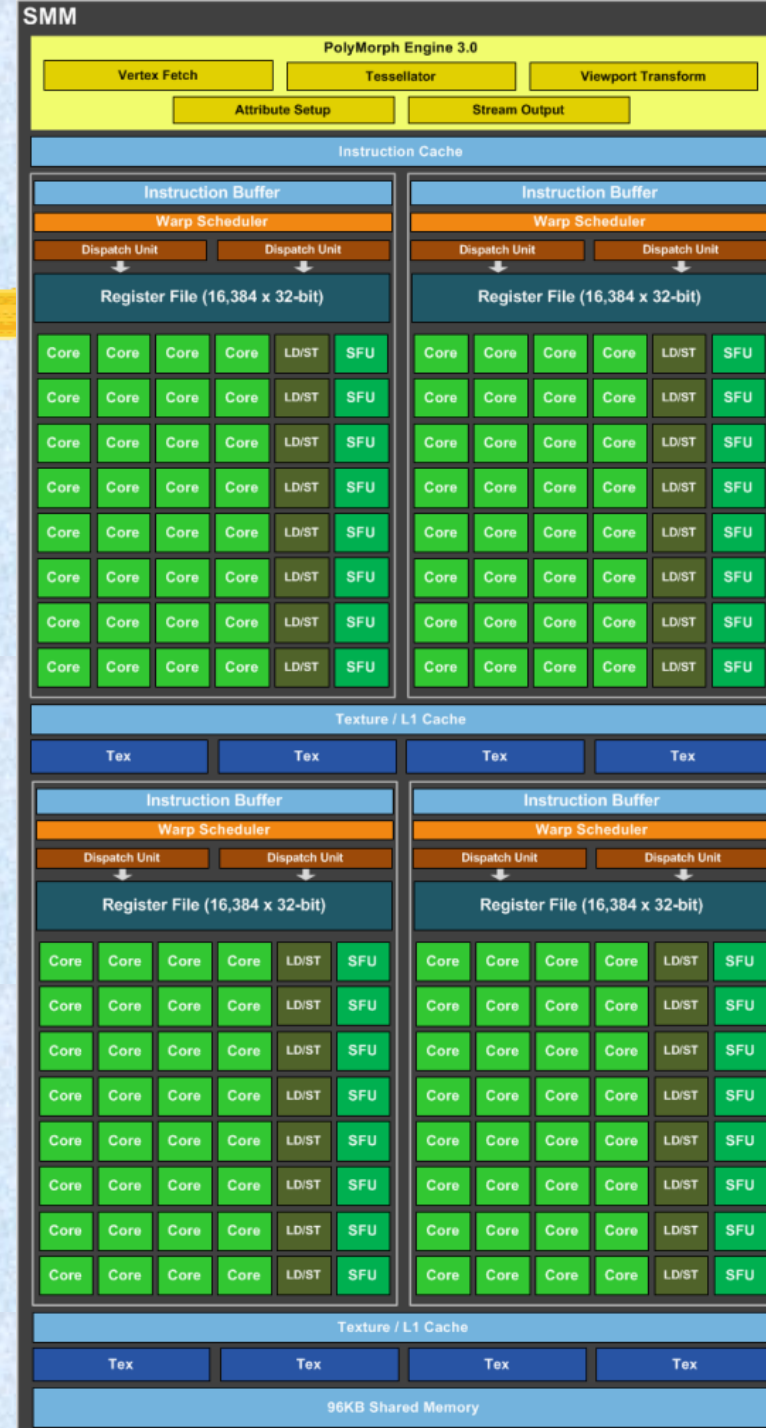
GeForce Notebook Products

GPU	Compute Capability
GeForce RTX 2080	7.5
GeForce RTX 2070	7.5
GeForce RTX 2060	7.5
GeForce GTX 1080	6.1
GeForce GTX 1070	6.1
GeForce GTX 1060	6.1
GeForce GTX 980	5.2
GeForce GTX 980M	5.2
GeForce GTX 970M	5.2
GeForce GTX 965M	5.2
GeForce GTX 960M	5.0
GeForce GTX 950M	5.0
GeForce 940M	5.0
GeForce 930M	5.0
GeForce 920M	3.5

Feature support (unlisted features are supported for all compute capabilities)	Compute capability (version)											
	1.0	1.1	1.2	1.3	2.x	3.0	3.5	3.7	5.0	5.2	6.0	6.1
Integer atomic functions operating on 32-bit words in global memory	No	Yes										
atomicExch() operating on 32-bit floating point values in global memory												
Integer atomic functions operating on 32-bit words in shared memory	No	Yes										
atomicExch() operating on 32-bit floating point values in shared memory												
Integer atomic functions operating on 64-bit words in global memory												
Warp vote functions												
Double-precision floating-point operations	No			Yes								
Atomic functions operating on 64-bit integer values in shared memory	No				Yes							
Floating-point atomic addition operating on 32-bit words in global and shared memory												
_ballot()												
_threadfence_system()												
_syncthreads_count(), _syncthreads_and(), _syncthreads_or()												
Surface functions												
3D grid of thread block	No				Yes							
Warp shuffle functions												
Funnel shift												
Dynamic parallelism	No						Yes					

MAXWELL

- Four 32-core processing blocks each with a dedicated warp scheduler that can dispatch 2 instructions per clock
- Larger shared memory (dedicated to SM)
- Larger L2 cache (shared by SMs)
- SFU SIMF Functional units



PASCAL



Perhaps the first for HPC

More SMs per GPU

FP16 computation (2× faster than FP32): mixed precision.

FP64 units (Dpunits)

Less cores but same # registers:
more registers per core

5.3 TFlops

Fast HBM2 memory interface (better than DDR4 or GDDR5)

Fast NVLink bus (mem transfers)

Unified memory: programs can access both CPU and GPU RAM



VOLTA

New tensor cores

Unified L1 / shared memory

Independent FP32 and INT32 cores

More SMs per GPU

Larger L2 cache

New L0 instruction cache (accessed directly from functional units)



CUDA – Compute Unified Device Architecture

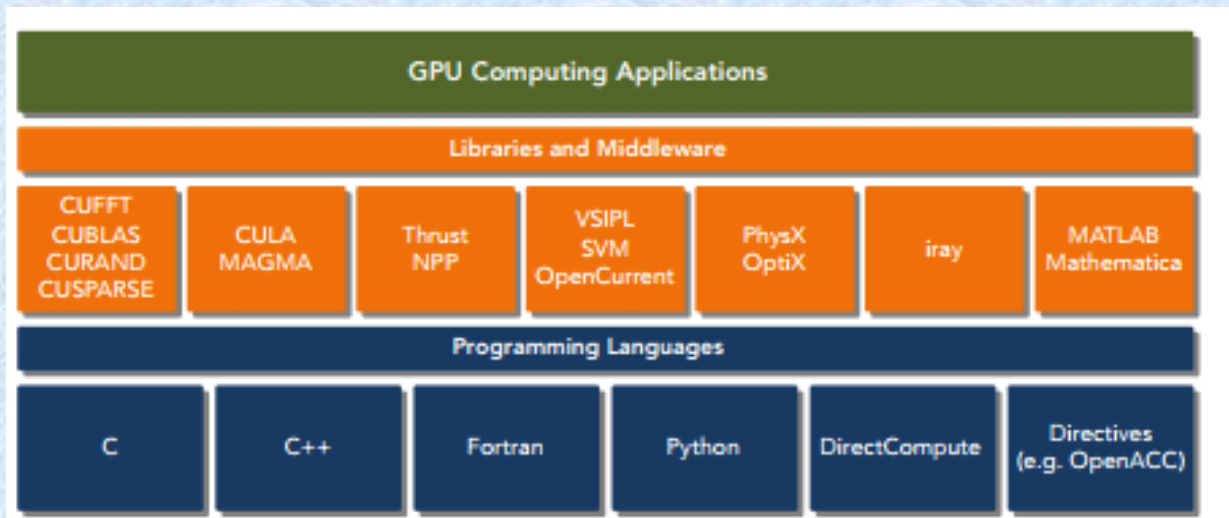


- It enables a **general-purpose programming model** on NVIDIA GPUs
- Current CUDA SDK is 13.2
- Enables explicit GPU **memory management**
- The **GPU** is viewed as a **compute device** that:
 - Is a **co-processor** to the CPU (or host)
 - Has its own **DRAM** (*global memory* in CUDA parlance)
 - Runs many **threads** in parallel

CUDA – Compute Unified Device Architecture



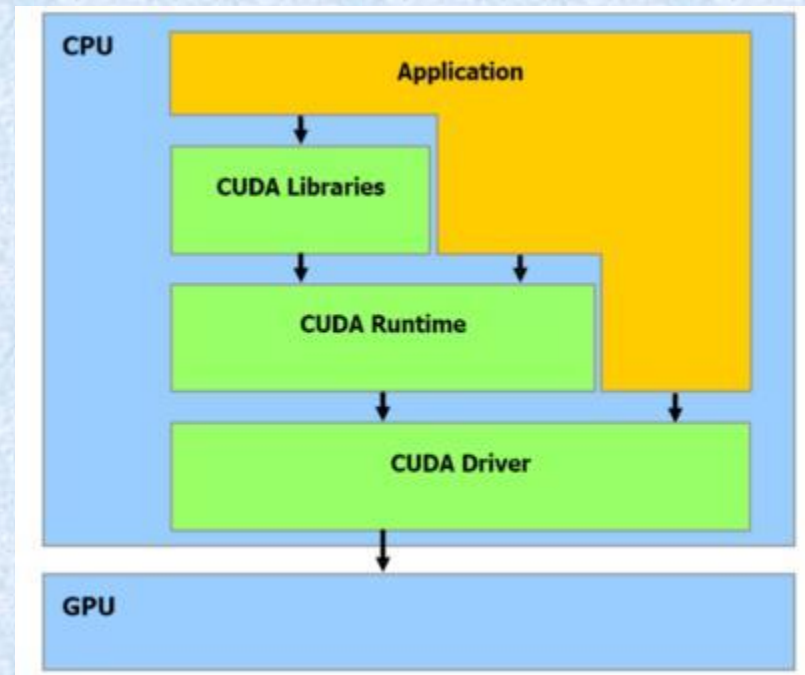
- The CUDA platform is accessible through CUDA-accelerated libraries, compiler directives, application programming interfaces (API), and extensions to industry-standard programming languages, including C, C++, Fortran, and Python
- CUDA C is an extension of standard ANSI C with a handful of language extensions to enable heterogeneous programming, and straightforward APIs to manage devices, memory, and other tasks.



CUDA – Compute Unified Device Architecture



- CUDA provides two levels of API to GPU and thread management
 - **CUDA Driver API**
 - **CUDA Runtime API**
- **API drivers** are low level APIs **not easy to program but with a better control of the GPU**
- **API runtime** are high level APIs implemented onto API driver
- Every function of API runtime is subdivided in different basic operations provided by API driver





Driver vs runtime APIs

The driver and runtime APIs are very similar and can for the most part be used interchangeably. However, there are some key differences worth noting between the two.

Complexity vs. control

The **runtime API eases** device code **management** by providing **implicit initialization**, **context management**, and **module management**. This leads to simpler code, but it also **lacks** the **level of control** that the **driver API** has.

In comparison, the **driver API offers** more **fine-grained control**, especially over contexts and module loading. **Kernel launches** are much **more complex** to implement, as the execution **configuration** and kernel **parameters** must be specified with **explicit** function calls.

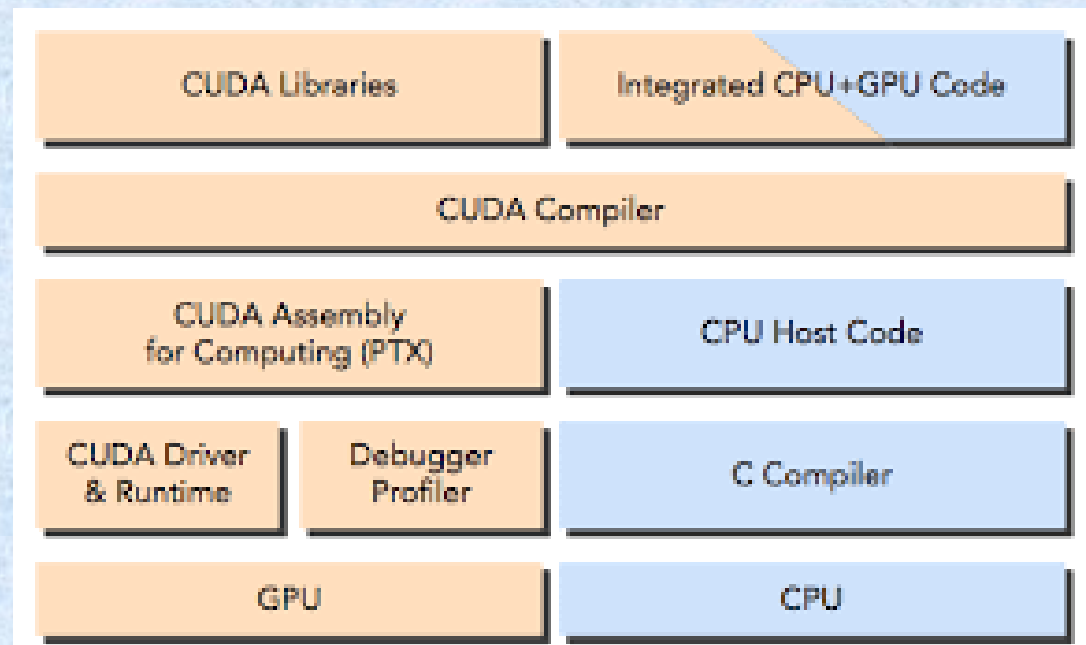
However, unlike the runtime, where all the kernels are automatically loaded during initialization and remain loaded for as long as the program runs, with the **driver API** it is possible to only keep loaded those modules that are currently needed, or even dynamically reload modules. The **driver API** is also **language-independent** as it only deals with **cubin** objects.



Driver vs runtime APIs

A CUDA program consists of a mixture of the following two parts:

- The **host code** runs on CPU
- The **device code** runs on GPU
- NVIDIA's CUDA **nvcc** compiler separates the *device* code from the *host* code during the compilation process.



Compiling in CUDA



Optimized libraries:
`math.h`, BLAS, FFT

Integrated CPU and
GPU source code

CUDA C Compiler (`nvcc`)

Machine independent
assembly (PTX)

CPU Host Code

CUDA
Driver

Debugger
Profiler

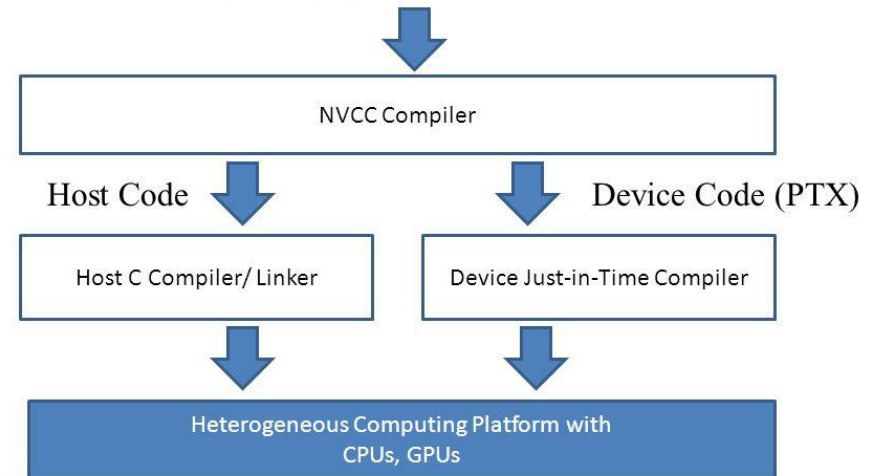
Standard C compiler

GPU

CPU

Compiling A CUDA Program

Integrated C programs with CUDA extensions





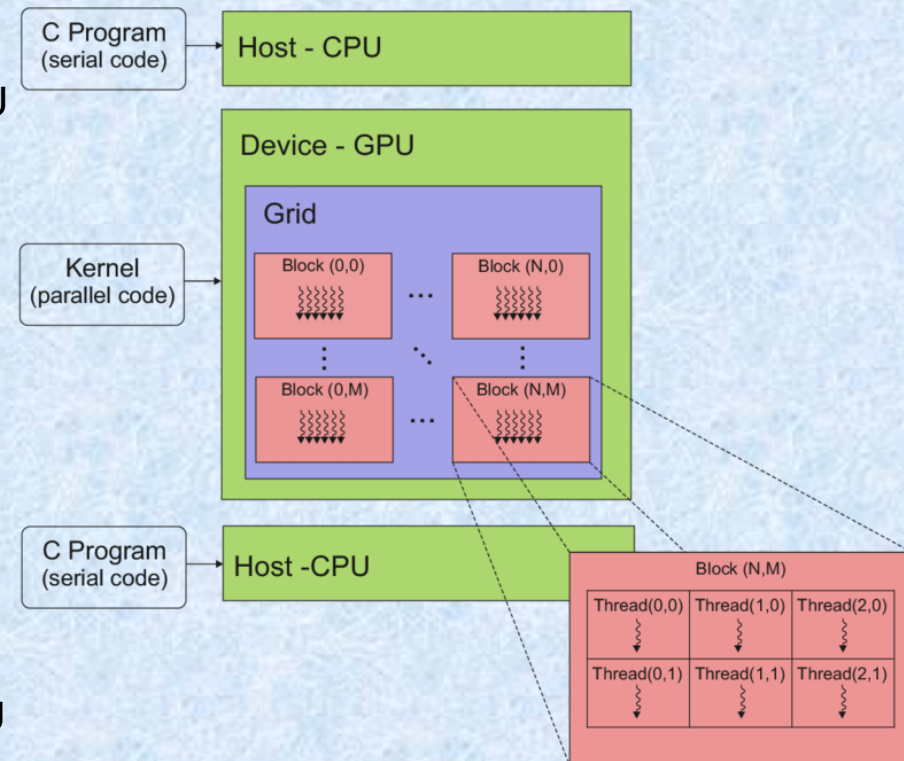
Programming in CUDA means ...

- **Thinking parallel**: clearly know what features the GPU makes available to the programmer
- **Know the GPU architecture** to scale over thousands of threads as if they were one
 - Low level cache management allows to exploit the locality principle
 - Know the scheduling of thread blocks and the thread and memory hierarchy (less latencies)
 - Make direct use of shared memory (reduces latencies such as caches)
 - Manage synchronizations directly (barriers between threads)
- We work similarly to *pthread* or *OpenMP*
- We write a portion of CUDA C (simple extension of C) for sequential execution and extend it to miles of threads (allows you to think 'again' sequentially)
- **CUDA DEVELOPMENT ENVIRONMENT**
 - NVIDIA Nsight™ integrated development environment
 - CUDA-GDB command line debugger
 - Visual and command line profiler for performance analysis
 - CUDA-MEMCHECK memory analyzer
 - GPU device management tools

Key elements of CUDA programming



- **Control**: threads and data memory are in the hands of the programmer
- **Kernel**: sequential program executed in the GPU
- **Host**: operates independently of the Device (for most of the operations)
- **Host code**: program in ANSI C, Python, ...
- **Device Code**: program in CUDA C, Python, ...
- **Asynchronous**: computations (kernel) on GPU and CPU
- **Synchronous**: transfers between CPU and GPU memories
- **Compiler**: nvcc NVIDIA generates executable code for host and device (fat-binary)





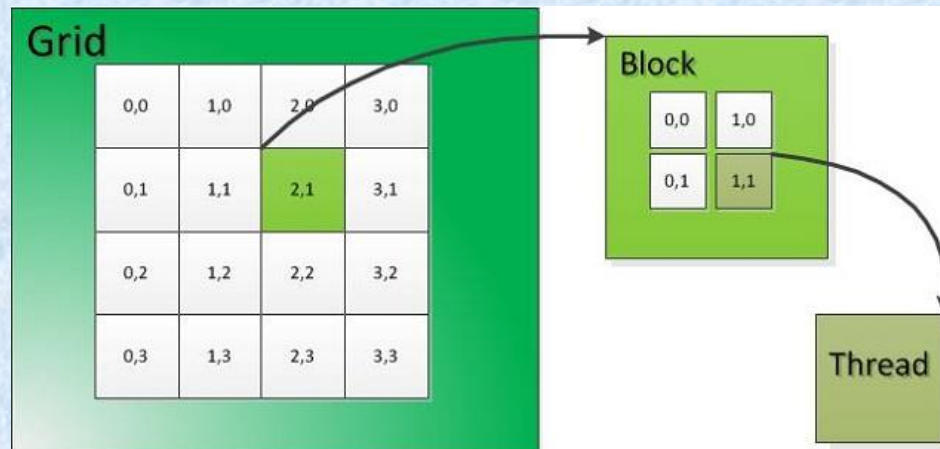
Basic rules of CUDA programming

- Host data **setup** (CPU-accessible memory)
- **Allocate** memory for data on the GPU
- **Copy** data from host to GPU
- **Allocate** memory for host output
- **Allocate** memory for GPU output
- **Run** the kernel on the GPU
- **Copy** output from GPU to host
- **Free** the memories



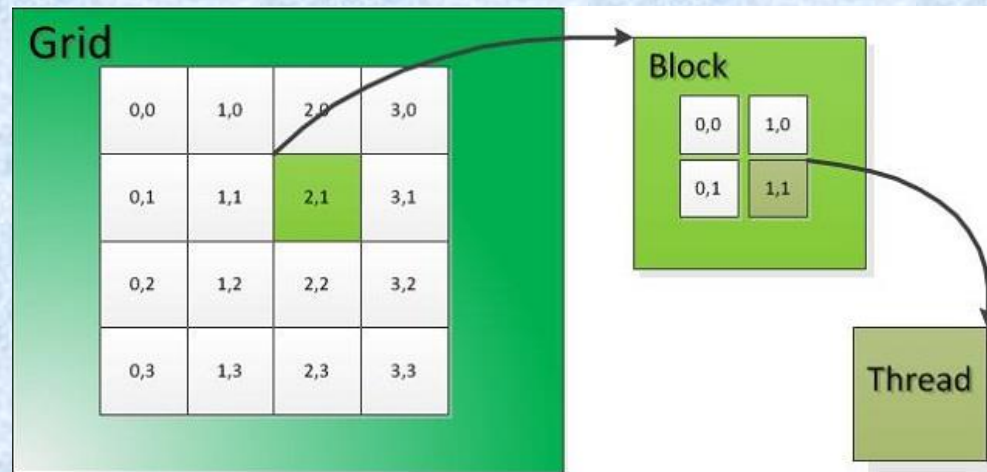
Thread organisation

- CUDA features a two-level abstract **thread hierarchy** that consists of
 - **grid**: an ordered collection of blocks
 - **block**: an ordered collection of threads
- *Structure*: grids and blocks can have the following dimensions
 - **grid**: 1D, 2D and 3D
 - **block**: 1D, 2D and 3D
- 9 possible combinations, but generally the same structure is used both for grid and for block
- The choice of dimensions has to be defined according to the size of the task data structure





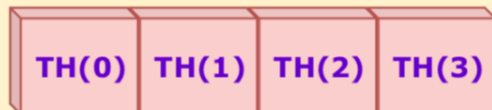
Thread organisation



GRID 1D

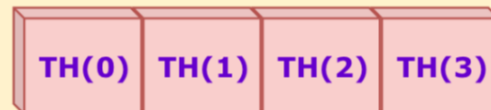
BLOCK 1D

BK(0)



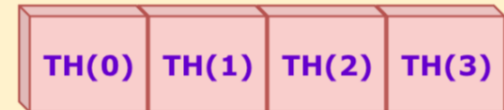
BLOCK 1D

BK(1)



BLOCK 1D

BK(2)





1D grid and 2D blocks

GRID 1D

BLOCK 2D

BK(0)

TH(0,0)	TH(1,0)	TH(2,0)
TH(0,1)	TH(1,1)	TH(2,1)
TH(0,2)	TH(1,2)	TH(2,2)

BLOCK 2D

BK(1)

TH(0,0)	TH(1,0)	TH(2,0)
TH(0,1)	TH(1,1)	TH(2,1)
TH(0,2)	TH(1,2)	TH(2,2)

BLOCK 2D

BK(2)

TH(0,0)	TH(1,0)	TH(2,0)
TH(0,1)	TH(1,1)	TH(2,1)
TH(0,2)	TH(1,2)	TH(2,2)



2D grid and 2D blocks

GRID 2D

BLOCK 2D

BK(0,0)

TH(0,0)	TH(1,0)	TH(2,0)
TH(0,1)	TH(1,1)	TH(2,1)
TH(0,2)	TH(1,2)	TH(2,2)

BLOCK 2D

BK(1,0)

TH(0,0)	TH(1,0)	TH(2,0)
TH(0,1)	TH(1,1)	TH(2,1)
TH(0,2)	TH(1,2)	TH(2,2)

BLOCK 2D

BK(2,0)

TH(0,0)	TH(1,0)	TH(2,0)
TH(0,1)	TH(1,1)	TH(2,1)
TH(0,2)	TH(1,2)	TH(2,2)

BLOCK 2D

BK(0,1)

TH(0,0)	TH(1,0)	TH(2,0)
TH(0,1)	TH(1,1)	TH(2,1)
TH(0,2)	TH(1,2)	TH(2,2)

BLOCK 2D

BK(1,1)

TH(0,0)	TH(1,0)	TH(2,0)
TH(0,1)	TH(1,1)	TH(2,1)
TH(0,2)	TH(1,2)	TH(2,2)

BLOCK 2D

BK(2,1)

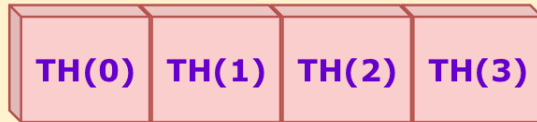
TH(0,0)	TH(1,0)	TH(2,0)
TH(0,1)	TH(1,1)	TH(2,1)
TH(0,2)	TH(1,2)	TH(2,2)

Maximum 1024 threads x block



BLOCK 1D

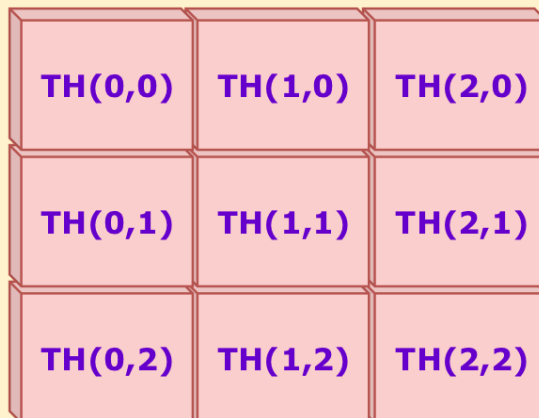
BK(0)



- (32, 1, 1)
- (128, 1, 1)
- ...
- (1024, 1, 1)
- ...
- (4096, 1, 1)

BLOCK 2D

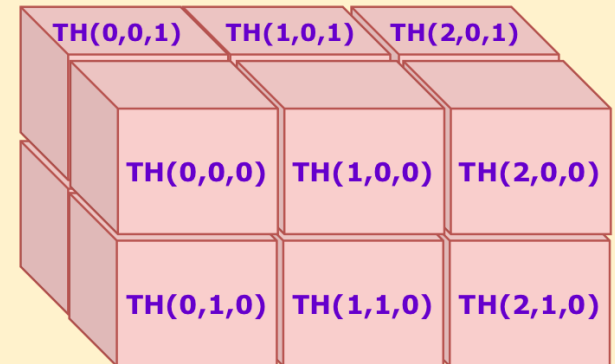
BK(0)



- (32, 32, 1)
- (128, 4, 1)
- ...
- (512, 2, 1)
- (64, 16, 1)
- ...
- (64, 32, 1)

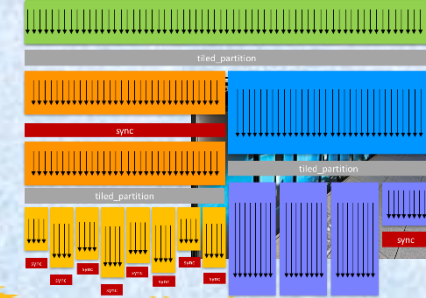
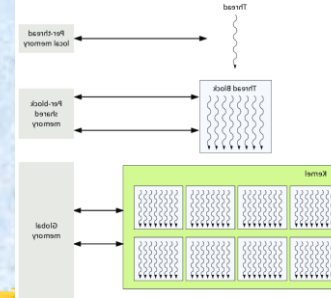
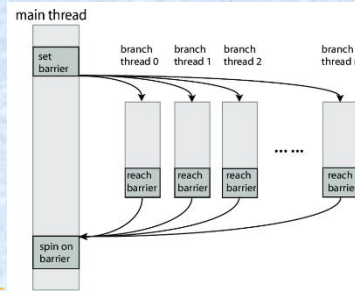
BLOCK 3D

BK(0)

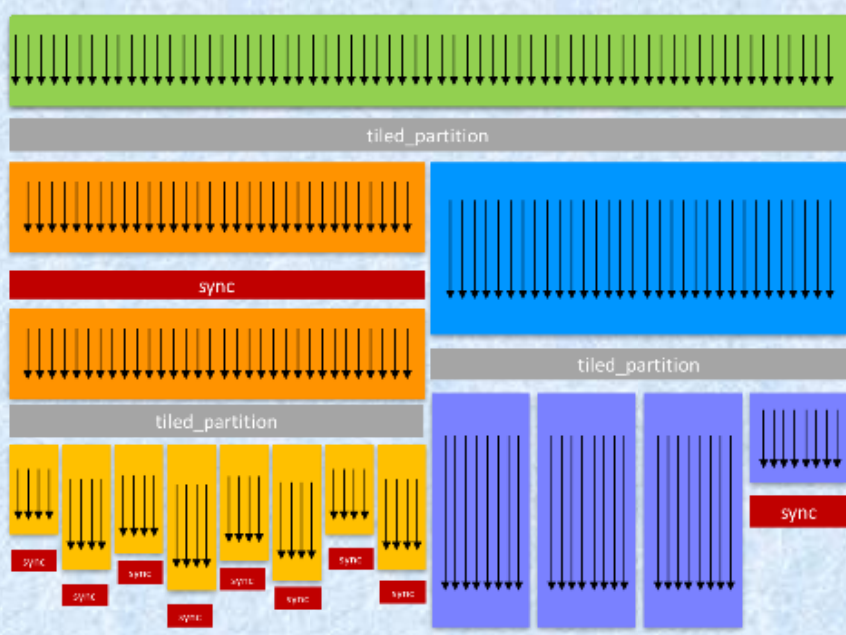
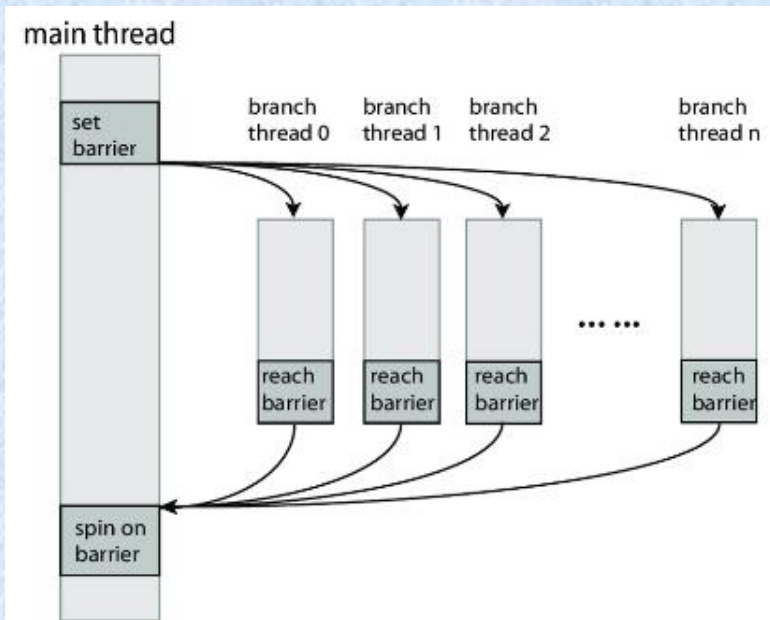


- (16, 4, 8)
- (8, 8, 8)
- ...
- (64, 2, 4)
- (32, 4, 4)
- ...
- (32, 32, 4)

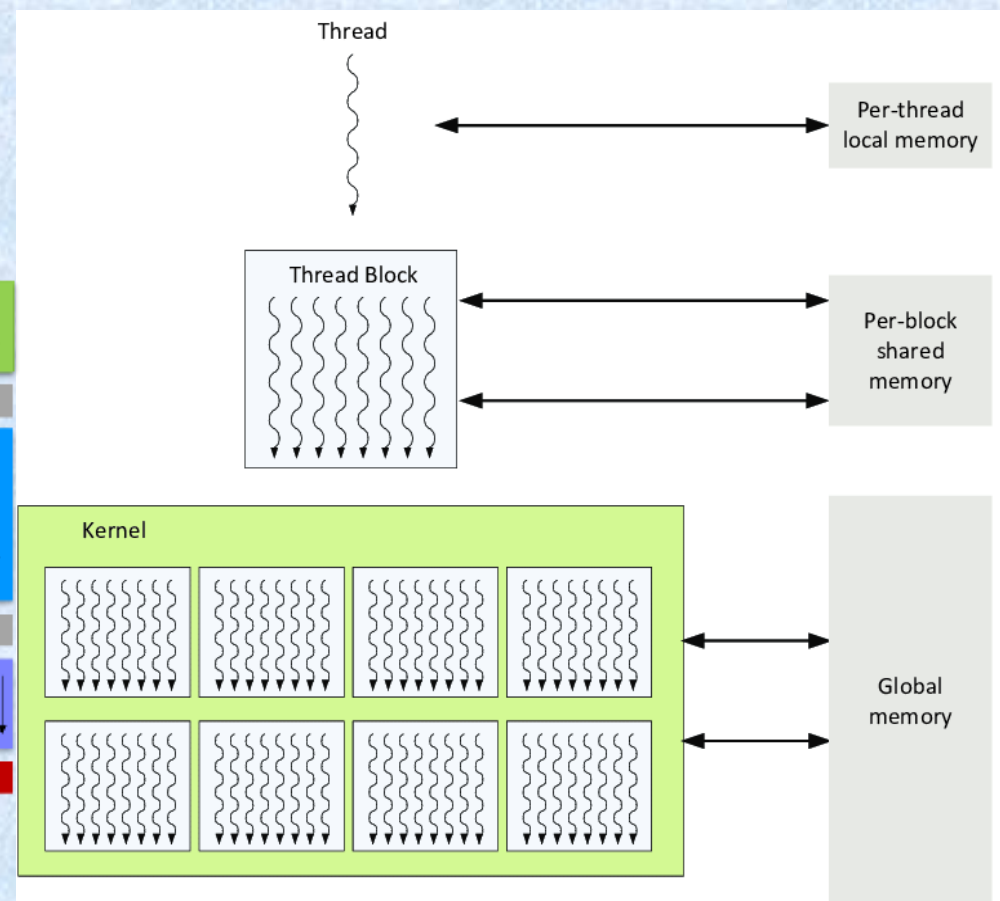
Threads block



- A *thread block* is a group of threads that can cooperate with each other by:
 - Block-local synchronization
 - Block-local shared memory
- Threads of different blocks can cooperate as Cooperative Groups (starting from CUDA 9.0 and Compute Capability 3.0+)
- All the threads in a **grid** share the same **global memory space**
- Threads are uniquely identified by the following two coordinates:
 - blockIdx** (block index in the **grid**), **threadIdx** (index of threads in the **block**)
- The coordinates are pre-initialized **uint3** type and can be accessed within the kernel
- When a kernel works, blockIdx and threadIdx are assigned to threads by CUDA runtime
- Each component in a variable of type uint3 is accessible through the fields x, y, z
 - blockIdx.x, blockIdx.y, blockIdx.z** **threadIdx.x, threadIdx.y, threadIdx.z**



Synchro through barriers



Synchro through memory



Threads and block dimensions

-**Grid and block sizes** are specified by the following built-in variables:

-**blockDim** (block size, measured in threads)

-**gridDim** (size of the grid, measured in blocks)

-These variables are of type **dim3**, a type of integer vector based on **uint3** concept

-When defining a variable of type **dim3**, each unspecified component is initialized to 1

-Each component in a variable of type **dim3** is accessible through the fields x, y, z

blockDim.x, blockDim.y, blockDim.z

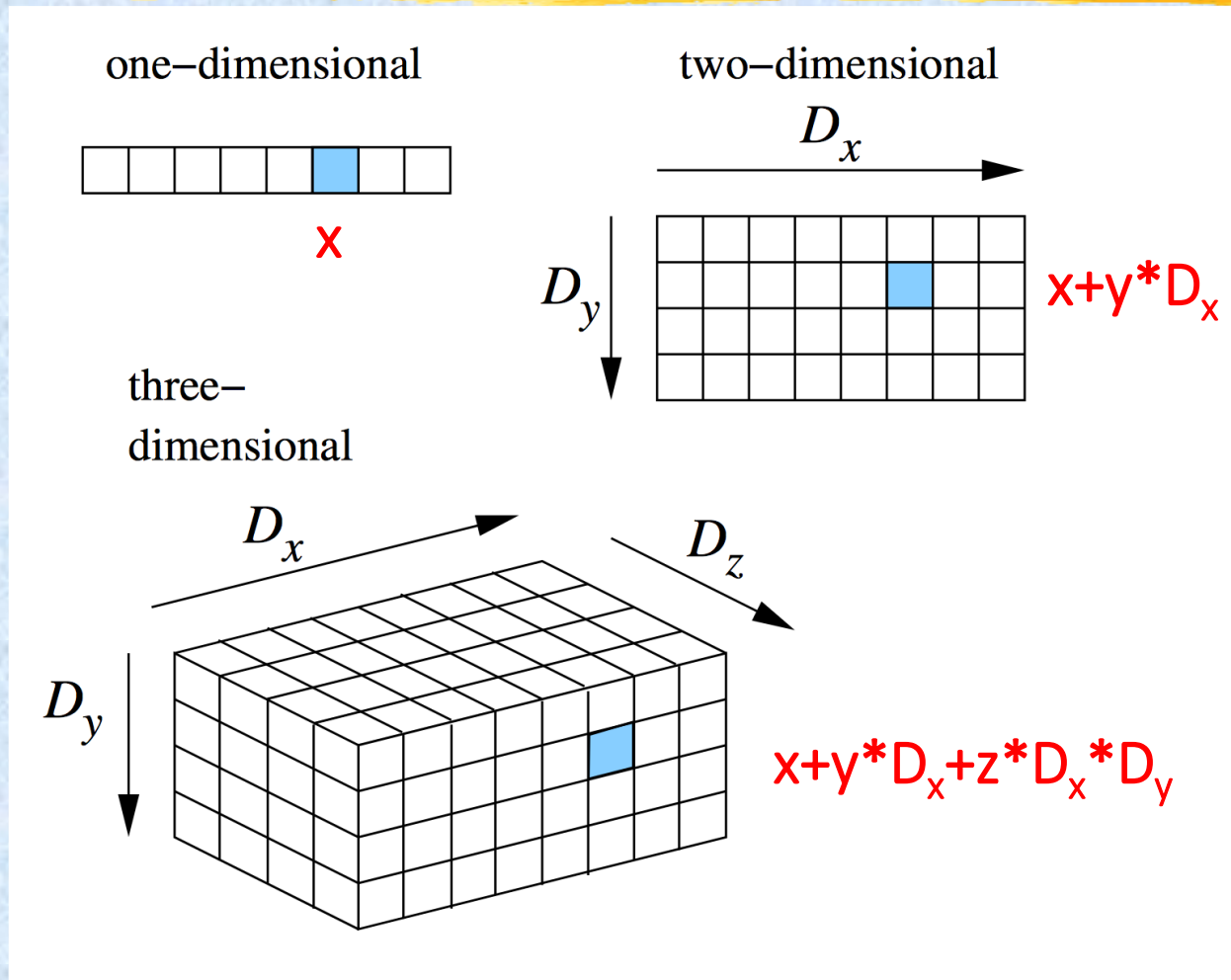
gridDim.x, gridDim.y, gridDim.z



Index in 1D, 2D, 3D blocks

Data are stored in memory linearly (sequentially)

How to determine the correct index (address)?





ID thread in 1D grids

grid 1D block 1D

$$\text{IDth} = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$$

grid 1D block 2D

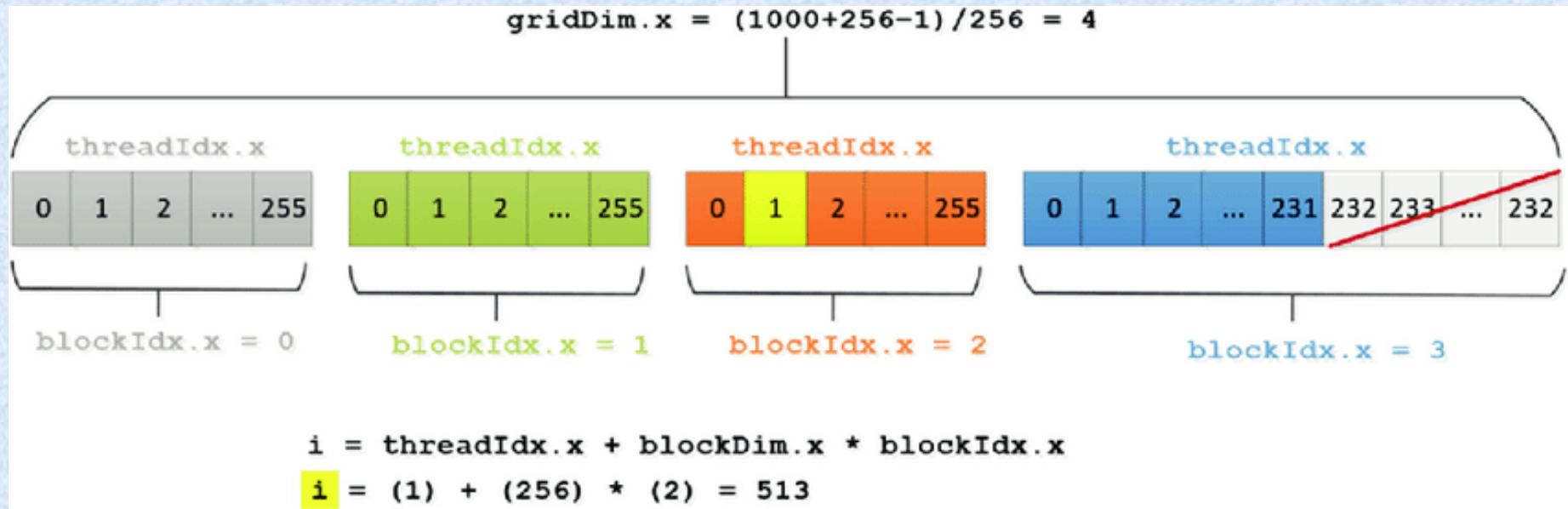
$$\begin{aligned} \text{IDth} = & \text{blockIdx.x} * \text{blockDim.x} * \text{blockDim.y} + \\ & \text{threadIdx.y} * \text{blockDim.x} + \\ & \text{threadIdx.x} \end{aligned}$$

grid 1D block 3D

$$\begin{aligned} \text{IDth} = & \text{blockIdx.x} * \text{blockDim.x} * \text{blockDim.y} * \text{blockDim.z} + \\ & \text{threadIdx.z} * \text{blockDim.y} * \text{blockDim.x} + \\ & \text{threadIdx.y} * \text{blockDim.x} + \\ & \text{threadIdx.x} \end{aligned}$$



ID thread in 1D grids





ID thread in 2D grids

grid 2D block 1D

$$\begin{aligned} \text{IDbk} &= \text{blockIdx.y} * \text{gridDim.x} + \text{blockIdx.x} && \text{(block index)} \\ \text{IDth} &= \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x} && \text{(thread index)} \end{aligned}$$

grid 2D block 2D

$$\begin{aligned} \text{IDth} = & \text{IDbk} * (\text{blockDim.x} * \text{blockDim.y}) + \\ & \text{threadIdx.y} * \text{blockDim.x} + \\ & \text{threadIdx.x} \end{aligned}$$

grid 2D block 3D

$$\begin{aligned} \text{IDth} = & \text{IDbk} * (\text{blockDim.x} * \text{blockDim.y} * \text{blockDim.z}) + \\ & \text{threadIdx.z} * (\text{blockDim.x} * \text{blockDim.y}) + \\ & \text{threadIdx.y} * \text{blockDim.x} + \\ & \text{threadIdx.x} \end{aligned}$$



ID thread in 3D grids

grid 3D block 3D

IDbk = $\text{blockIdx.x} + \text{blockIdx.y} * \text{gridDim.x} +$ (block index)

$\text{gridDim.x} * \text{gridDim.y} * \text{blockIdx.z};$

IDth = $\text{IDbk} * \text{blockDim.x} + \text{threadIdx.x};$ (thread index)

grid 3D block 1D

IDth = $\text{IDbk} * (\text{blockDim.x} * \text{blockDim.y}) +$
 $\text{threadIdx.y} * \text{blockDim.x} +$
 $\text{threadIdx.x};$

grid 3D block 2D

IDth = $\text{IDbk} * (\text{blockDim.x} * \text{blockDim.y} * \text{blockDim.z}) +$
 $\text{threadIdx.z} * (\text{blockDim.x} * \text{blockDim.y}) +$
 $\text{threadIdx.y} * \text{blockDim.x} +$
 $\text{threadIdx.x};$

CUDA kernel activation



A CUDA kernel is a direct extension of C *function* syntax with additional parameters of configuration of the execution within triple-angle brackets

`kernel_name <<< grid, block >>> (argument list);`

The first parameter is the **size** of the grid = **number of blocks** containing the threads

The second value is the block size = **number of threads** within the block

For example, suppose we have a set of 32 elements, we can group 8 elements into 4 blocks, or 32 in one block ...

`kernel_name <<< 4, 8 >>> (argument list);`

`kernel_name <<< 1, 32 >>> (argument list);`

Since data is stored linearly in global memory, we can use the built-in variables **blockIdx.x** and **threadIdx.x** to

- Identify a single thread in the grid
- Establish a map between threads and individual data elements

Concurrent kernels (2D grids and blocks)



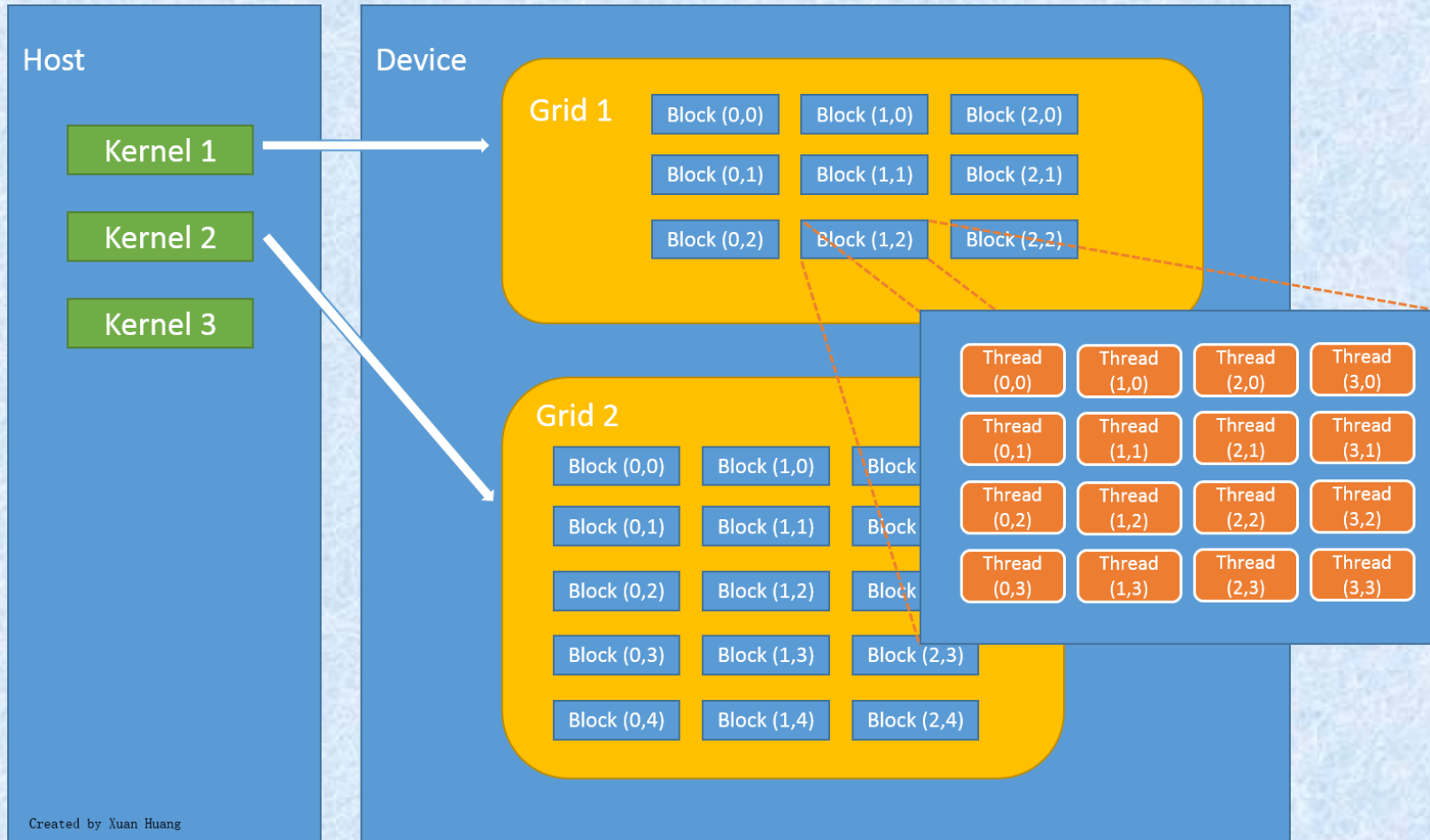
Grid 1:

- Grid (3,3)
- Block (4,4)

Grid 2:

- Grid (3, 5)
- Block (4,4)

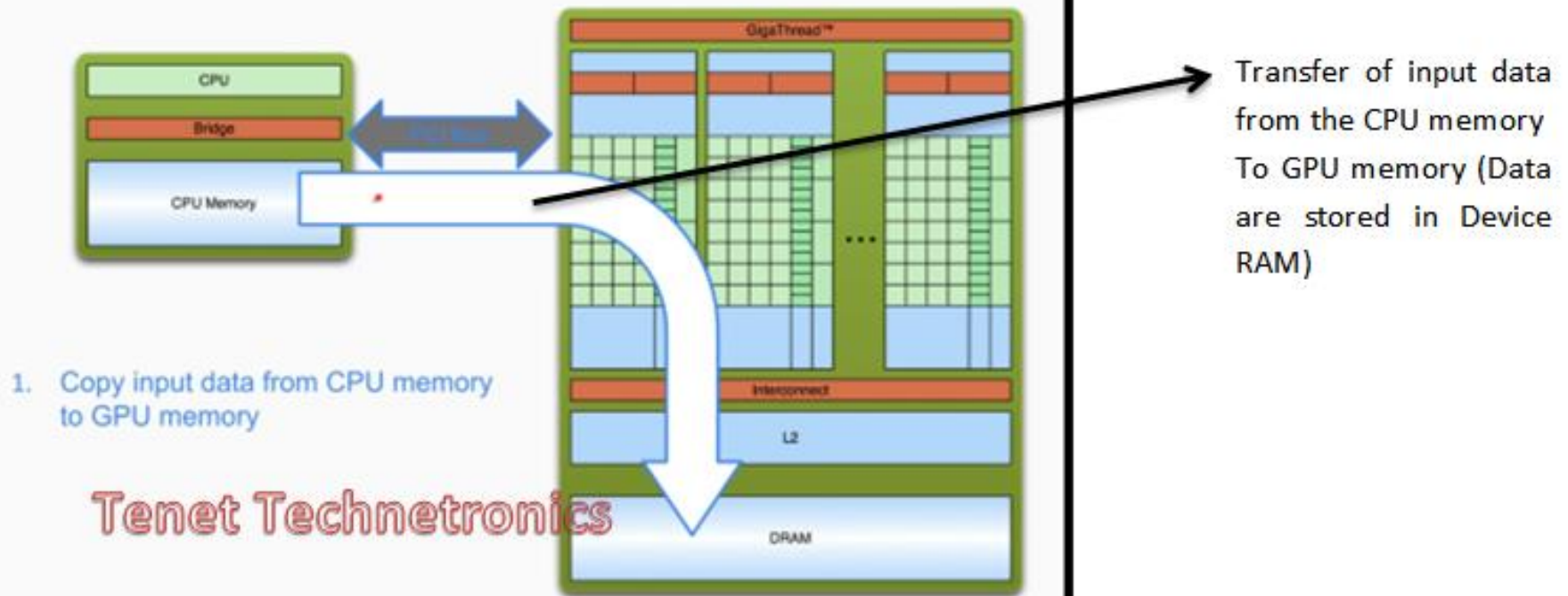
To decide if 2D,
3D we can
specify a
udim3 variable
properly
dimensioned



Processing flow



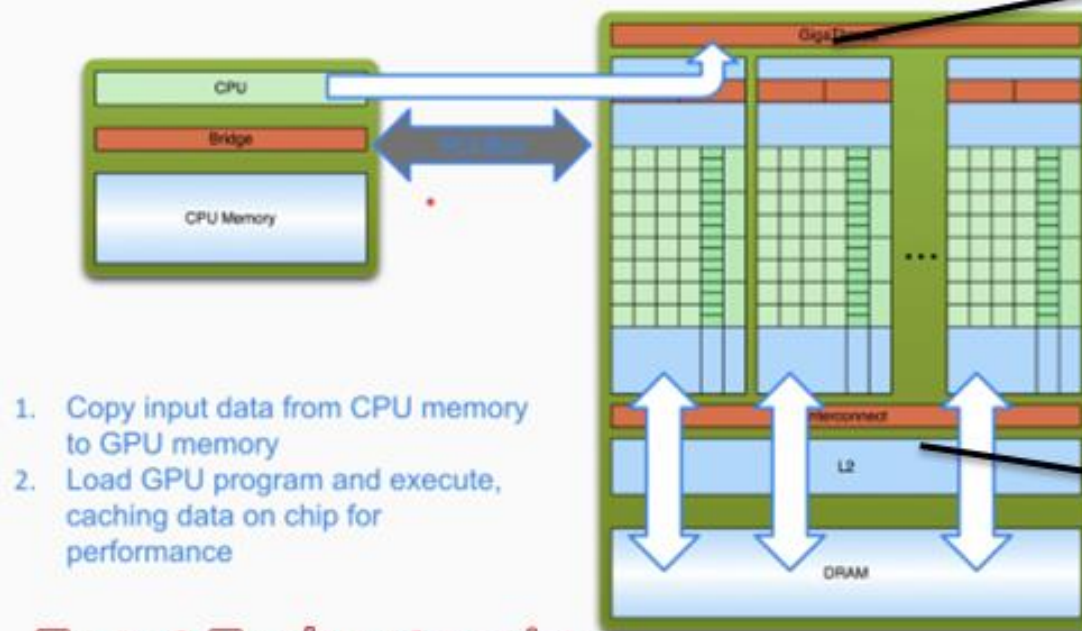
Simple Processing Flow



Processing flow



Simple Processing Flow



1. Copy input data from CPU memory to GPU memory
2. Load GPU program and execute, caching data on chip for performance

The instruction or part of the code is transferred to the GPU threads.

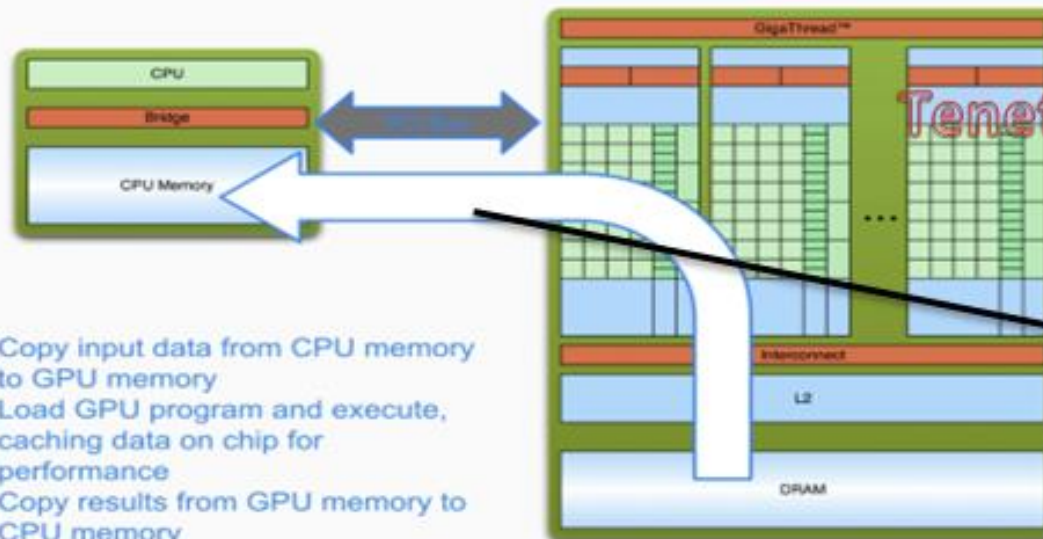
Further these instructions are executed on the data which are already loaded in the GPU memory

Tenet Technetronics

Processing flow



Simple Processing Flow



1. Copy input data from CPU memory to GPU memory
2. Load GPU program and execute, caching data on chip for performance
3. Copy results from GPU memory to CPU memory

After the computation are done, we transfer the result from GPU memory to CPU memory

CUDA kernel activation



```
int binarization(struct image *src, struct image *dst, int threshold){
    int npixel;
    npixel=src->col*src->row;
    unsigned int memsize=npixel*sizeof(int);
    dst->col=src->col;
    dst->row=src->row;
    dst->max=255;
    dst->data=(int*)malloc(npixel*sizeof(int));

    int *in, *out;

    cudaMalloc((void**)&in,memsize);
    cudaMalloc((void**)&out,memsize);

    cudaMemcpy(in,src->data,memsize,cudaMemcpyHostToDevice);

    int trd=32;
    int blocchi=npixel/32+1;

    dim3 threads(trd,1);
    dim3 bloc(blocchi,1);

    binGPU<<<bloc,threads>>>(in,out,threshold,npixel);
    cudaDeviceSynchronize();

    cudaMemcpy(dst->data,out,memsize,cudaMemcpyDeviceToHost);

    cudaFree(in);
    cudaFree(out);
}
```

```
__global__ void binGPU(int *in, int *out, int threshold, int npixel){
    int idx=blockIdx.x*blockDim.x+threadIdx.x;
    if(idx>npixel)
        return;

    if(in[idx]<threshold)
        out[idx]=0;
    else
        out[idx]=255;
}
```

Verifying and debugging



Beyond advanced debugging tools two simple alternatives can be used

Printf (available in devices from Fermi on)

Kernel <<<1,1>>> that is single block and single thread execution as in serial procs
but

cudaDeviceSynchronize or **cudaDeviceReset** are needed to have a correct printf on
the standard output ...

Warning (by NVIDIA)

«Kernel calls are asynchronous, i. e. control is returned to the calling (CPU) thread before the kernel completes. If the application terminates shortly thereafter, then the kernel will have no opportunity to print»



API runtime Synchronize

```
__host__ __device__ cudaError\_t cudaDeviceSynchronize ( void )
```

Wait for compute device to finish.

Returns

[cudaSuccess](#)

Description

Blocks until the device has completed all preceding requested tasks. [cudaDeviceSynchronize\(\)](#) returns an error if one of the preceding tasks has failed. If the [cudaDeviceScheduleBlockingSync](#) flag was set for this device, the host thread will block until the device has finished its work.

Note:

Note that this function may also return error codes from previous, asynchronous launches.

See also:

[cudaDeviceReset](#), [cuCtxSynchronize](#)

API runtime `cudaDeviceReset`



`__host__ cudaError\_t cudaDeviceReset ( void )`

Destroy all allocations and reset all state on the current device in the current process.

Returns

[cudaSuccess](#)

Description

Explicitly destroys and cleans up all resources associated with the current device in the current process. Any subsequent API call to this device will reinitialize the device.

Note that this function will reset the device immediately. It is the caller's responsibility to ensure that the device is not being accessed by any other host threads from the process when this function is called.

Note:

Note that this function may also return error codes from previous, asynchronous launches.

See also:

[cudaDeviceSynchronize](#)

Kernel properties



QUALIFIERS	EXECUTED BY	INVOKED BY	NOTES
<code>_global_</code>	DEVICE	HOST AND DEVICE (from cc 3)	Returns void
<code>_device_</code>	DEVICE	DEVICE	
<code>_host_</code>	HOST	HOST	Can be omitted

`_global_` and `_device_` can be used together and in this case the code is compiled both for the host and for the device



Kernel restrictions

It accesses only to the memory of the device

Must return a void value

Does not support a variable number of arguments (argc, argv ...)

Does not support static variables

Does not support function pointers (void, no static dynamic)

Is asynchronous to the host



Runtime vs Driver API

The driver and runtime APIs are very similar and can for the most part be used interchangeably

Complexity vs. control

- The runtime API eases device code management by providing:
 - **implicit initialization, context management, and module management**
 - **simpler code**, but it also **lacks** the level of **control** that the driver API has
- The driver API offers more **fine-grained control**, especially over contexts and module loading
 - **Kernel launches** are much **more complex** to implement, as the execution configuration and kernel parameters must be specified with **explicit function calls**
 - However, unlike the **runtime**, where all the kernels are **automatically loaded** during initialization and stay loaded for as long as the program runs, with the **driver** API it is possible to only keep the modules that are **currently needed loaded**, or even dynamically reload modules
 - driver API is also **language-independent** as it only deals with **cubin objects**

SW levels in CUDA



All levels operate with user privileges except the final one of the driver operating in kernel mode

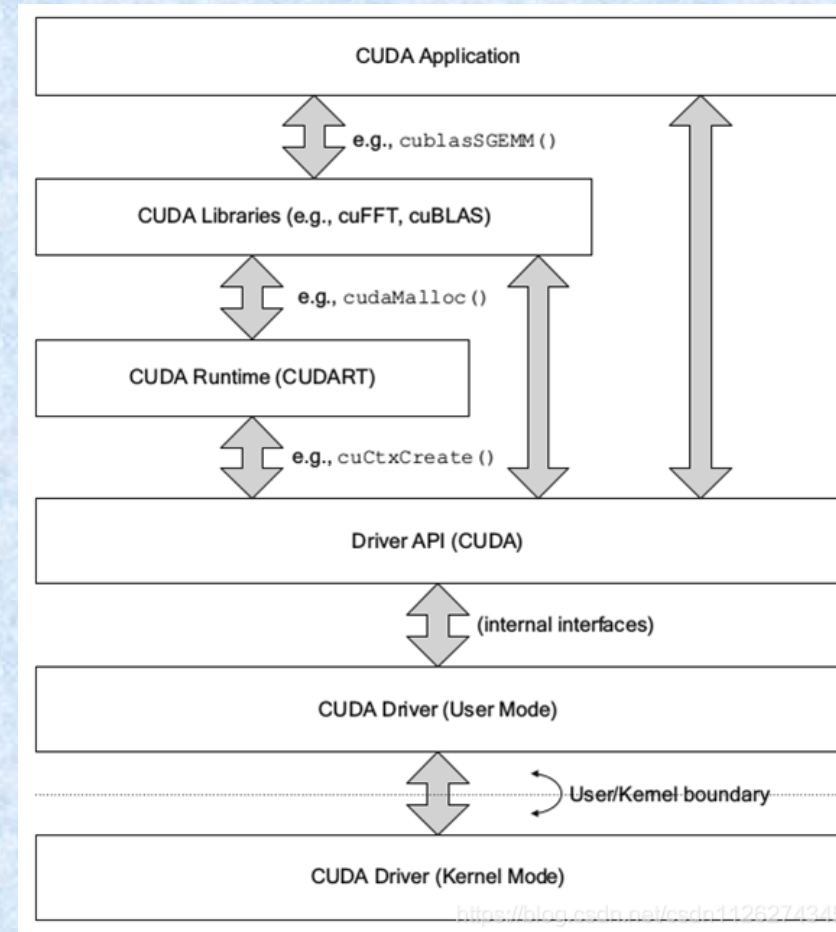
In CUDA host and device memory allocated to a CUDA program cannot be accessed from another CUDA program

The CUDA runtime is the library supporting the C++ / GPU toolchain integrated into CUDA

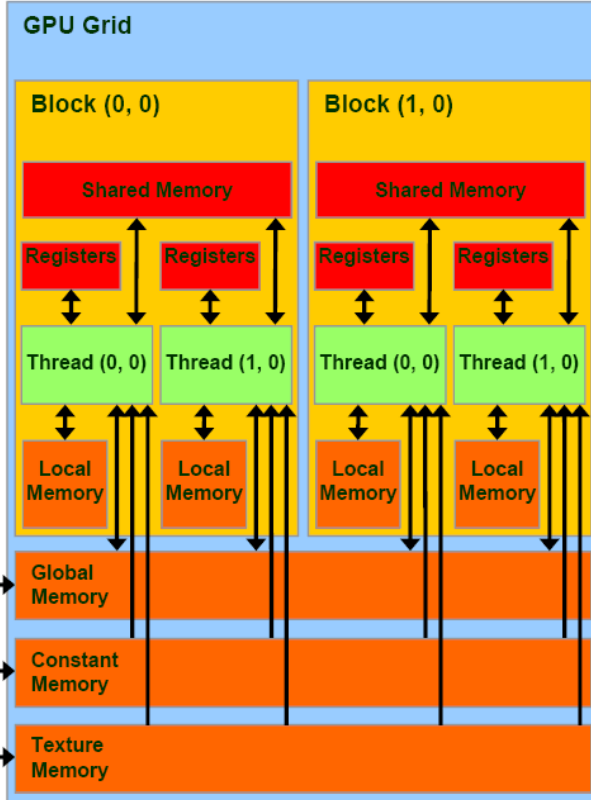
The nvcc compiler splits the .cu files into portions of host and device code: the portion redirected to the host automatically generates calls to CUDA runtime to facilitate operations such as launching a kernel

The CUDA driver maintains compatibility towards the low, supporting programs independently from the version of CUDA or old programs

Export low-level “API driver” (in cuda.h) allows direct use of resources that you interface to kernel mode



Memory management



Implicit synchronization between host and device. When `cudaMemcpy` is used the host waits for data completion.

Standard C functions

`malloc`

`memcpy`

`memset`

`free`

CUDA C functions

`cudaMalloc`

`cudaMemcpy`

`cudaMemset`

`cudaFree`

Global, constant and texture (specialized RAM for 3D graphics and video rendering) memories **persist** across kernels called by the same host application

The host can R/W global, constant and texture memories

Each thread can R/W its own local memory and registers, shares (R/W) the Block shared memory and the Grid global (R/W), constant and texture memories (read only).

Memory management

Standard C functions	CUDA C functions
malloc	cudaMalloc
memcpy	cudaMemcpy
memset	cudaMemset
free	cudaFree



cudaError_t cudaMalloc (void devPtr, size_t size)**

devPtr is a pointer to a global memory pointer
Double pointer since host and device memory areas are different and
if we use a normal pointer the operative system will impede this

cudaError_t cudaMemcpy (void* dst, const void* src, size_t count, cudaMemcpyKind kind)
the host program is blocked until the completion of the memory transfer

The variable **kind** specifies where destination and source point, we have four possibilities

cudaMemcpyHostToHost, cudaMemcpyHostToDevice, cudaMemcpyDeviceToHost,
cudaMemcpyDeviceToDevice

cudaError_t cudaFree (void* devPtr)

free the global memory pointed by devPtr

Error management



typedef enum cudaerror **cudaError_t** (error types)

every call in Cuda returns a *cudaError_t* code (except the kernel)

Ex. *Success* = **cudaErrorMemoryAllocation**

cudaError_t **cudaGetLastError**(void) returns the code of the last error

char ***cudaGetErrorString**(cudaError_t)

Ex. `printf("%s\n", cudaGetErrorString(cudaGetLastError()));`



Allocation on host and device

CPU host allocation

```
float *h_A, *h_B, *h_C;  
h_A = (float *) malloc(nBytes);  
h_B = (float *) malloc(nBytes);  
h_C = (float *) malloc(nBytes);
```

GPU device allocation

```
float *d_A, *d_B, *d_C;  
cudaMalloc((void **) &d_A, nBytes);  
cudaMalloc((void **) &d_B, nBytes);  
cudaMalloc((void **) &d_C, nBytes);
```

Notice that in the host we address the virtual memory in the GPU the physical one.

CudaMalloc must update the single pointer (e.g., `float *d_ptr`) in “host” (CPU) code so that it points to the new memory address just allocated on the GPU. If we pass only `d_ptr` (a single pointer), the function would receive a copy of the address and would not be able to modify the original one. By passing the pointer address (`&d_ptr`), *CudaMalloc* can dereference this address (`*(&d_ptr)`) and assign it the new value of the GPU memory address.

Malloc wrapping



Wrapping: to create personalized functions and not standard ones

Cast void** allocation

A wrapper double pointer must be used

Correct version

```
int *p = NULL;
```

```
void myMalloc((void *) p, int size) {  
    p = malloc(size);  
    return;  
}
```

```
myMalloc ...  
p[0] = 0; // SEGMENTATION FAULT
```

```
int *p = NULL;
```

```
void myMalloc((void **) p, int size) {  
    *p = malloc(size);  
    return;  
}
```

```
myMalloc((void**) &ptr, size);  
ptr[0] = 0;
```



Host and device pointers

A common error is to confuse (mix) the **pointers** of two different CPU and GPU **memory areas**

A device memory pointer cannot be de-referenced in the host code and viceversa

A wrong assignment could be:

```
gpuRef = d_C // ERROR!
```

Instead of using:

```
cudaMemcpy(gpuRef, d_C, nBytes, cudaMemcpyHostToDevice)
```

This kind of errors do not happen in the **Unified Memory** version (introduced in CUDA 6) that allows the access to the CPU and GPU memories using a single pointer



Example: sum of vectors

```
#include <stdio.h>
```

```
...
```

```
__global__ void add_vect(...)
```

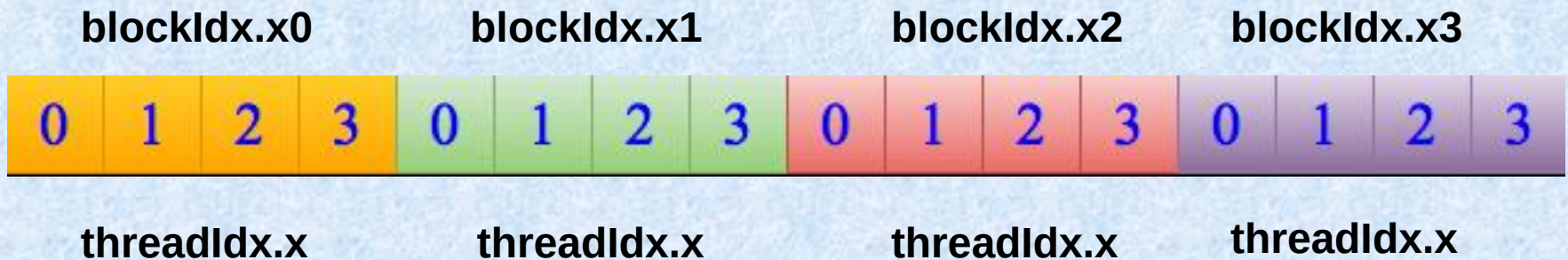
```
...
```

- Allocate the device memory for h_a, h_b e h_c (the vectors)
- Launch the kernel with h_a, h_b e h_c as arguments
- Copy from the device memory to the host memory the results in h_c
- Free the host and device memories with **cudaFree(*ptr);**

Once the data size is known the steps to fix **grid** and **block dimensions** are:

- Define the **block size** ($\leq$ of 1024)
- Calculate the **grid size** on the basis of the data dimension
- For some devices the maximum dimension per every x, y, and z *grid* is 65535
- Set the block dimension considering:
 - **Kernel Performance**
 - GPU Resources **limitations**

1D grid: 4 blocks each with 4 threads



Linear index

```
int idx = blockDim.x * blockIdx.x + threadIdx.x;
```



Sum of vectors: kernel

```
#include <stdio.h>
```

```
#include <cuda_runtime.h>
```

```
#define N 1024*1024 // vector size
```

```
#define TxB 32 // threads x block
```

```
/* kernel: vector add */
```

```
__global__ void add_vect(int *a, int *b, int *c) {
```

```
    int idx = blockDim.x * blockIdx.x + threadIdx.x;
```

```
    if (idx < N)  /* the thread number could be different from the data size */  
        c[idx] = a[idx] + b[idx];
```

```
}
```




Sum of vectors: memory allocation

```
int main(void) {  
    int *a, *b, *c;  
    int *dev_a, *dev_b, *dev_c;  
    int nBytes = N * sizeof(int);  
  
    // malloc host memory  
    a = (int *) malloc(nBytes); b = (int *) malloc(nBytes); c = (int *) malloc(nBytes);  
  
    // malloc device memory  
    cudaMalloc((void**) &dev_a, nBytes);  
    cudaMalloc((void**) &dev_b, nBytes);  
    cudaMalloc((void**) &dev_c, nBytes);  
    ...  
  
    // Free host memory  
    free(a); free(b); free(c);  
  
    // free the memory allocated on the GPU  
    cudaFree(dev_a); cudaFree(dev_b); cudaFree(dev_c);  
}
```



Sum of vectors: calculation

...

// fill the arrays 'a' and 'b' on the CPU

```
for (int i = 0; i < N; i++) {  
    a[i] = rand() % 10;  b[i] = rand() % 10;  
}
```

// copy the arrays 'a' and 'b' to the GPU

```
cudaMemcpy(dev_a, a, nBytes, cudaMemcpyHostToDevice);  
cudaMemcpy(dev_b, b, nBytes, cudaMemcpyHostToDevice);
```

```
add_vect <<<N / TxB, TxB>>> (dev_a, dev_b, dev_c);    /*kernel*/
```

// copy the array 'c' back from the GPU to the CPU

```
cudaMemcpy(c, dev_c, nBytes, cudaMemcpyDeviceToHost);
```

// display the results

```
for (int i = 0; i < N; i++) {  
    printf("%d + %d = %d\n", a[i], b[i], c[i]);  
}
```

...

Error management



Since many CUDA **calls** are **asynchronous** it's often difficult to identify what function or routine is responsible for an error

A suitable **macro** devoted to errors management to **wrap the** CUDA API calls simplifies the process:

...

```
#define CHECK(call) {  
    const cudaError_t error = call;  
    if (error != cudaSuccess) {  
        printf("Error: %s:%d, ", __FILE__, __LINE__);  
        printf("code:%d, reason: %s\n", error, cudaGetErrorString(error));  
        exit(1);  
    }  
}
```

Then we can execute

```
CHECK(cudaMemcpy(d_C, gpuRef, nBytes, cudaMemcpyHostToDevice));
```

To debug the kernel we can use

```
kernel_function<<<grid, block>>>(argument list);  
CHECK(cudaDeviceSynchronize());
```




Time measuring

gettimeofday returns the overall clock time (include sys/time.h)

```
double cpuSecond() {  
    struct timeval tp;  
    gettimeofday(&tp, NULL);  
    return ((double)tp.tv_sec + (double)tp.tv_usec*1.e-6);  
}
```

// measurement of the kernel execution time

```
double iStart = cpuSecond();
```

```
kernel_name<<<grid, block>>>(argument list);
```

```
cudaDeviceSynchronize();    /* CPU waits until all tasks completed */
```

```
double iElaps = cpuSecond() - iStart;
```

```
struct timeval {  
    long tv_sec; /*seconds since 1/1/1970*/  
    long tv_usec; /*microseconds since tv_sec*/  
};
```



And profiling

Since CUDA 5.0 a profiling tool (**nvprof**) is included to achieve the timeline of CPU and GPU activities: kernel execution, memory transfers, CUDA functions calls, API, etc...

\$ nvprof [nvprof_args] <application> [application_args]

VisualC provides a similar option (graphical)

```
$ nvprof add_vect
==8132== Profiling application: a.out
==8132== Profiling result:
Time(%)      Time      Calls      Avg      Min      Max      Name
37.38%    7.3920us        1    7.3920us    7.3920us    7.3920us    add_vect(int*, int*, int*)
32.36%    6.4000us        2    3.2000us    2.8480us    3.5520us    [CUDA memcpy HtoD]
30.26%    5.9840us        1    5.9840us    5.9840us    5.9840us    [CUDA memcpy DtoH]

==8132== API calls:
Time(%)      Time      Calls      Avg      Min      Max      Name
99.73%    174.54ms         3    58.179ms    3.7540us    174.53ms    cudaMalloc
0.10%     172.88us         3    57.625us    6.9580us    149.96us    cudaFree
0.07%     115.60us        83    1.3920us    100ns     53.943us    cuDeviceGetAttribute
0.06%     103.47us         3    34.490us    18.391us    49.043us    cudaMemcpy
0.03%      46.886us         1    46.886us    46.886us    46.886us    cudaLaunch
0.01%      17.483us         1    17.483us    17.483us    17.483us    cuDeviceGetName
0.00%       8.0370us         1     8.0370us    8.0370us    8.0370us    cuDeviceTotalMem
0.00%       4.7450us         3     1.5810us    267ns     3.8460us    cudaSetupArgument
0.00%       3.3510us         1     3.3510us    3.3510us    3.3510us    cudaConfigureCall
0.00%       1.0780us         2         539ns    531ns     547ns    cuDeviceGetCount
0.00%         697ns         2         348ns    339ns     358ns    cuDeviceGet
```

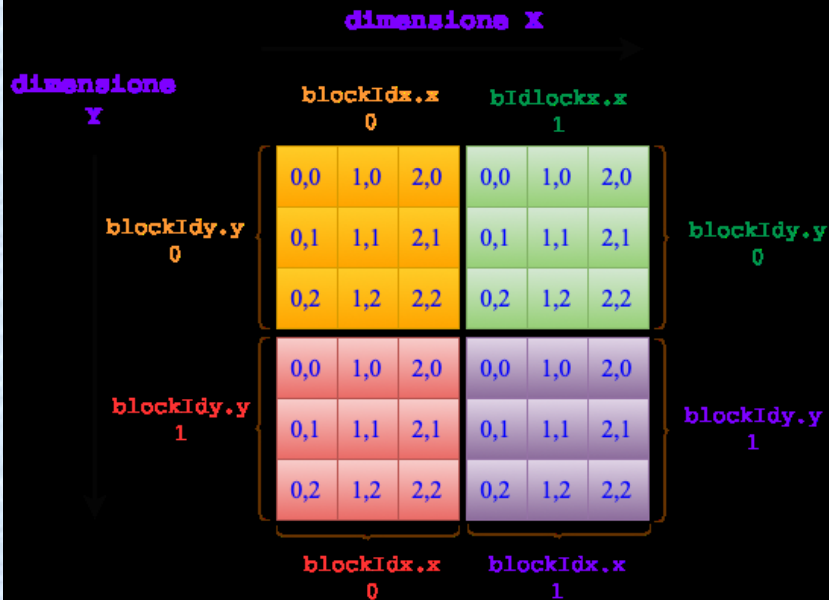
2D Grids



```
grid 2 x 2
block 3 x 3
```

```
gridDim = (2,2,1)
blockDim = (3,3,1)
```

```
threadIdx.x = 0,1,2
threadIdx.y = 0,1,2
```



dimensione X = blockDim.x * blockIdx.x + threadIdx.x
dimensione Y = blockDim.y * blockIdx.y + threadIdx.y

0,0	1,0	2,0	0,0	1,0	2,0	0,2	1,2	2,2	0,2	1,2	2,2
X=0		X=2	X=3		X=5	X=0		X=2	X=3		X=5
Y=0		Y=0	Y=0		Y=0	Y=2		Y=2	Y=3		Y=2

0,0	1,0	2,0	0,0	1,0	2,0	0,2	1,2	2,2	0,2	1,2	2,2
X=0		X=2	X=3		X=5	X=0		X=2	X=3		X=5
Y=3		Y=3	Y=3		Y=3	Y=3		Y=5	Y=5		Y=5

2D Grids



In 2D we have 3 kind of indexes:

- Block and thread indexes
- Matrix element
- Global linear memory index (offset)

The first step is to pass from 2D thread coordinates to the (ix,iy) of the matrix

```
int ix = blockDim.x * blockIdx.x +  
threadIdx.x;  
int iy = blockDim.y * blockIdx.y +  
threadIdx.y;
```

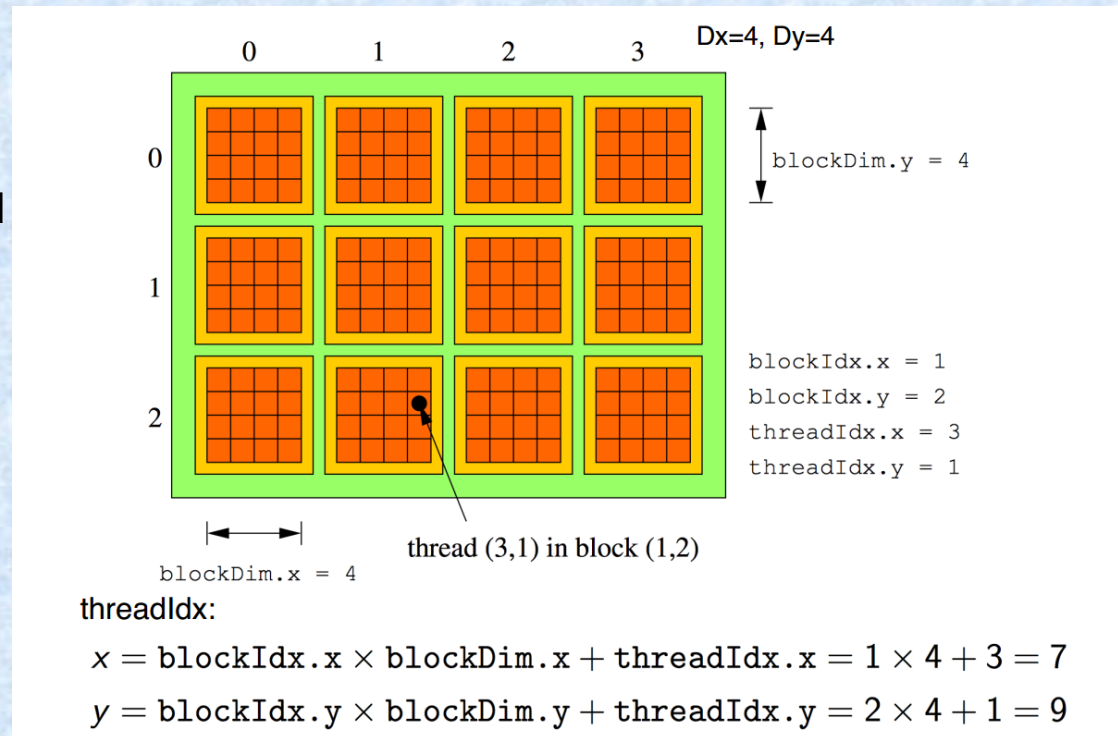
Then evaluate the linear index

```
idx = iy * nx + ix
```

If the grid dimension does not fit with the matrix one, a control is required

```
if (ix < nx) ...
```

```
if (iy < ny) ...
```





Sum of matrices: kernel

kernel: matrix add */

```
__global__ void add_matrix(int *A, int *B, int *C, int nx, int ny) {
```

```
int idx = blockDim.x * blockIdx.x + threadIdx.x;
```

```
int idy = blockDim.y * blockIdx.y + threadIdx.y;
```

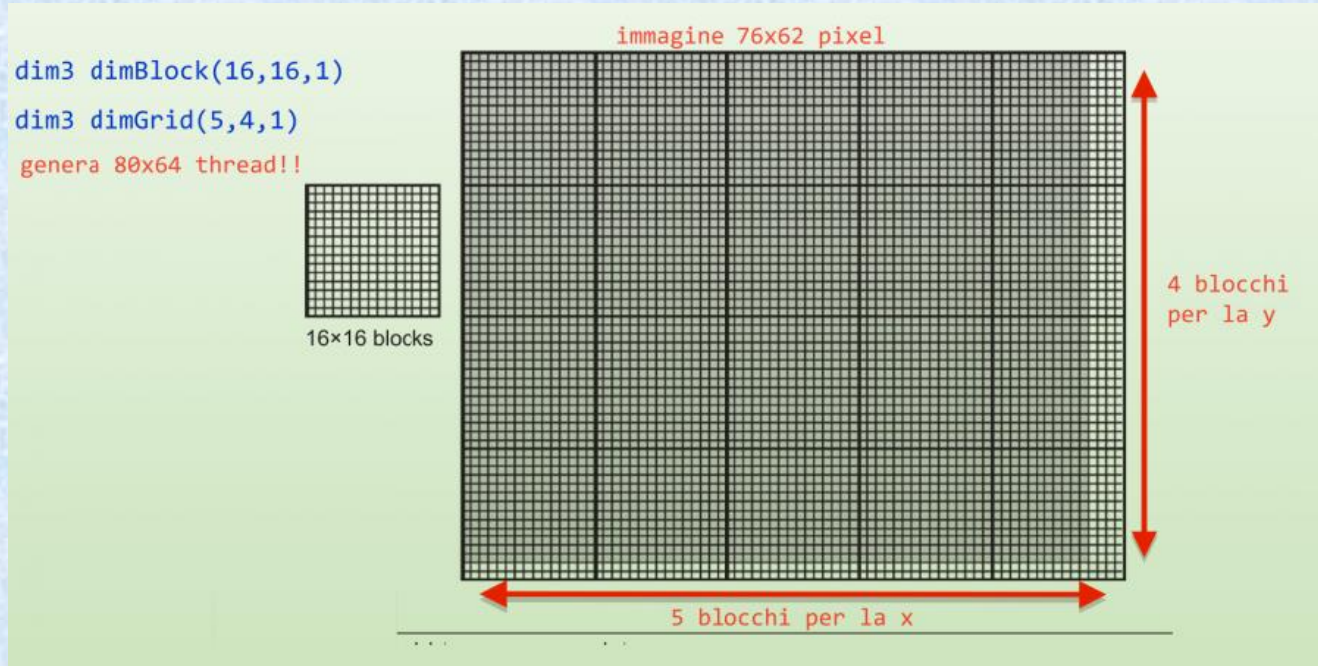
```
int id_elem = idy * nx + idx
```

```
if (idx < nx & idy < ny)
```

```
    C[id_elem] = A[id_elem] + B[id_elem];
```

```
}
```

Blocks and matrices sizes



Let's represent an image (76x62 pixels) through a matrix

We define fixed size blocks (16x16) hence the grid (5x4). This leads to 5120 threads.

If the image does not fit perfectly (in this case $76 \times 62 = 4712$ pixels) controls between threads indexes and matrix ones must be implemented.

CUDA EXECUTION MODEL

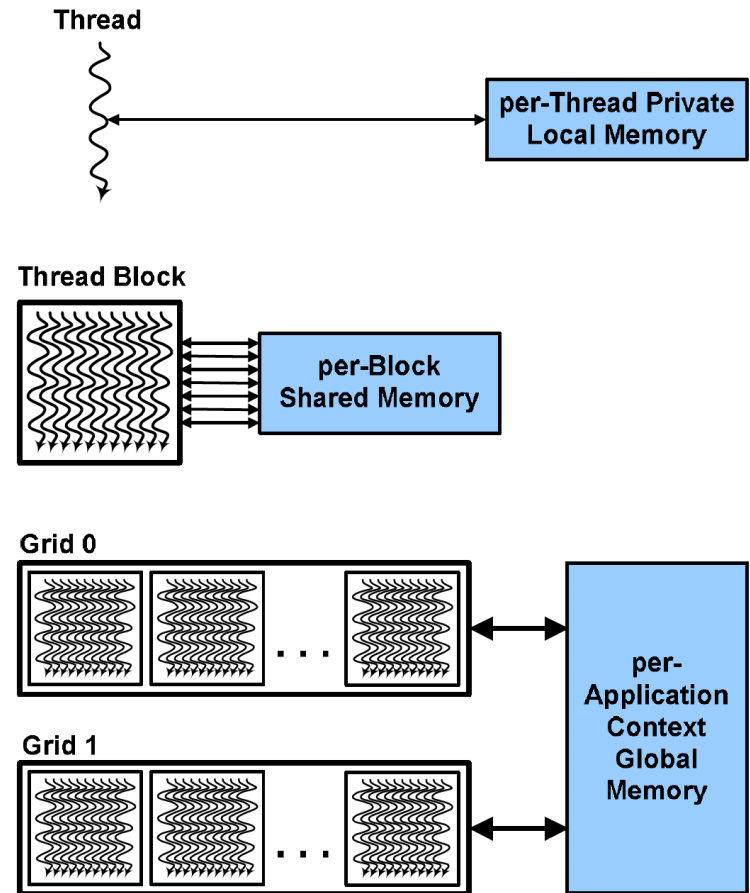


Each **thread** runs on a CUDA core and has its own private memory space for registers, function calls and automatic C variables

A thread **block** is a group of threads processed concurrently and cooperating by means of **synchronization barriers**

A thread block uses **shared memory** for inter-thread communication and data sharing

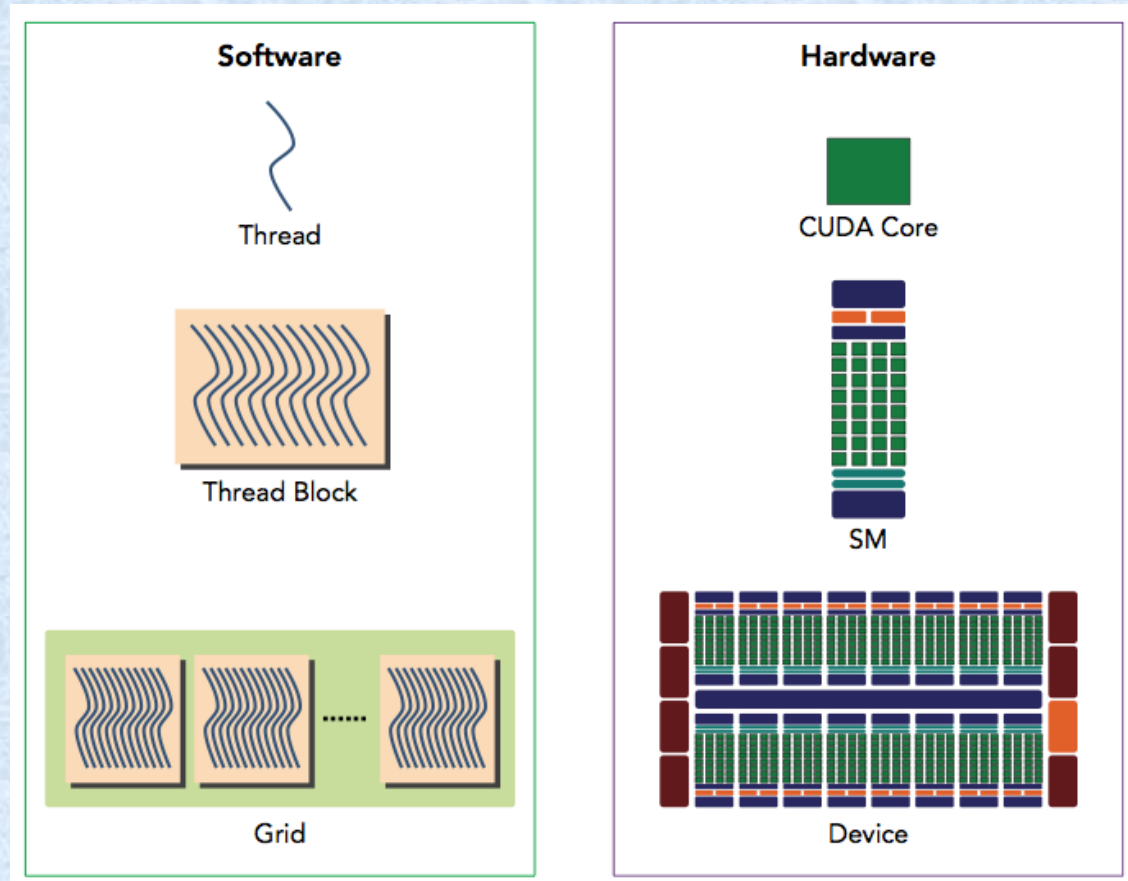
A **grid** is an array of thread blocks running the same kernel, reading and writing global memory and synchronizing kernel calls reciprocally dependent



CUDA Hierarchy of threads, blocks, and grids, with corresponding per-thread private, per-block shared, and per-application global memory spaces.



THREAD LOGICAL MAPPING



PROCESSING HIERARCHY



A *thread hierarchy* runs on an array of Streaming Multiprocessors (*processors hierarchy*)

GPU runs one or more kernels

Each SM runs one or more thread *blocks*

A thread block is scheduled *on only one* SM

In every SM the threads are processed by cores or other functional units

Threads are grouped in *warps* (32 threads each)

SIMT ARCHITECTURE



Warp = 32 threads (with consecutive ID!)

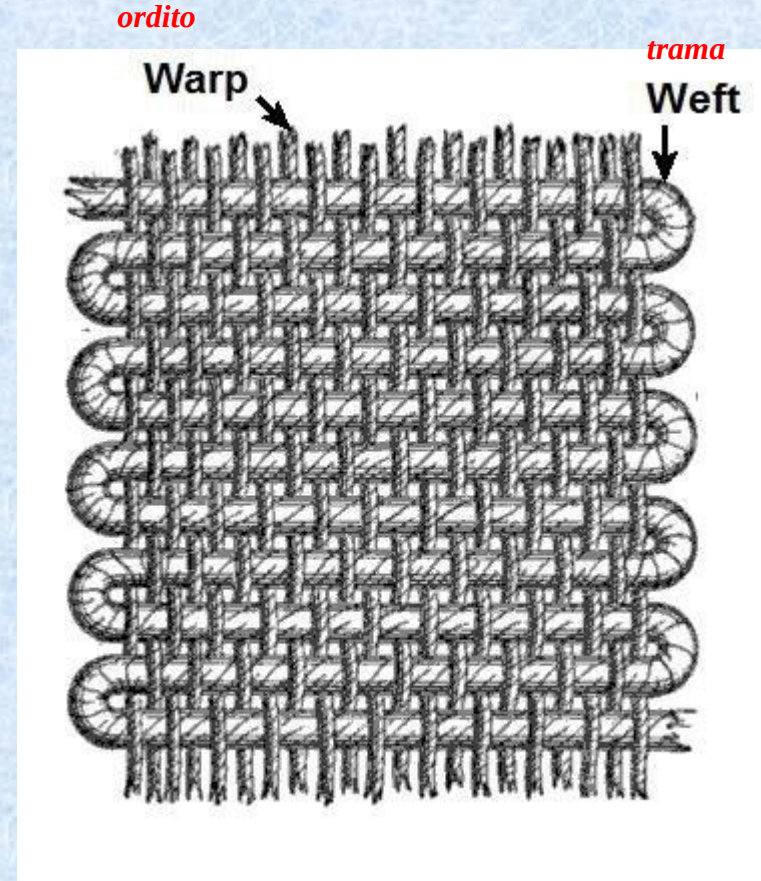
Ideally all the threads of a warp run in parallel (SIMD model)

Every thread has its own **program counter** and **state register** executing the current instruction on its assigned data

Every thread can execute the instructions in a distinct way from the others (*thread level parallelism*)

Threads belonging to the same warp start at the same program address

SIMT: Single Instruction Multiple Thread



HW MULTI THREADING



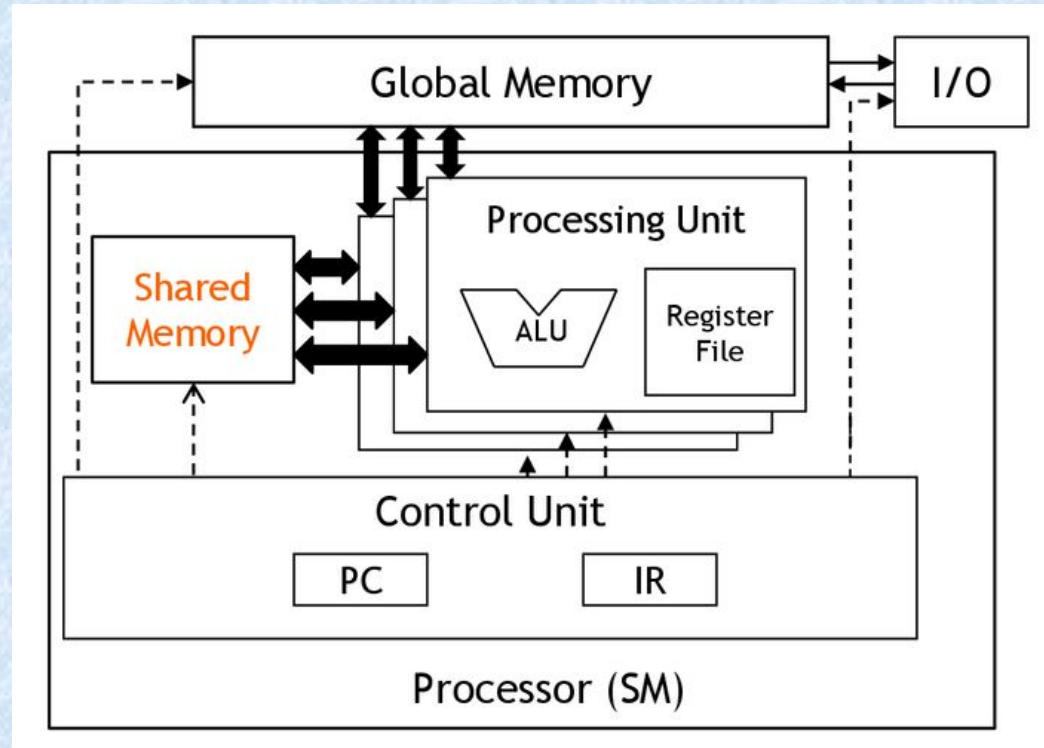
Each warp has a runtime *execution context* transparent to the programmer with:

- program counter
- 32-bit registers split between threads
- memory shared between blocks

The shared memory allows threads in the block to exchange information

The data in the registers are private for the *thread* and their *lifetime* is that one of the kernel

The warp context is kept on-chip for the duration of the warp (free context switch)



REGISTERS



Registers contain automatic local variables (scalars not array) and the coordinates of the threads (i. e. *Row*, *Col* and *val* in the kernel below)

```
__global__ void matrix_prod(float* A, float* B, float* C) {  
  
    int Row = blockIdx.y * blockDim.y + threadIdx.y;  
    int Col = blockIdx.x * blockDim.x + threadIdx.x;  
  
    // each thread computes an entry of the product matrix  
    if ((Row < N) && (Col < M)) {  
        float val = 0;  
        for (int k = 0; k < M; k++)  
            val += A[Row * M + k] * B[k * M + Col];  
        C[Row * M + Col] = val;  
    }  
}
```

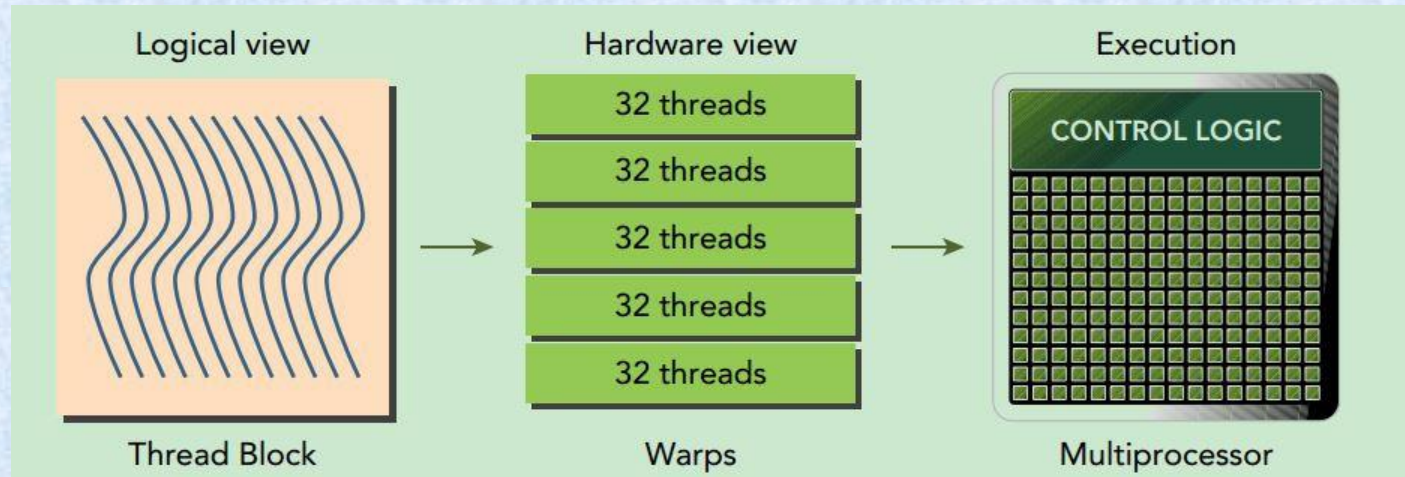

REGISTERS



Threads can be logically conceived so as they have 1D, 2D or 3D Ids

From the HW point of view they are sequentially organised with a progressive ID

A 128 thread block will be subdivided in 4 warps:



Warp 0: thread 0, thread 1, thread 2, ... thread 31
Warp 1: thread 32, thread 33, thread 34, ... thread 63
Warp 2: thread 64, thread 65, thread 66, ... thread 95
Warp 3: thread 96, thread 97, thread 98, ... thread 127

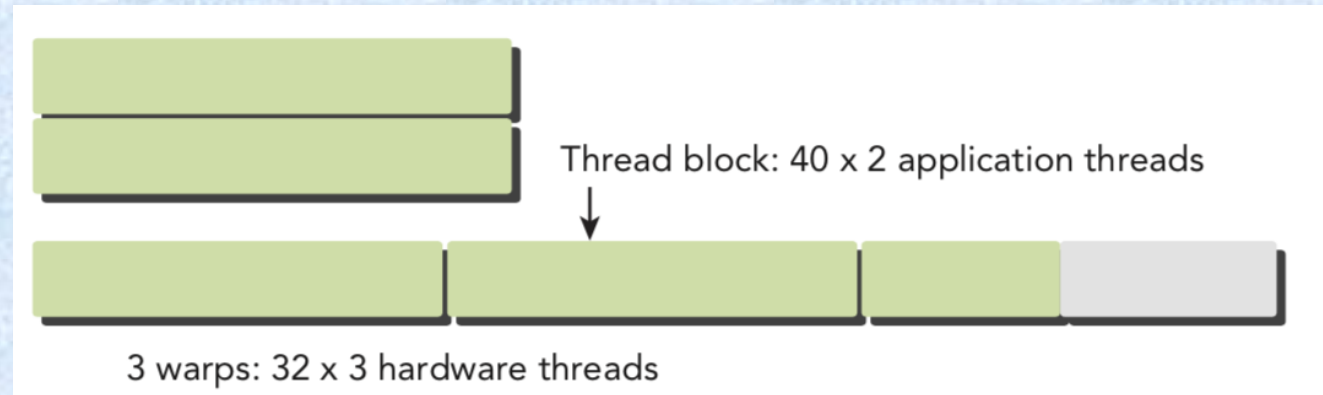


WARP ALLOCATION

Logic view: a 2D thread block with 40 threads in the x dimension and 2 in the y dimension

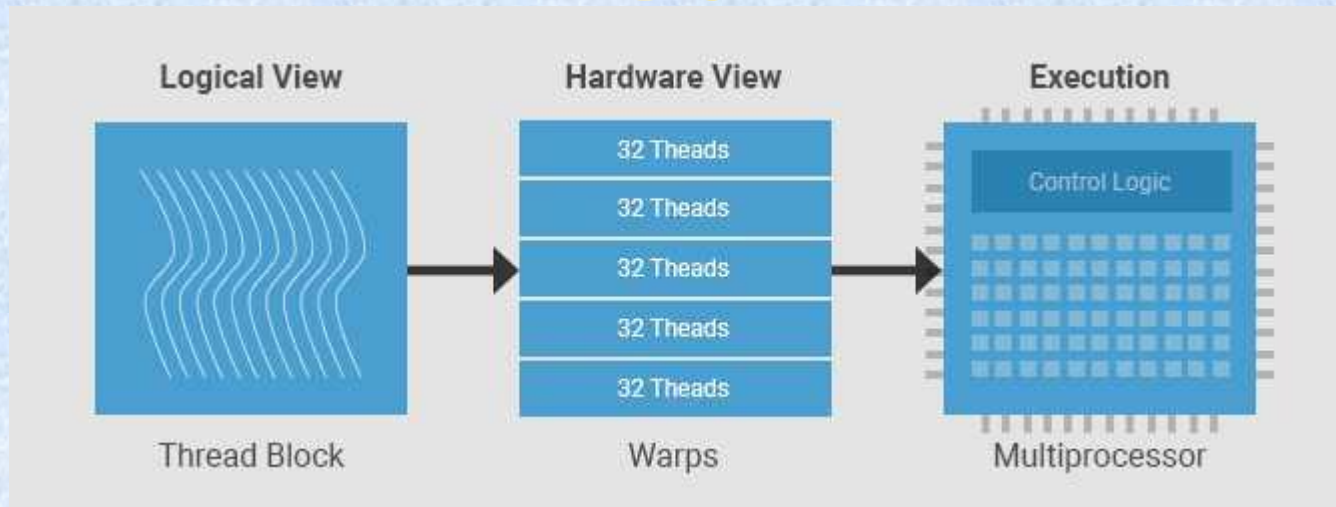
HW view: 3 HW warps will be allocated with a total of 96 threads supporting the 80 SW (logic) ones

The last warp will be partially not active but sources are allocated also in this case.



$$\#warp \times Block = \left\lceil \frac{\#thread \times Block}{warpSize} \right\rceil = \left\lceil \frac{80}{32} \right\rceil = 2$$

BLOCK SCHEDULING



Let's suppose 1000 blocks with 128 threads each

- Probably 8-12 blocks are allocated to the same SM (depends on the board generation)
- Each block has 4 warps -> 48 warps max running on each SM
- At every clock cycle the **warp scheduler** decides which warp is the next to go running
- The candidate is chosen from those that:
 - are not waiting for data from the device memory (**no memory latency**)
 - are not completing a previous instruction (**no pipeline delay**)
- All these details are transparent to the programmer

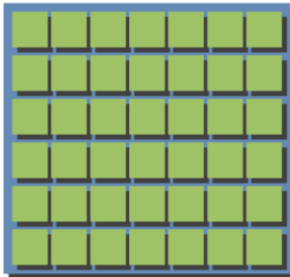
RESOURCE SHARING



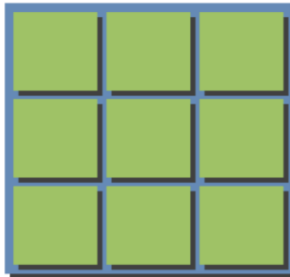
Registers per SM

Kepler: 64K

Fermi: 32K



More threads with fewer
registers per thread

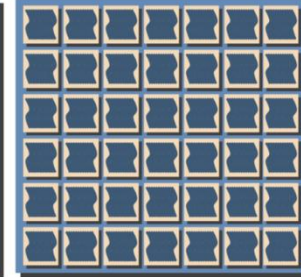


Fewer threads with more
registers per thread

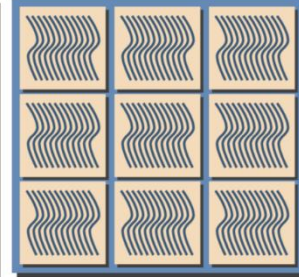
Shared Memory per SM

Kepler: up to 48K

Fermi: up to 48K



More blocks with less
shared memory per block



Fewer blocks with more
shared memory per block

Each SM has a set of 32-bit registers distributed among the threads

Each SM has a fix amount of shared memory distributed among the threads block

The number of thread blocks and warps that reside simultaneously on a SM for a given kernel depends on the number of registers and the amount of shared memory available on the SM and on those required by the kernel

RESOURCE CONSTRAINTS (Fermi/Kepler)



TECHNICAL SPECIFICATIONS	COMPUTE CAPABILITY			
	2.0	2.1	3.0	3.5
Maximum number of threads per block	1,024			
Maximum number of concurrent blocks per multiprocessor	8		16	
Maximum number of concurrent warps per multiprocessor	48		64	
Maximum number of concurrent threads per multiprocessor	1,536		2,048	
Number of 32-bit registers per multiprocessor	32 K		64 K	
Maximum number of 32-bit registers per thread	63		255	
Maximum amount of shared memory per multiprocessor	48 K			



TO MEASURE THE ACTIVE WARPS

OCCUPANCY = # active warps / maximum # of warps

```
$ nvprof --metrics achieved_occupancy ./Application
```

Invocations	Metric Name	Metric Description	Min	Max	Avg
Device "GeForce GT 650M (0)"					
Kernel: matrix_prod(float*, float*, float*)					
1	achieved_occupancy	Achieved Occupancy	0.879916	0.879916	0.879916

```
$ ./sumMatrix 32 32    sumMatrixOnGPU2D <<< (512,512), (32,32) >>> elapsed 60 ms
$ ./sumMatrix 32 16    sumMatrixOnGPU2D <<< (512,1024), (32,16) >>> elapsed 38 ms
$ ./sumMatrix 16 32    sumMatrixOnGPU2D <<< (1024,512), (16,32) >>> elapsed 51 ms
$ ./sumMatrix 16 16    sumMatrixOnGPU2D <<< (1024,1024),(16,16) >>> elapsed 46 ms
```

```
$ nvprof --metrics achieved_occupancy ./sumMatrix 32 32 sumMatrixOnGPU2D <<<(512,512), (32,32)>>> Achieved Occupancy
0.501071
$ nvprof --metrics achieved_occupancy ./sumMatrix 32 16 sumMatrixOnGPU2D <<<(512,1024), (32,16)>>> Achieved Occupancy
0.736900
$ nvprof --metrics achieved_occupancy ./sumMatrix 16 32 sumMatrixOnGPU2D <<<(1024,512), (16,32)>>> Achieved Occupancy
0.766037
$ nvprof --metrics achieved_occupancy ./sumMatrix 16 16 sumMatrixOnGPU2D <<<(1024,1024),(16,16)>>> Achieved
Occupancy 0.810691
```




WARP DIVERGENCE

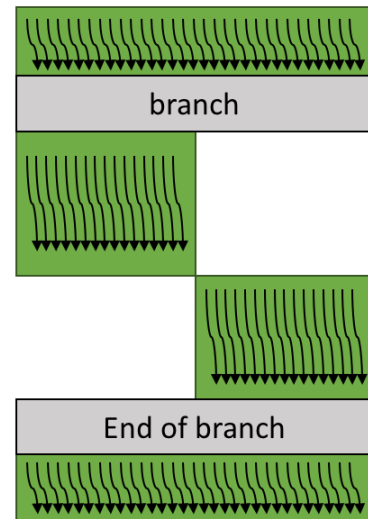
Typical CPUs include complex hardware mechanisms for branching prediction, i.e. predicting at each conditional step which branch the flow of control will be undertaken

If the prediction is not correct the CPU can get stuck for a high number of clock cycles until the pipeline is reset

GPUs are simpler devices without HW implementation of prediction branch strategies

Since (in principle) all threads in a warp must execute the same instruction in the same cycle, if a group of threads, within the same warp, takes a different path, then we are in the presence of a *warp divergence*

```
...  
if ( threadIdx.x < 16 )  
{  
    ... A ...  
}  
else  
{  
    ... B ...  
}  
...
```





WARP DIVERGENCE

```
__global__ void kernel(int* x, int* y) {  
  
    int i = threadIdx.x + blockDim.x * blockIdx.x;  
    int t;  
  
    bool b = f(x[i]);  
  
    if ( b ) {  
        // evaluate g(x)  
        t = g(x[i]);  
    }  
    else {  
        // evaluate h(x)  
        t = h(x[i]));  
    }  
    y[i] = t;  
}
```

If the threads of a warp diverge, the warp execute each branch path **serially**, disabling threads that do not belong to that given path

Divergence in warps introduces a significant degradation of performance ...

loss of parallelism up to 32 times!

Note that the divergence occurs only within a warp

The conditional steps in different warps do not cause divergence

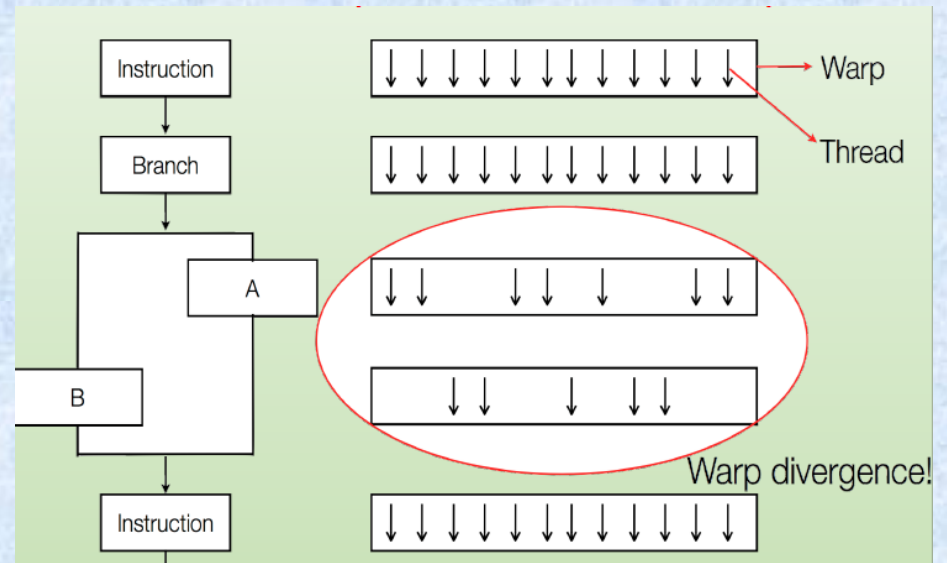


WARP DIVERGENCE

```
__global__ void dv(int* x) {  
    int i = threadIdx.x + blockDim.x * blockIdx.x;  
    switch (i % 32) {  
        case 0 :  
            x[i] = a(x[i]);  
            break;  
        case 1 :  
            x[i] = b(x[i]);  
            break;  
        ...  
        case 31:  
            x[i] = v(x[i]);  
            break;  
    }  
}
```

Even worse!

Parallelism gets worse 32 times!



OPTIMISATIONS



```
if (x < 0.0)
    z = x - 2.0;
else
    z = sqrt(x);
```

```
p = (x<0.0);
p: z = x-2.0;      // single instruction
!p: z = sqrt(x);
```

Predicates

p: c=a+b // computed only if p is true

All the threads compute the logic predicate and the two corresponding instructions but the results is depending on the exit of the test

SYNCHRONISATION

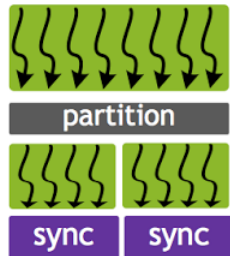


SYNCHRONIZE AT ANY SCALE

Three Key Capabilities

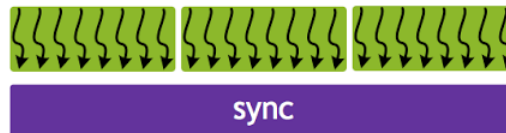
FLEXIBLE GROUPS

Define and Synchronize Arbitrary Groups of Threads

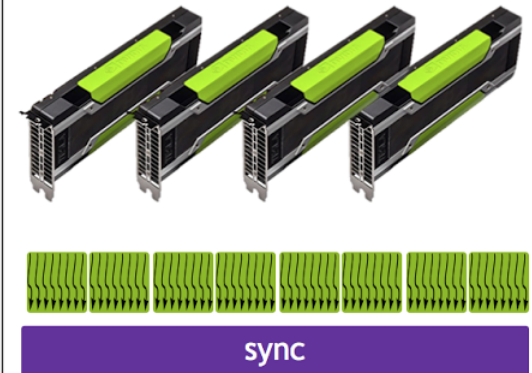


WHOLE-GRID SYNCHRONIZATION

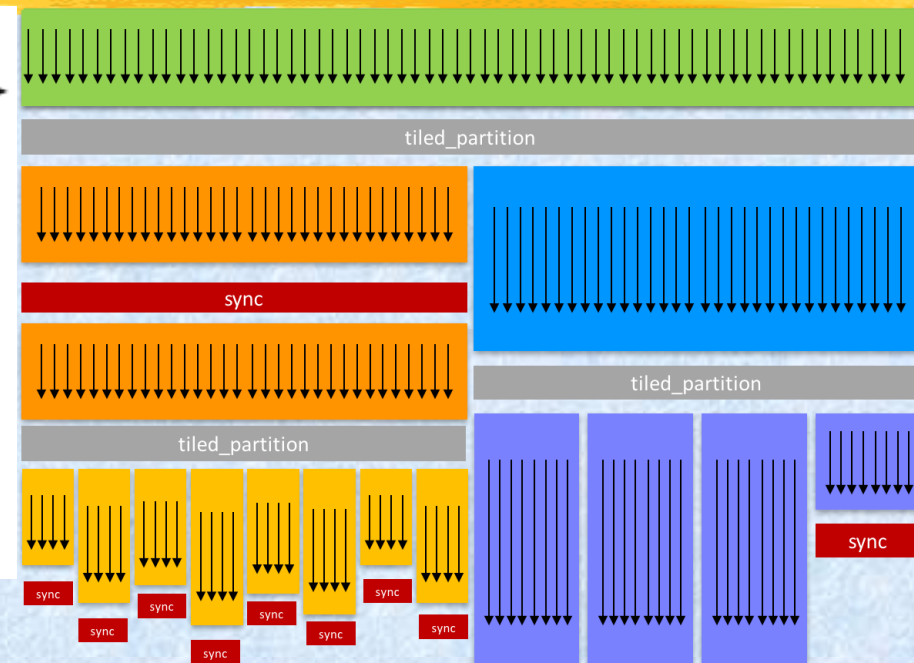
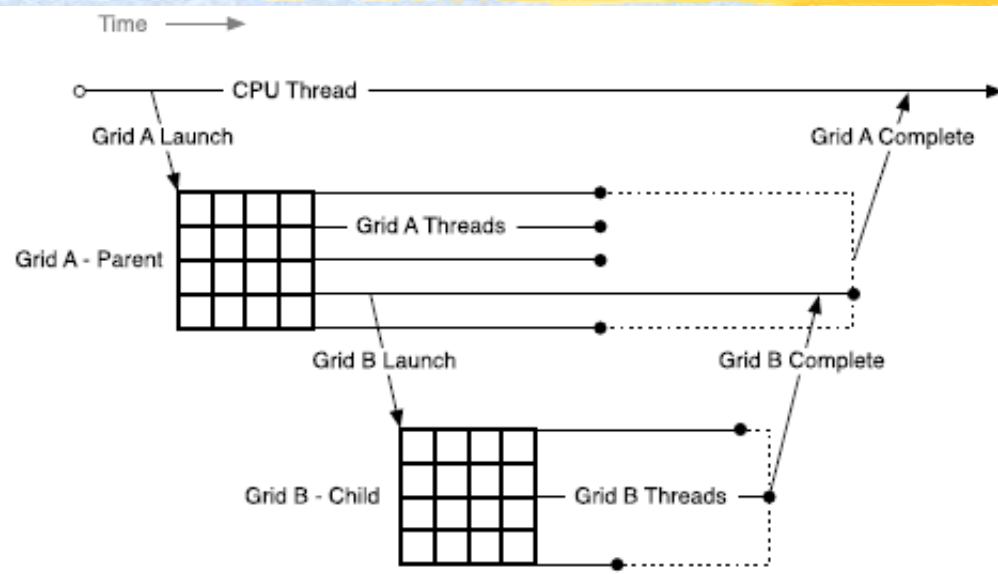
Synchronize Multiple Thread Blocks



MULTI-GPU SYNCHRONIZATION



SYNCHRONISATION AT BLOCK LEVEL



Synchronization barriers

If a barrier is present it must be executed by all threads of the block

Threads in different blocks cannot synchronize (blocks are independent)

Synchronization mechanisms prevent from possible race conditions i. e. different threads make access simultaneously to same memory locations with unpredictable behaviors

Pay attention to the *if-then-else* branch: thread on distinct branches cannot synchronize (infinite wait!)

SYNCHRONISATION



We typically use two levels of synchronization:

System level: when waiting that a task running both in the host and in the device is completed

`cudaError_t cudaDeviceSynchronize(void);`

It blocks the application running into the host up to the completion of all the CUDA tasks

Block level: when waiting that all the threads in a block reach the same execution point

`__device__ void __syncthreads(void);`

Synchronises all the threads within a block

DEADLOCK



```
//some device functions F1 & F2
if (threadIdx.x < 16) {
    F1();
    __syncthreads();
}

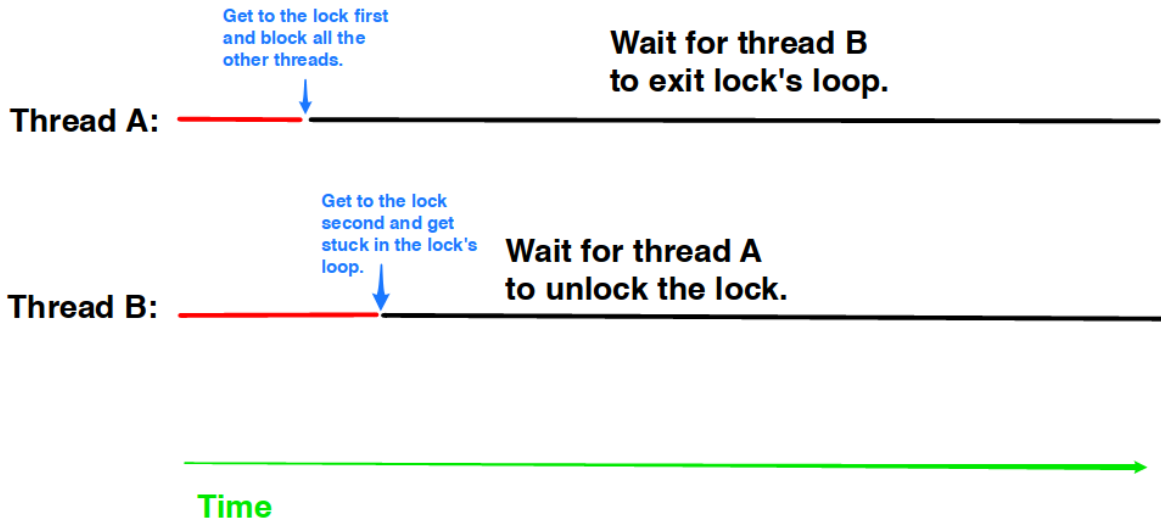
else if (threadIdx.x >= 16) {
    __syncthreads();
    F2();
}
```

Divergence can cause deadlock!

The first half of the threads in the warp executes F1 and then waits for the second one.

The second one waits the first one and will never execute F2

Threads in the same warp:



REDUCTION



A common operation called **reduction** is the sum of the elements of a large array like in:

- calculation of the average in general (Monte Carlo simulation, RMS, ...)
- computation of the inner product between vectors in linear algebra
- similar operations such as the calculation of the minimum and maximum

$$(x_1, x_2, \dots, x_n) \Rightarrow S = \sum_{i=1}^N X_i$$

The properties required for a reduction operator $\oplus$ are:

- **commutative** $a \oplus b = b \oplus a$
- **associative** $a \oplus (b \oplus c) = (a \oplus b) \oplus c$

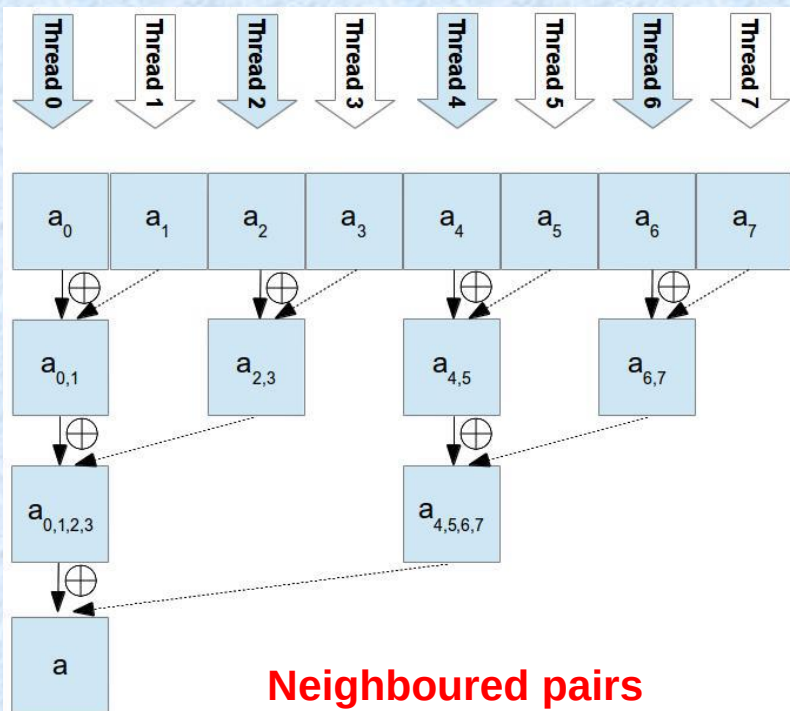
These two properties allow to reorder and/or combine elements in whatever way

PARALLEL REDUCTION



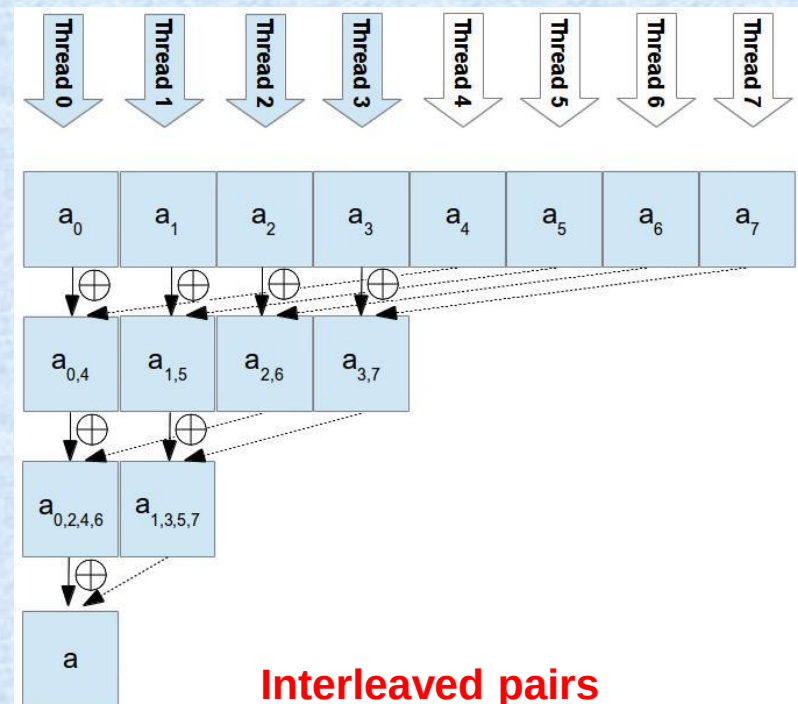
Sequential:

```
int sum = 0;
for (int i = 0; i < N; i++)
    sum += array[i];
```



Parallel:

- Partition the vector in smaller parts
- Assign each partial sum to a thread
- Add the partial results



Ex. RECURSIVE SUM OF A VECTOR



```
int recursiveReduce(int *data, int const size) {  
    // exit  
    if (size == 1)  
        return data[0];  
    // renew the stride  
    int const stride = size / 2;  
    // partial sum on half the vector  
    for (int i = 0; i < stride; i++)  
        data[i] += data[i + stride];  
    // recursive call  
    return recursiveReduce(data, stride);  
}
```

Parallel strategy:

- *At every step a number of threads equal to the half of the vector size computes the partial sums in the first half of the elements*
- *The number of the active threads must be halved every step (stride renewal)*
- *Threads must be synchronized at every step before going the next one*

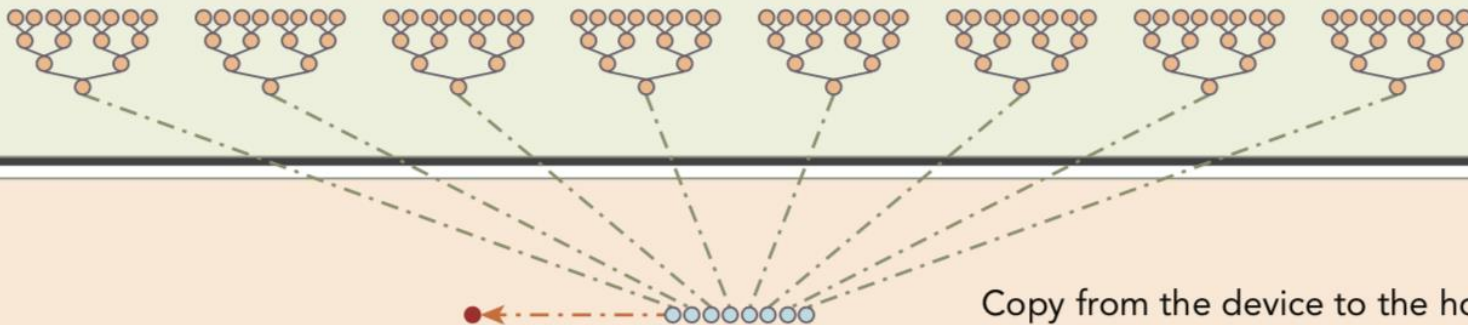
Ex. RECURSIVE SUM OF A VECTOR



Parallel strategy:

- *Local partial sum of the elements belonging to the grid blocks obtained from the parallel reduction*
- *Constraint: the size of a block should be necessarily a power of 2*
- *Drawback: an increasing **divergence** at the warp level*

Reduce at the device side with a large amount of blocks in parallel.



Reduce at the host side in sequentially.

REDUCING WITH NEIGHBOURED PAIRS



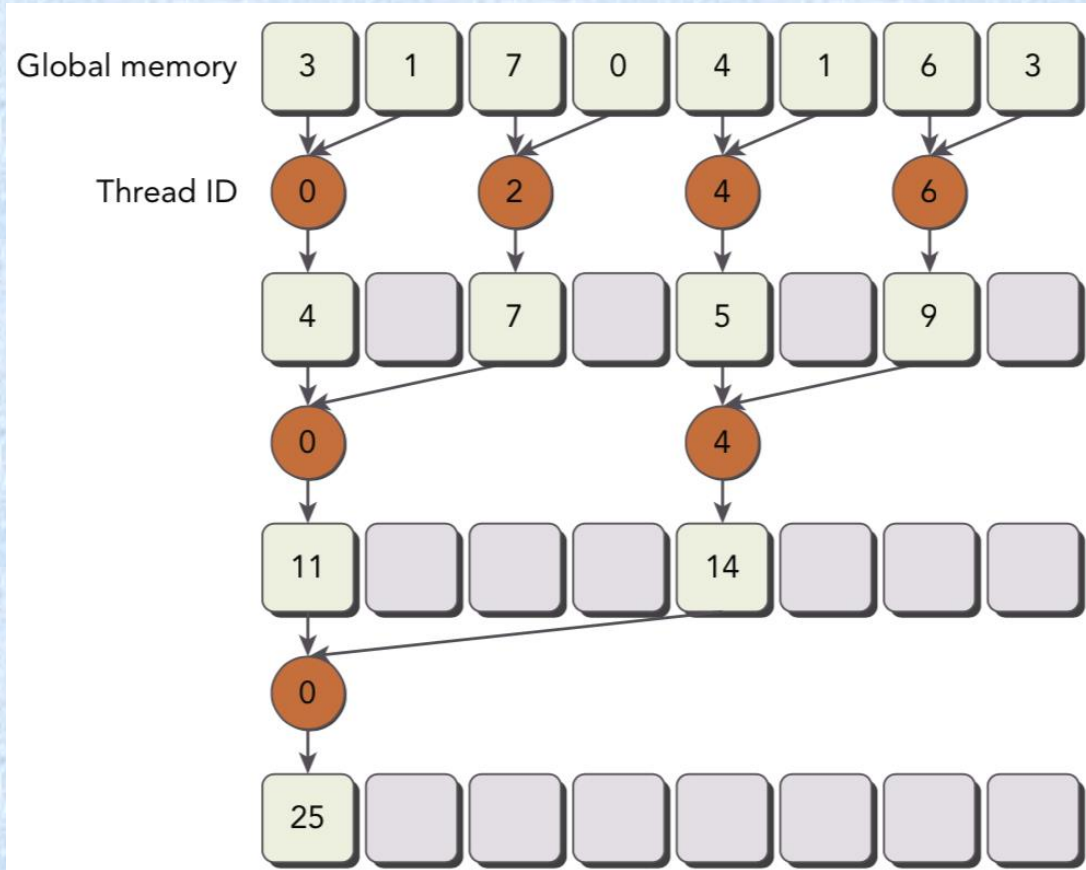
Implementing *neighbored pair approach*.

Use two global memory arrays: one large array to store the entire array to reduce, and one smaller array to hold the partial sums of each thread block.

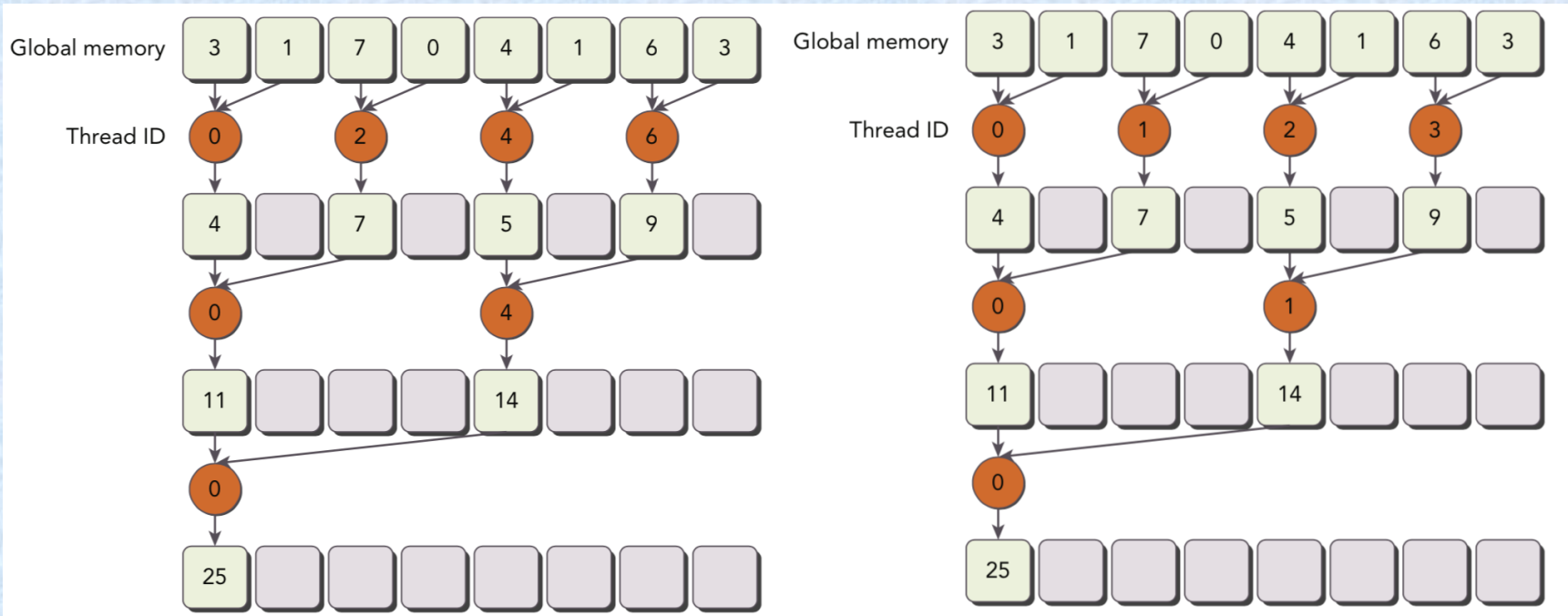
The reduction is *done in-place*, which means that the values in global memory are replaced by partial sums at each step.

__syncthreads ensures that all partial sums for every thread in the current iteration have been saved to global memory before any thread in the same thread block enters the next iteration.

Because there is no synchronization between thread blocks, the partial sum produced by each block is copied back to the host and summed sequentially there.



IMPROVEMENT



What's wrong with the above implementation?

if ($tid \% (stride * 2) == 0$) causes thread **divergence** (left in the figure). In the first iteration, 1/2 of the threads are active; in the second iteration, 1/4 of the threads are active; ...

Modification: (right in the figure) forcing the usage of neighborhood threads; the store location of partial sums has not changed, but the working threads are updated

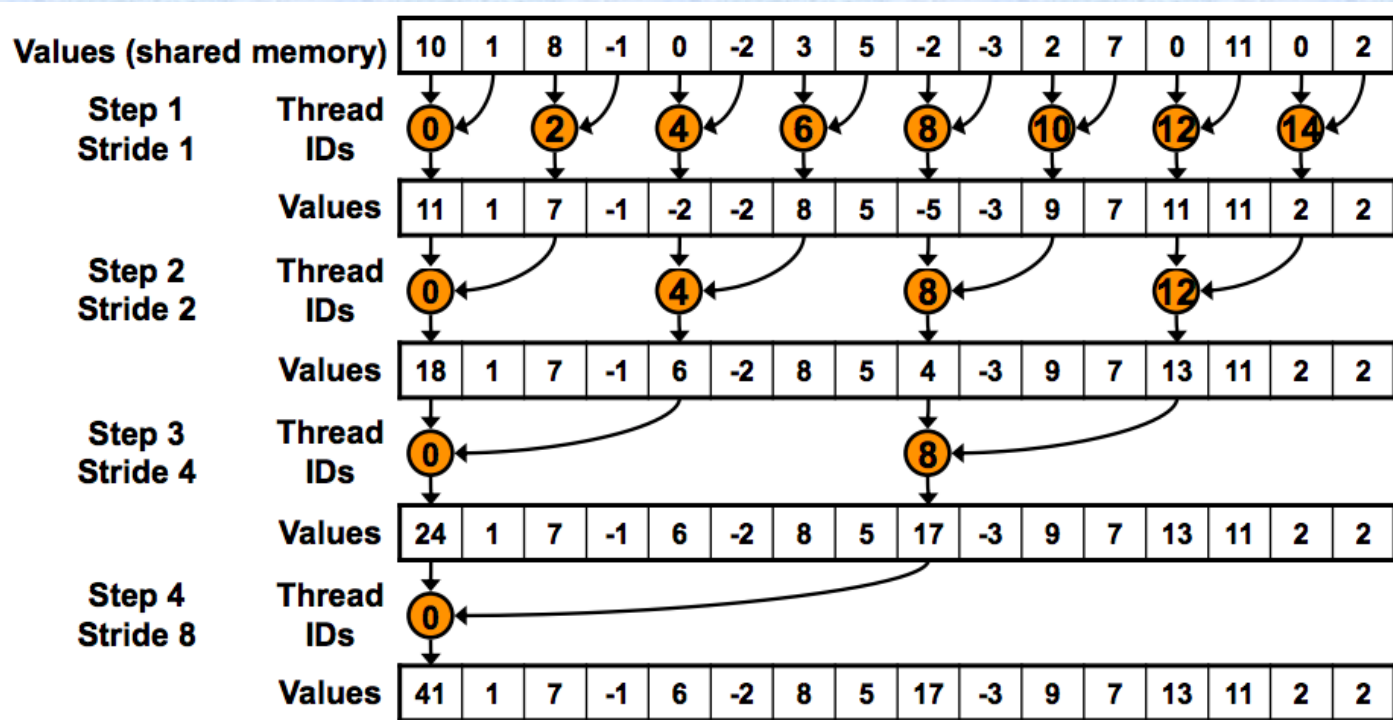
Ex. RECURSIVE SUM OF A VECTOR



/* Block by block parallel implementation with divergence */

```
__global__ void block_parallel_reduce(int *array_in, int *array_out, unsigned int n) {  
    unsigned int tid = threadIdx.x;  
    unsigned int idx = blockIdx.x * blockDim.x + threadIdx.x;  
    // boundary check  
    if (idx >= n) return;  
    // convert global data pointer to the local pointer of this block  
    int *array_off = array_in + blockIdx.x * blockDim.x;  
    // in-place reduction in global memory  
    for (int stride = 1; stride < blockDim.x; stride *= 2) {  
        if ((tid % (2 * stride)) == 0)  
            array_off[tid] += array_off[tid + stride];  
    // synchronize within thread block  
        __syncthreads();  
    }  
    // write result for this block to global mem  
    if (tid == 0) array_out[blockIdx.x] = array_off[0];  
}
```


REDUCTION 1



Naive parallel reduction



THE DIVERGENCE PROBLEM

```
for (int stride = 1; stride < blockDim.x; stride *= 2) {  
    if ((tid % (2 * stride)) == 0)  
        array_off[tid] += array_off[tid + stride];  
    // synchronize within threadblock  
    __syncthreads();  
}
```

- **If true only for even threads** this causes high divergence in the warps
- In the **first iteration** of the parallel reduction only **half** of the thread executes the if
- ... however all the threads (even and odd) are scheduled
- In the **second iteration** only **1/4** of the thread is active
- The % operator is quite slow

SOLUTION



```
// in-place reduction in global memory
for (int stride = 1; stride < blockDim.x; stride *= 2) {

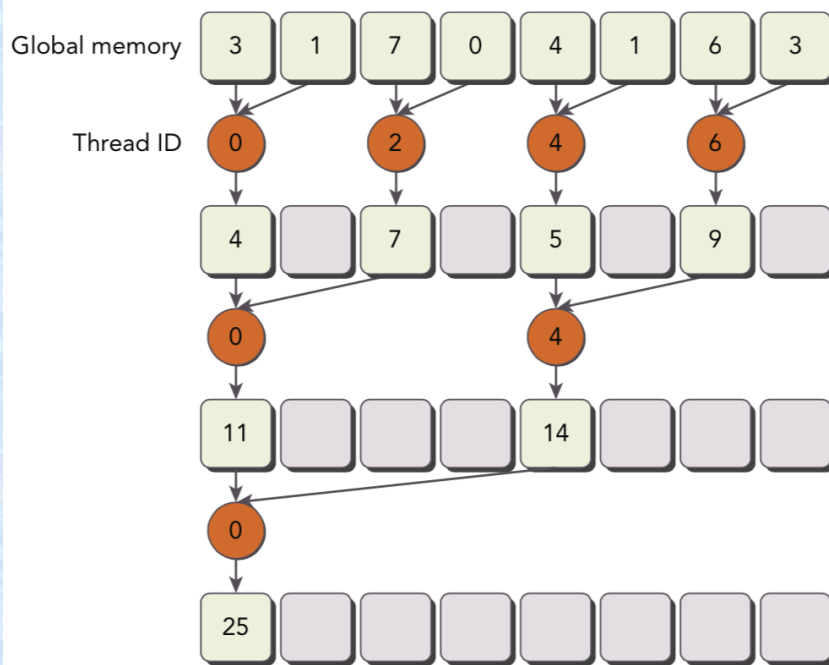
    // convert tid into local array index
    int index = 2 * stride * tid;

    if (index < blockDim.x)
        idata[index] += idata[index + stride];

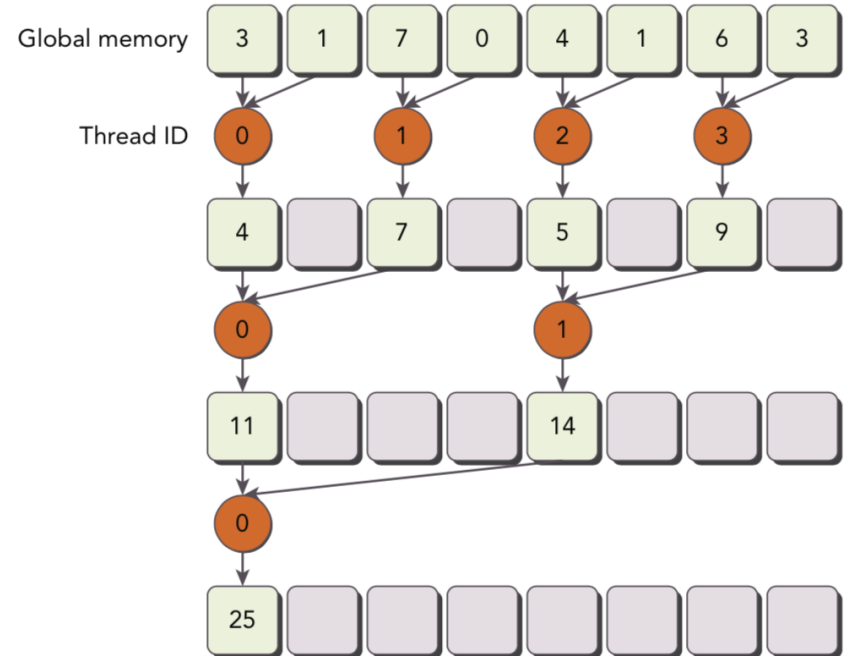
    // synchronize within threadblock
    __syncthreads();
}
```

- The if clause has been eliminated in all the threads
- The location of the partial sums is the same but the thread index is different

DIVERGENCE IN PARALLEL REDUCTION

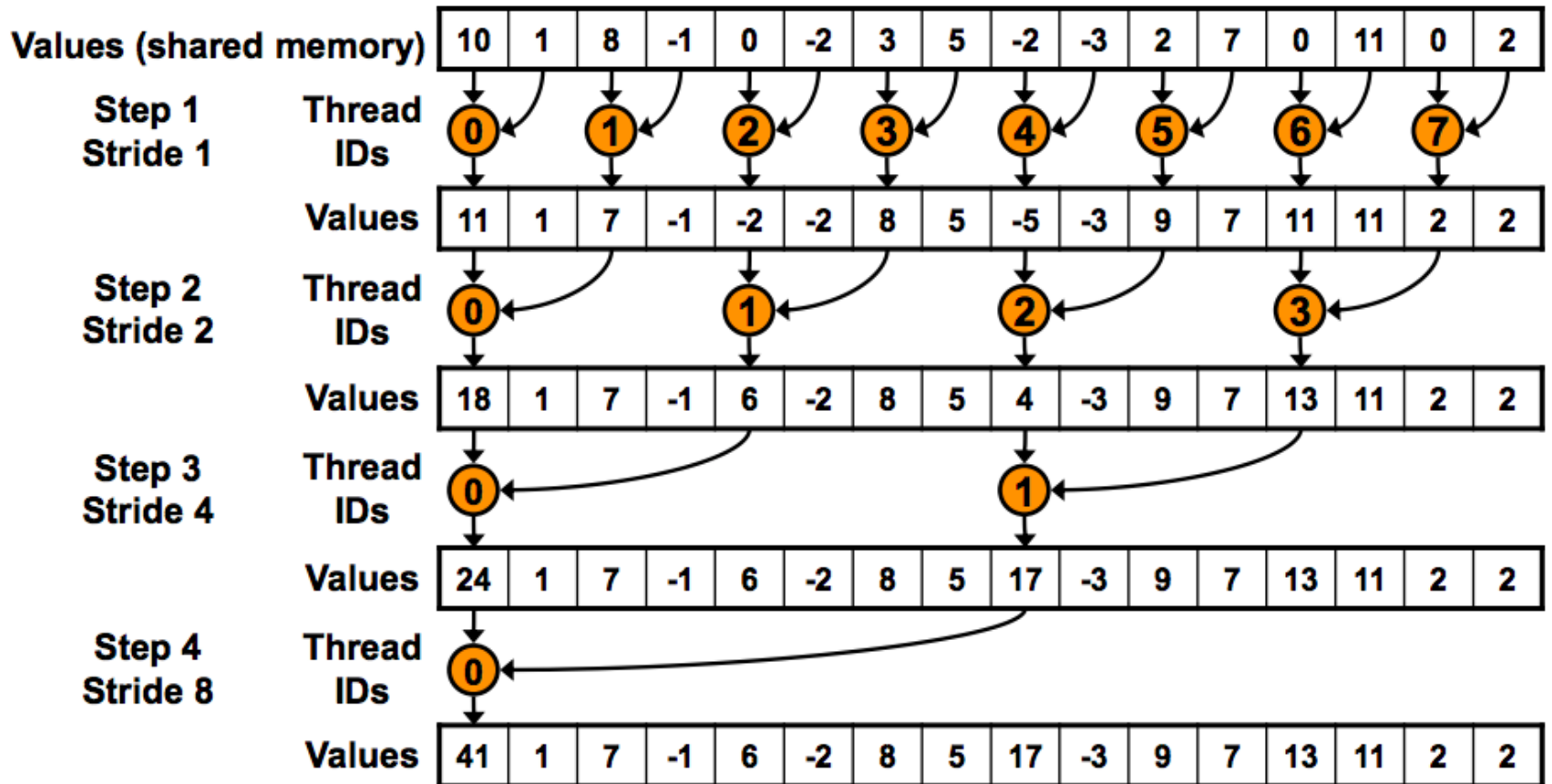


```
// in-place reduction in global memory
for (int stride = 1; stride < blockDim.x; stride *= 2) {
    if ((tid % (2 * stride)) == 0)
        idata[tid] += idata[tid + stride];
}
```



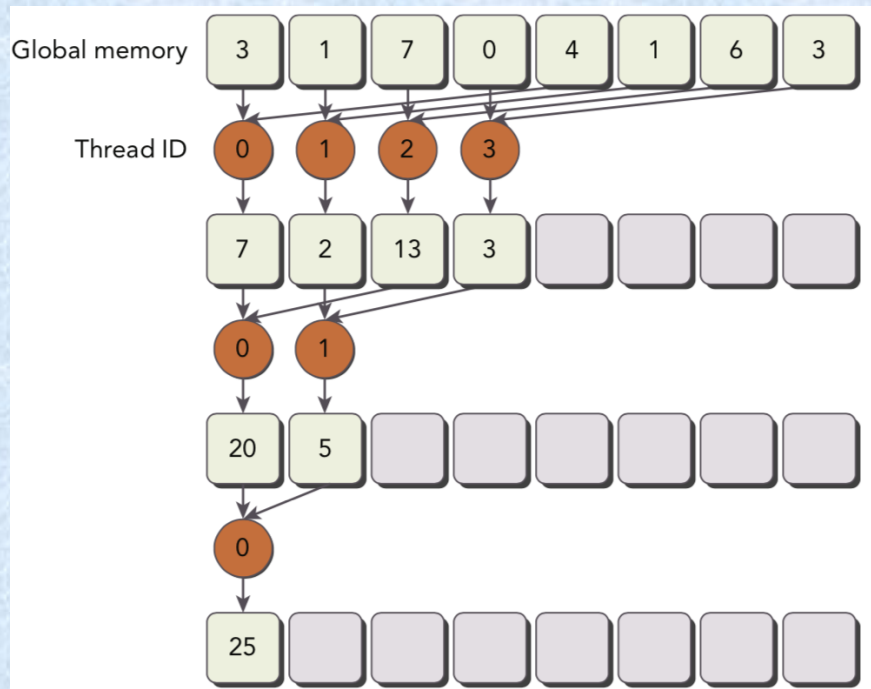
```
// in-place reduction in global memory
for (int stride = 1; stride < blockDim.x; stride *= 2) {
    // convert tid into local array index
    int index = 2 * stride * tid;
    if (index < blockDim.x)
        idata[index] += idata[index + stride];
}
```

REDUCTION 2



Changed threads performing reduction

INTERLEAVED SCHEME



same amount of warp divergence, but better performance due to the global memory load and store patterns

```
__global__ void reduceInterleaved(int *g_idata, int *g_odata,
const int n) {
```

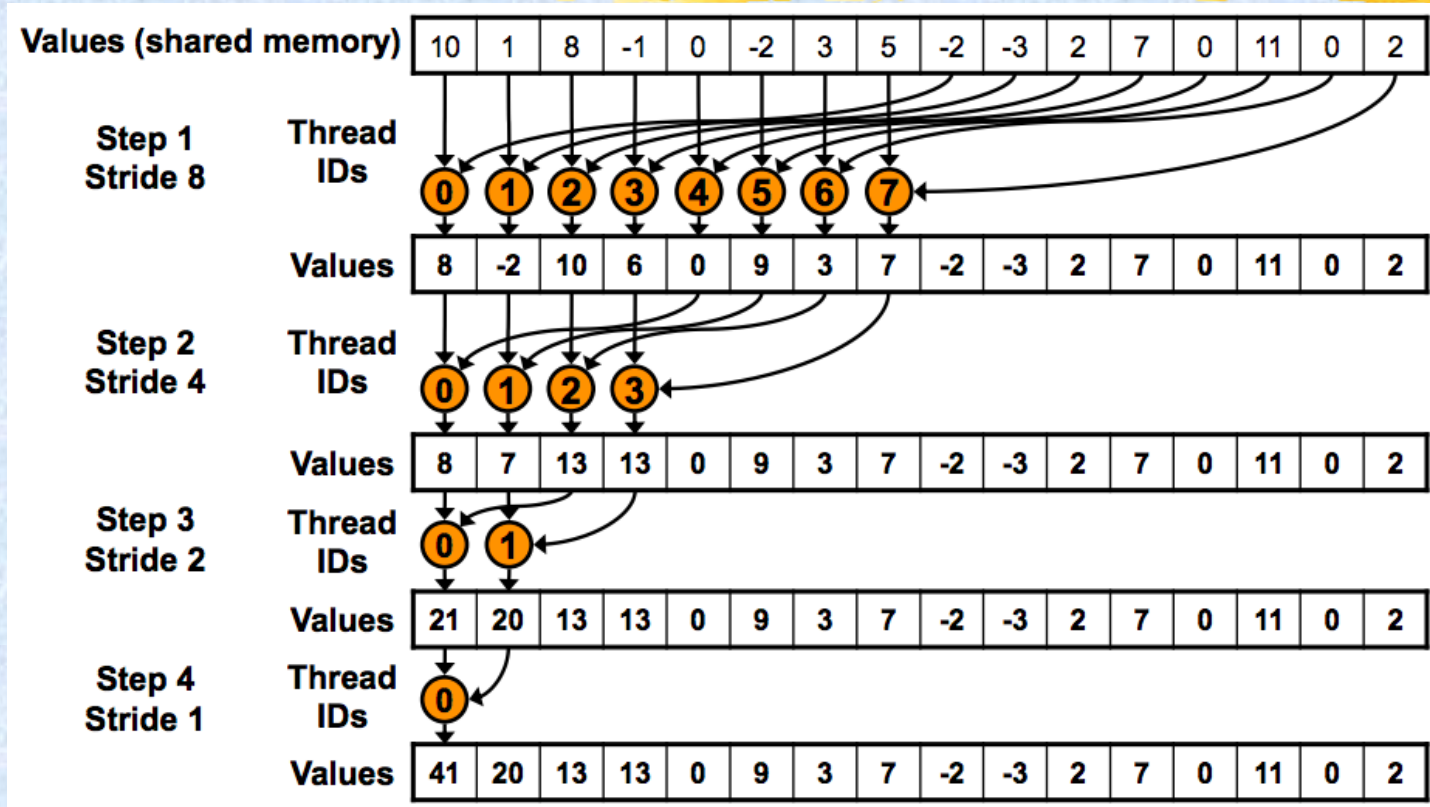
```
    const int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx >= n) return;
    const int tid = threadIdx.x;
```

```
    // convert global data pointer to the local pointer of this block
    int *idata = g_idata + blockIdx.x * blockDim.x;
```

```
    // in-place reduction in global memory
    for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
        if (tid < stride) {
            idata[tid] += idata[tid + stride];
        }
        __syncthreads();
    }
```

```
    if (tid == 0) g_odata[blockIdx.x] = idata[0];
}
```


REDUCTION 3



Accessing connected memory space.

A further optimisation performing the first reduction step when copying data to shared memory,
In any case we have half the number of threads that we needed in the previous reduced versions

LOOP UNROLLING



Loop unrolling is a technique that attempts to **optimize loop** execution by reducing the frequency of **branches** and loop maintenance **instructions**

Instead of writing the **body** of a loop **once** and using a loop to **execute** it **repeatedly**, the **body is written** in code **multiple times**

Any enclosing loop then either has its **iterations reduced** or is **removed entirely**

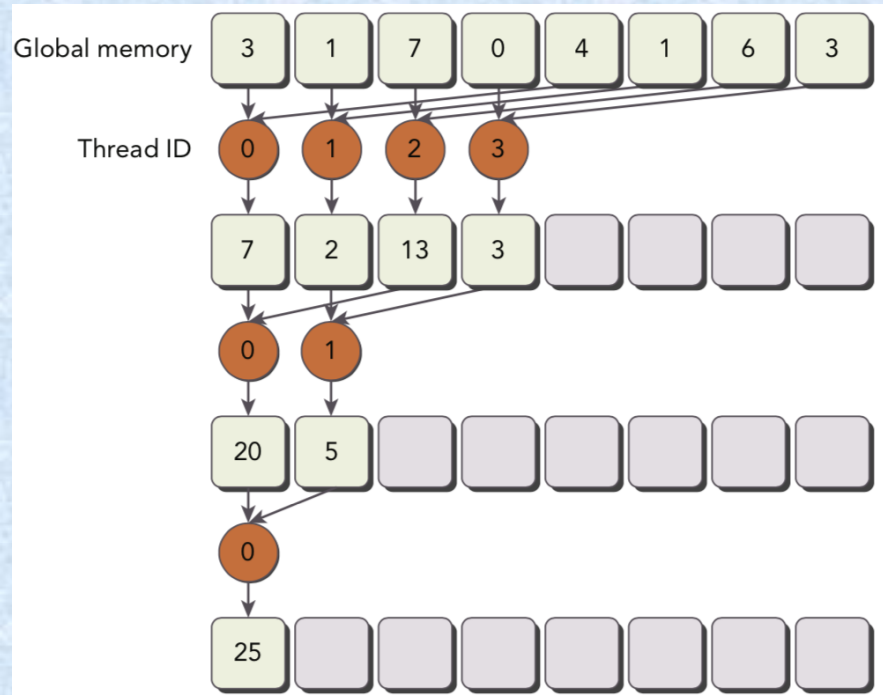
The **number** of **copies** made of the loop body is called the loop **unrolling factor**

Loop unrolling is **most effective** at improving performance for **sequential array processing** loops where the number of iterations is known before the execution of the loop

```
for (int i = 0; i < 100; i++) {  
    a[i] = b[i] + c[i];  
}
```

```
for (int i = 0; i < 100; i += 2) {  
    a[i] = b[i] + c[i];  
    a[i+1] = b[i+1] + c[i+1];  
}
```

LOOP UNROLLING



```
__global__ void blockParReduce2(int *in, int *out, ulong n) {
    uint tid = threadIdx.x;
    ulong idx = blockIdx.x * blockDim.x + threadIdx.x;

    // boundary check
    if (idx >= n) return;

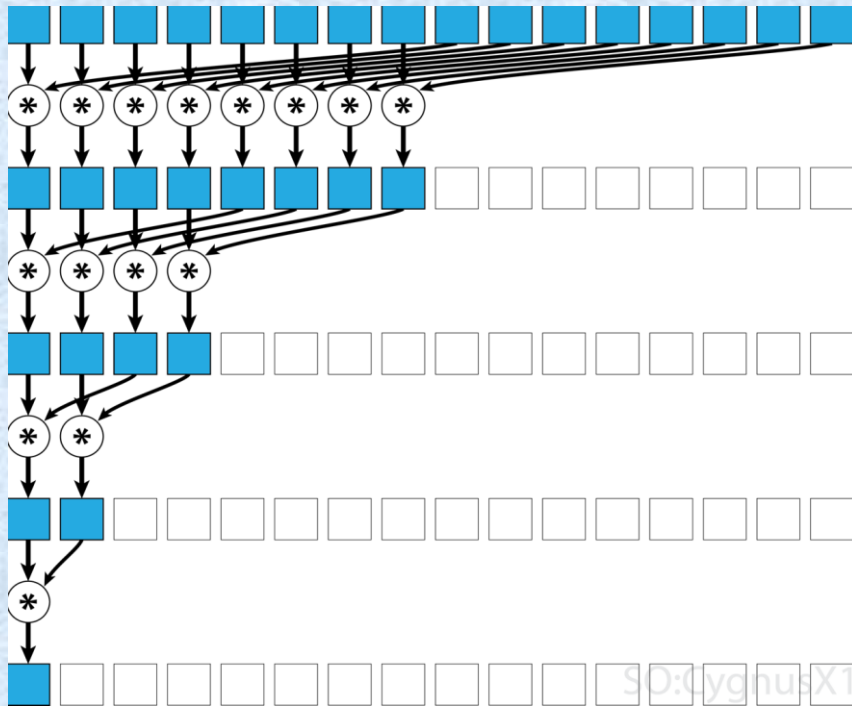
    // convert global data ptr to the local ptr of this block
    int *thisBlock = in + blockIdx.x * blockDim.x;

    // in-place reduction in global memory
    for (int stride = blockDim.x / 2; stride > 0; stride >>= 1) {
        if (tid < stride)
            thisBlock[tid] += thisBlock[tid + stride];

        // synchronize within threadblock
        __syncthreads();
    }

    // write result for this block to global mem
    if (tid == 0) out[blockIdx.x] = thisBlock[0];
}
```


LOOP UNROLLING



;

`vmem[tid] += vmem[tid + 8];`

`vmem[tid] += vmem[tid + 4];`

`vmem[tid] += vmem[tid + 2];`

`vmem[tid] += vmem[tid + 1];`

;

SO: CygnusX1



ATOMIC OPERATIONS

What happens if many threads run the **incr** kernel incrementing **temp** and updating ***ptr**?

We could use atomic operations that cannot be interrupted (locked)

In CUDA we have **atomic** operations (i. e. that cannot be interrupted by other threads) only for mathematical operations

```
__int atomicAdd(int *M, int V);
```

Main atomic operations:

- **Mathematical**: add, subtract, maximum, minimum, increment, and decrement
- **Bitwise**: AND, bitwise OR, bitwise XOR
- **Swap**: replace a memory value with a new one

```
__global__ void incr(int *ptr) {  
  
    int temp = *ptr;  
    temp = temp + 1;  
    *ptr = temp;  
  
}
```

```
__global__ void incr(int *ptr) {  
  
    int temp = atomicAdd(ptr, 1);  
  
}
```



ATOMIC OPERATIONS: examples

```
__global__ void testAtomic(int *g_odata) {
```

```
    // access thread id
```

```
    const unsigned int tid = blockDim.x * blockIdx.x + threadIdx.x;
```

```
    // Arithmetic atomic instructions
```

```
    atomicAdd(&g_odata[0], 10);
```

```
// Atomic addition
```

```
    atomicSub(&g_odata[1], 10);
```

```
// Atomic subtraction
```

```
    atomicMax(&g_odata[3], tid);
```

```
// Atomic maximum
```

```
    atomicMin(&g_odata[4], tid);
```

```
// Atomic minimum
```

```
    atomicInc((unsigned int *)&g_odata[5], 17);
```

```
// Atomic increment
```

```
    atomicDec((unsigned int *)&g_odata[6], 137);
```

```
// Atomic decrement
```

```
    // Arithmetic atomic instructions
```

```
    atomicExch(&g_odata[2], tid);
```

```
// Atomic exchange
```

```
    atomicCAS(&g_odata[7], tid-1, tid);
```

```
// Atomic compare-and-swap
```

```
    // Bitwise atomic instructions
```

```
    atomicAnd(&g_odata[8], 2*tid+7);
```

```
// Atomic AND
```

```
    atomicOr(&g_odata[9], 1 << tid);
```

```
// Atomic OR
```

```
    atomicXor(&g_odata[10], tid)
```

```
// Atomic XOR
```

```
}
```




SHARED MEMORY



SHARED MEMORY

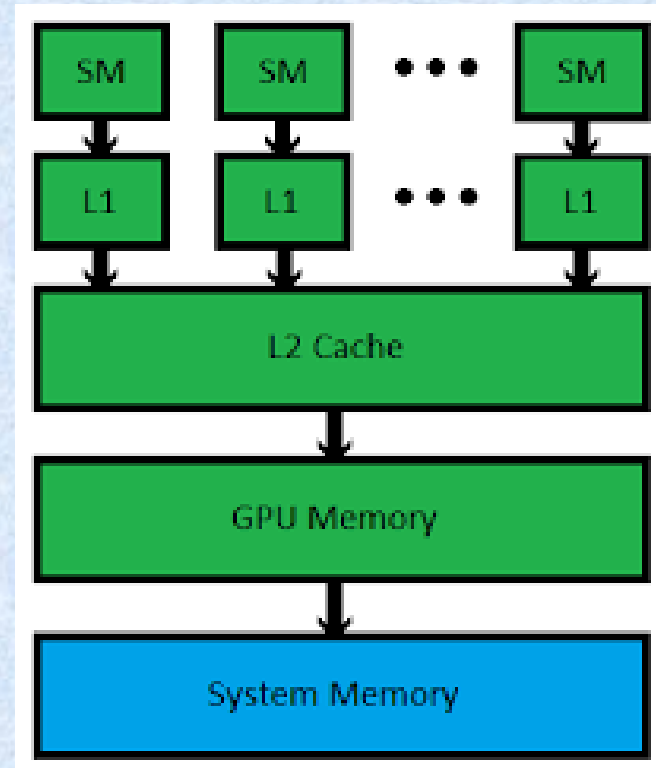
Memory is a key element in the design of **modern computer architectures**

Computations cannot be fast if data cannot be **moved** enough quickly

Large amount of memory is required for big data applications

Very fast (and large) memories are also very expensive

No alternatives if not a **hierarchical design**



SHARED MEMORY



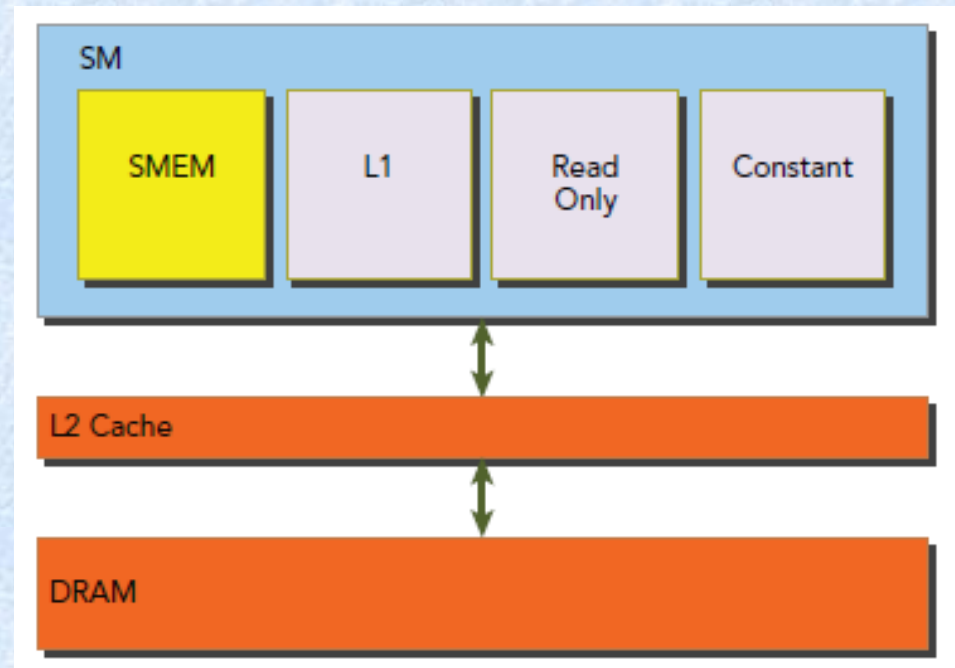
Communication channel for intrablock threads

The cache for the global memory can be managed directly by the programmer

SMEM it's a scratch-pad memory to process onchip-data and improve the access patterns to the global memory

Latency reduced roughly ~ 30 times compared to global memory

Greater Bandwidth ~ 10 times greater compared to global memory





SHARED MEMORY

On-chip programmable memory, therefore with much higher bandwidth and lower latency than the local or global memory

It is divided into modules with the same width, called **banks** (32)

Access requests made of n addresses corresponding to n different banks are served simultaneously

Each SM has a limited amount of shared memory which is split across blocks of threads

Be careful not to over-use it so as not to limit the number of active warps

Shared memory serves as the basis for inter-thread communication: threads in a block can cooperate by exchanging data stored in shared memory

The accesses must be synchronized by means of the `__syncthreads ()` runtime call

SMEM ORGANISATION



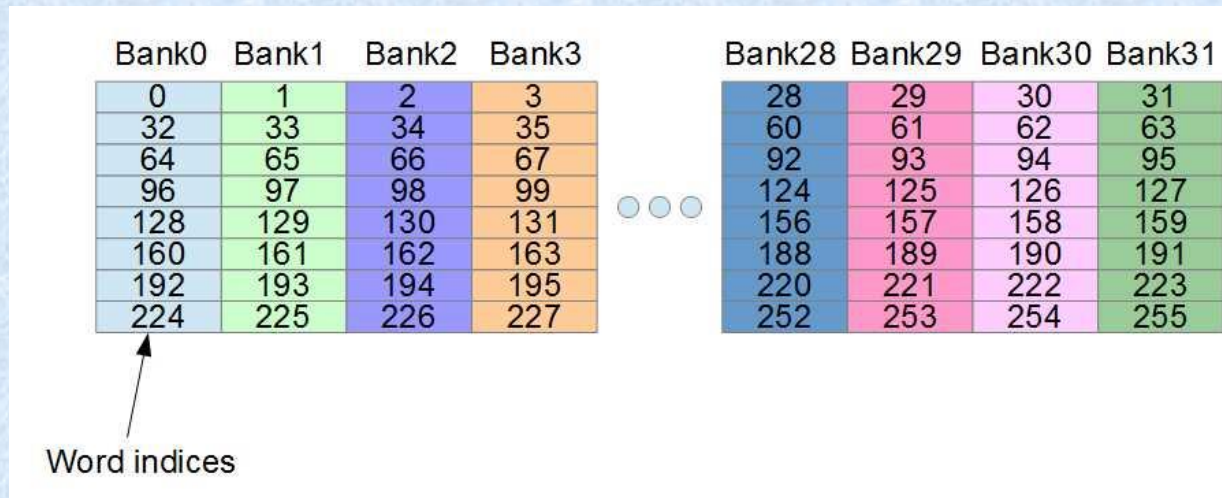
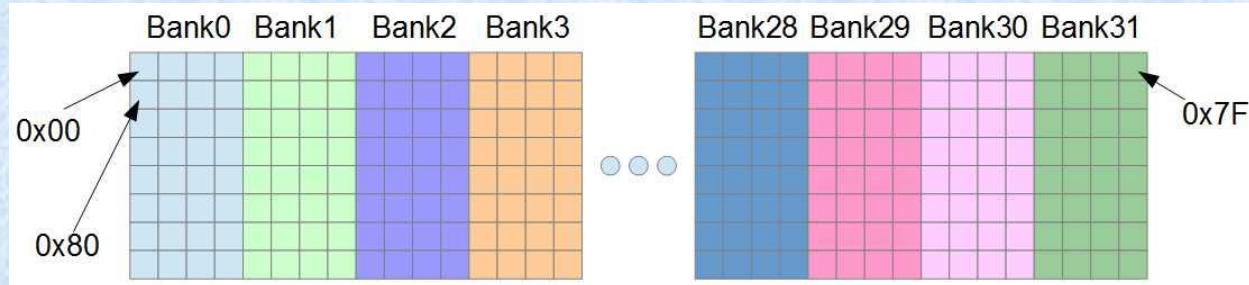
Example: the SMEM of Fermi arch consists of 4-byte blocks (called **words**) or 32-bit blocks

Each word can contain 1 integer, 1 float, 2 short, 4 char, 2 half float, ...

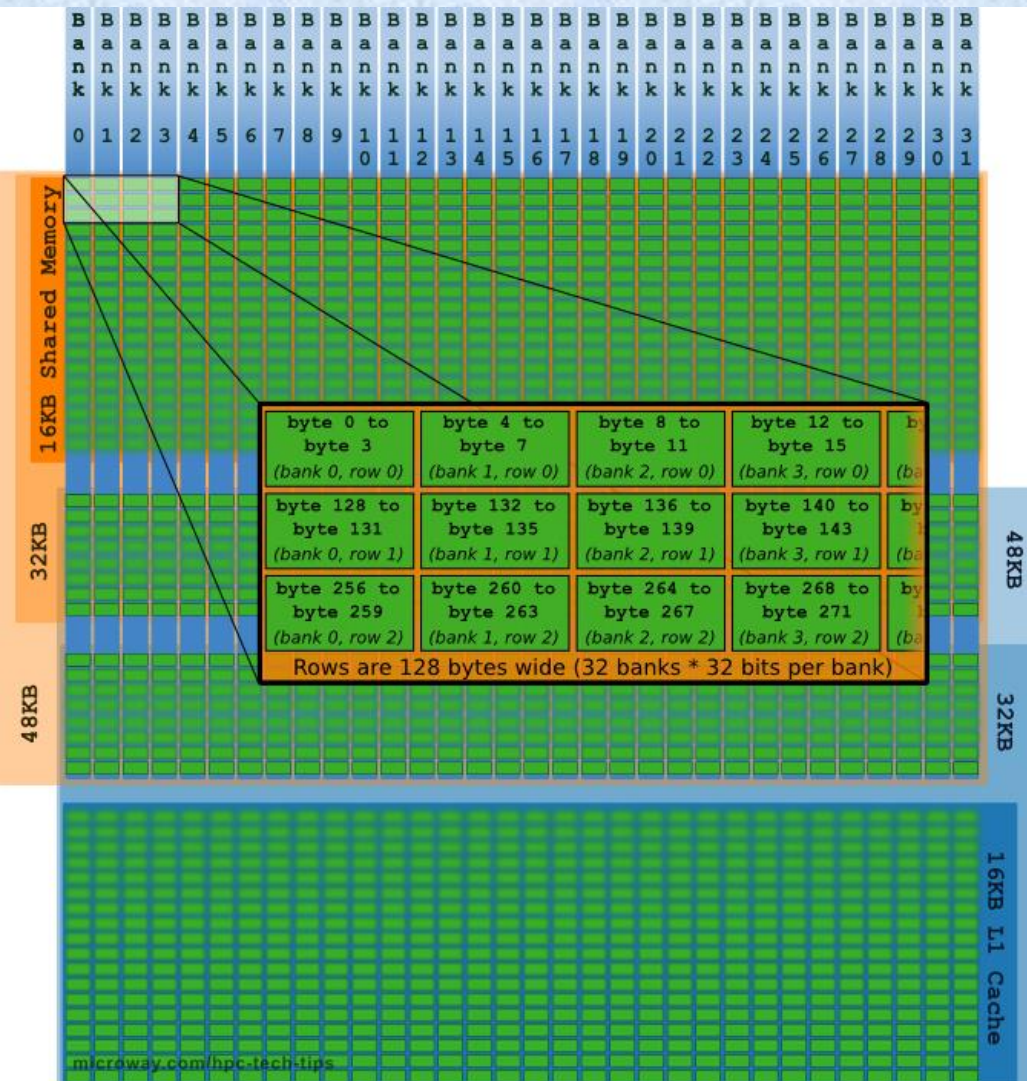
If a single byte is requested the entire word into which it is contained is read

The bandwidth is 32 values every 2 clock cycles

Given 32 banks, each word is stored in one of 32 different banks



SMEM ORGANISATION



A common problem arises when memory is shared: with all memory available to all threads, there will be many threads possibly accessing the data simultaneously

To alleviate this potential bottleneck, shared memory is divided into **32 logical banks**

Successive sections of memory are assigned to successive banks

If there are insufficient registers or shared memory on each SM to process at least one block, the kernel launch will fail



SMEM AT RUNTIME

SMEM is shared among all resident blocks in a SM (critical resource limiting the parallelism at device level)

The more shared memory a kernel requires, the fewer the number of concurrent active blocks

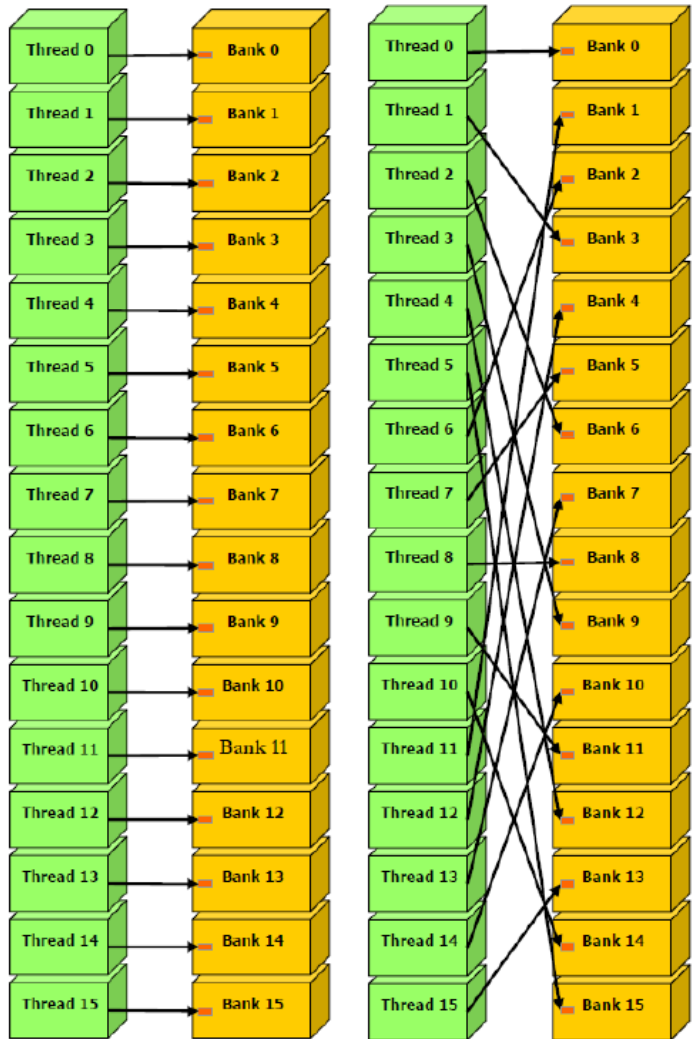
The content of the shared memory has the same lifetime as the block it was assigned

The access is *by warp*: best case 1 transaction x 32 threads, worst case 32 transactions

In the best case it serves 32 warp threads in 2 clock cycles!



SMEM CONFLICTS



No conflicts: a load/store operation executed by a warp requires at most a single access to a bank

Ex. `__shared__ float array[32];`

`float data = array[ BaseIndex + S*tid];`

The figure shows two examples with $S=1$ and $S=3$

Clearly each request can be made concurrently (in parallel). No Bank Conflict.

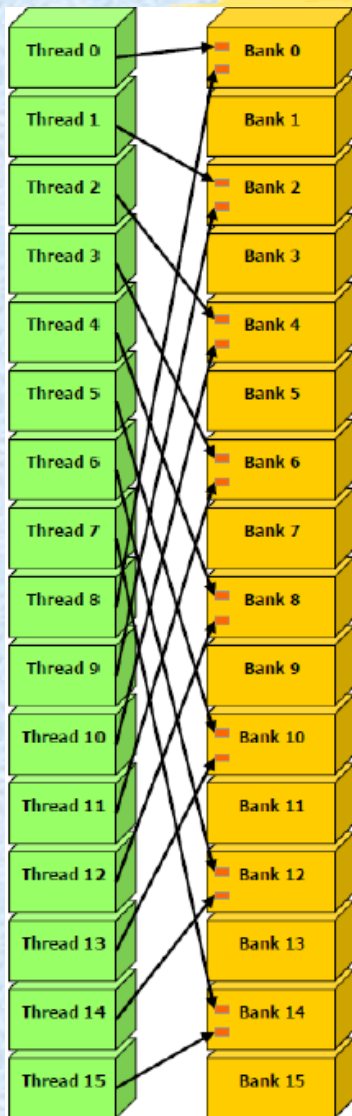
But why there is no bank conflict?

This is only bank-conflict-free if S shares no common factors with the number of banks.

Example: 16 banks, S must be odd!



SMEM CONFLICTS



Now let's declare

```
__shared__ double array1[32];           64 bit  
__shared__ short array2[32];           16 bit
```

and

```
double data1 = array1[BaseIndex + S*tid];  
short data2 = array2[BaseIndex + S*tid];
```

it is clear that two threads in a warp access a same bank memory location like thread 0 and thread 8 that access bank 0.

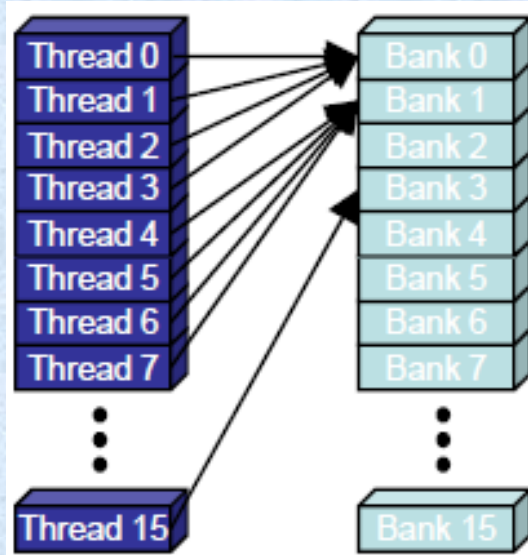
These requests will be serialized!

Known as **2-Way bank conflict!**


```
__shared__ double shared[32];  
or  
__shared__ short shared[32];
```



SMEM CONFLICTS



Now let's declare

```
__shared__ double array1[32];  
__shared__ short array2[32];
```

and

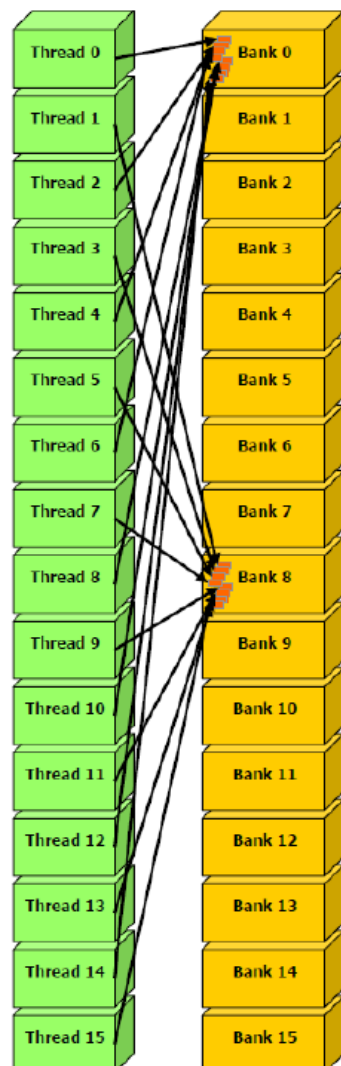
```
double data1 = array1[ BaseIndex + S*tid];  
short data2 = array2[ BaseIndex + S*tid];
```

it is clear that four threads in a warp access a same bank memory location like thread 0,1,2,3 access bank 0 and so on.

These requests will be serialized!

Known as **4-Way bank conflict!**

SMEM CONFLICTS



Now let's declare

```
struct vector { float x, y, z; };  
__shared__ struct vector vectors[64];
```

```
struct myType2 {float f; int c; };  
__shared__ struct myType2 myTypes2[64];
```

```
struct myType8 {float x,y,z,w; };  
__shared__ struct myType8 myTypes8[64];
```

Example 1: This has *no bank conflicts* for vector; struct size is 3 words
3 accesses per thread, contiguous banks (no common factor with 16)
Thread struct vector v = vectors [baseIndex + threadIdx.x];

Example 2: This has *2 way bank conflicts* for myType2;
(2 accesses per thread) (Have common factor with 16; 2)
struct myType2 m = myTypes4[baseIndex + threadIdx.x];

Example 3: This has *8 way bank conflicts* for myType8;
(8 accesses per thread) (Have common factor with 16; 8)
struct myType8 m = myTypes8[baseIndex + threadIdx.x];



SMEM/L1 CONFIG

Each SM has 64 KB of on-chip memory: shared memory and L1 cache **share** this hardware resource

CUDA provides two methods to configure L1 cache and shared memory:

```
cudaError_t cudaDeviceSetCacheConfig(cudaFuncCache cacheConfig);
```

cacheConfig values

- `cudaFuncCachePreferNone`: no preference(default)
- `cudaFuncCachePreferShared`: prefer 48KB shared memory and 16 KB L1 cache
- `cudaFuncCachePreferL1`: prefer 48KB L1 cache and 16 KB shared memory
- `cudaFuncCachePreferEqual`: prefer 32KB L1 cache and 32 KB shared memory

- If a lot of shared memory is used the 48 KB size is preferred
- From Kepler onwards, if many registers are used, the spilling of registers falls on the L1 cache
- The exact number of registers used by the kernel can be determined by specifying the `-Xptxas -v` option when compiling with `nvcc` and those available are used it is possible to increase the L1 cache
- It is also possible to configure runtime before running the kernel with:

```
cudaError_t cudaFuncSetCacheConfig(const void* func, cudaFuncCache cacheConfig);
```


OBSERVATIONS



Latency hiding: the delay between the requests made by the threads to the SMEM and the real data getting is not a problem even in the case of a bank conflict. If there are many threads running, the scheduler switches to another warp until those suspended complete the transfer of data from the SMEM (in this way the delay may be considered irrelevant)

Inter-block: there are no conflicts between threads belonging to different blocks, the problem exists only at the warp level of the same block

Maximum efficiency: the simplest way to maximize the performance is to organize warp accesses to consecutive words in shared memory

Caching: with an efficient scheduling, even in the presence of SMEM conflicts performance is far better than when data goes through the L2 cache or, worse, through the global memory

Clock(): this function can be used to measure the current number of clock cycles in the GPU due to the delay caused by SMEM conflicts



SM STATIC ALLOCATION

A variable in shared memory can also be declared both **local** to a kernel and **global** in a source file (global to all of the application kernels)

A variable in shared memory is declared with the qualifier **__shared__**

A variable in shared memory can be declared both **static** and **dynamic**

CUDA supports static declarations of 1D, 2D and 3D multi-dimensional arrays

It is important that the memory size is known at compile time

Examples of static allocation

```
__global__ void myKernel() {  
    __shared__ int i;  
    __shared__ float array_f[128];  
    __shared__ int array_i[10][10];  
    ...  
    array_i[a][b] = 1;  
    array_f[threadIdx.x] = 1.0;  
    ...  
}
```

Race condition: two or more threads modify the same shared variable ...

```
__global__ void myKernel() {  
    __shared__ int i;  
    i = threadIdx.x+1;  
    ...  
}
```



SM DYNAMIC ALLOCATION

If the size is not known, when compiling it is possible to declare a variable dimensionless with the **extern** keyword

- Only 1D arrays can be dynamically allocated!
- It can be both internal to the kernel (kernel scope) and external (file scope)
- Example of 1D array:

```
extern __shared__ int array[];
```

To dynamically allocate a shared memory variable a third argument must be included in the kernel call

```
kernel<<<grid, block, N * sizeof(int)>>>(...)
```


MULTIPLE DYNAMIC ALLOCATION



To allocate different arrays in the same memory area it is necessary to use explicitly the offset within a single global allocation for all the data

```
short array_s[128];  
float array_f[64];  
int array_i[256];
```

```
int main(void) {  
  
    int nBytes=128*sizeof(short)+64*sizeof(float)+256*sizeof(int);  
  
    kernel<<<1, 1, nBytes>>>();  
    cudaDeviceSynchronize();  
    return 0;  
}  
  
extern __shared__ float array[]; // dynamic shared memory  
__global__ void kernel() {  
    short* array0 = (short*)array; // offset inizio short  
    float* array1 = (float*)&array0[128]; // offset inizio fl  
    int* array2 = (int*)&array1[64]; // offset inizio int  
  
    for (int i = 0; i < 128; i++)  
        array0[i] = 'a';  
    for (int i = 0; i < 64; i++)  
        array1[i] = 1.0;  
    for (int i = 0; i < 256; i++)  
        array2[i] = 1;  
}
```



SHARED MEMORY USAGE

Typical use:

1. Upload data from device memory to shared memory
2. Synchronize the threads in the block after you finish copying each one execute on shared memory, (so that each thread can process certain data below)
3. Process the data in shared memory
4. Synchronize (if necessary) to make sure the shared memory contains updated results
5. Write the results from the device memory to the host memory



Projects for the exam

The development of a project is required to pass the exam.

The general requirements for a good project are:

1. **Report**: writing a concise report on
 - problem addressed,
 - adopted implementation strategies
 - obtained results in terms of speed up
2. **Coding**: writing a clear and well-documented code
3. **Testing**: a testing phase in which the correctness of the code is shown on meaningful benchmark instances
4. **Profiling**: performance analysis (speedup and profiling in general) derived from the parallel versus sequential algorithm



Projects for the exam

A list of plausible projects (in complexity and time to completion) is offered below.

However, it is possible to propose projects independently as long as they have the summary characteristics of those listed here.

The general requirements for a good project are:

1. SVM support vector machine
2. PCA Principal Component Analysis
3. NN Resnet18 network
4. K-means

Website to find the theory for the implementation of the serial algorithm:

[**towardsdatascience.com**](https://towardsdatascience.com)

LINEAR ALGEBRA and DATA ANALYSIS



LINEAR ALGEBRA

- Singular Value Decomposition or SVD (Singular Value Decomposition) for the factorization of a matrix based on the use of eigenvalues and eigenvectors (SVD)
- Calculation of the Moore-Penrose pseudo-inverse matrix for the least squares problem (Fast_Computation_Moore-Penrose)

DATA ANALYSIS

- Principal Component Analysis technique implementation to reduce data size
- K-means clustering algorithms



IMAGING, GRAPH COLORING, CRYPTOGRAPHY

IMAGE AND VIDEO PROCESSING

- Filters on images
- Histograms of Oriented Gradient (HOG)
- Multi-scale Local Binary Patterns
- Co-occurrence histograms of oriented gradients
- Gabor Filters as Texture Discriminator
- Scale-invariant feature transform

GRAPH COLORING

- A Comparison of Parallel Graph Coloring Algorithms
- Scalable Parallel Algorithms And Implementations For Large-scale Graph Analyses

CRYPTOGRAPHY

- Analysis of cryptographic keys of asymmetric systems via GPU
- Parallelization of algorithms based on hash functions for the generation of secure passwords
- Collision generator on SHA-3 in reduced rounds